



UNIVERSITAT
ROVIRA I VIRGILI



UNIVERSITAT^{DE}
BARCELONA



UNIVERSITAT POLITÈCNICA
DE CATALUNYA
BARCELONATECH

REINFORCEMENT LEARNING STRATEGIES FOR MINIMISING HEV EMISSIONS

NIKLAS LONG SCHIEFELBEIN

Thesis supervisor

CECILIO ANGULO BAHON (Department of Automatic Control)

Degree

Master's Degree in Artificial Intelligence

Master's thesis

School of Engineering

Universitat Rovira i Virgili (URV)

Faculty of Mathematics

Universitat de Barcelona (UB)

Barcelona School of Informatics (FIB)

Universitat Politècnica de Catalunya (UPC) - BarcelonaTech

Abstract

This thesis investigates reinforcement-learning-based control strategies for reducing tailpipe Nitrogen Oxides (NO_x) emissions in a light-duty hybrid diesel vehicle. The work addresses the multi-objective nature of hybrid-electric-vehicle energy management, where speed tracking, emission reduction, battery charge sustainment, and real-time feasibility must be balanced. During training, the controller interacts with a long short-term memory surrogate environment of the vehicle and emission behaviour instead of computationally expensive physical simulations.

The methodology follows a two-stage training and evaluation pipeline. First, Proximal Policy Optimisation (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC) are trained for speed tracking to establish stable baseline policies. Their hyperparameters are selected with Optuna and validated across ten random seeds per algorithm. Second, the selected checkpoints are fine-tuned with additional reward terms for NO_x reduction and charge sustainment. The resulting policies are evaluated on a synthetic staircase cycle, analysed through their engine operating points, and tested zero-shot on the WLTC against a rule-based benchmark controller.

The results show that all three algorithms learn speed tracking, but differ once emission and battery objectives are introduced. PPO provides the most robust multi-objective fine-tuning, SAC achieves the strongest emission reduction, and TD3 is limited by training instability. In the WLTC comparison, the best SAC seed reaches 80 mg/km tailpipe NO_x , compared with 147 mg/km for the rule-based controller, while also improving speed tracking and maintaining charge sustainment. This improvement comes at the cost of higher fuel consumption, highlighting the remaining trade-off between pollutant reduction and energy efficiency.

Overall, the thesis demonstrates that staged reinforcement learning can produce emission-aware hybrid-vehicle control policies in a surrogate environment and transfer meaningful behaviour to an unseen driving cycle. The results also underline the importance of seed-wise policy selection, explicit fuel objectives, and future validation in a real vehicle or high-fidelity simulation environment.

The full implementation is publicly available on GitHub.

Keywords: Hybrid Electric Vehicle (HEV), Energy Management System (EMS), Deep Reinforcement Learning (DRL), Multi-Objective Optimisation, Curriculum Learning, NO_x Emission Reduction, Charge Sustainment, Surrogate Models

Acknowledgements

I would like to express my sincere gratitude to everyone who contributed to this work.

First and foremost, I would like to thank Cecilio Angulo Bahón for welcoming me into this project, guiding my research, and helping me resolve the many questions I encountered along the way. I gratefully acknowledge the INIREC grant, which supported this work and enabled me to dedicate the necessary time and effort to it. I also thank Markus Frey, who was always available for questions and whose feedback gave me confidence in my work, and Marcial Francisco Mosqueda Otero for his excellent work in making the surrogate models more efficient.

I am grateful for the opportunity to have contributed to a sustainability-focused project as my first substantial contribution to the field of AI.

Finally, I would like to thank my wonderful friends in Berlin and Barcelona for always being there for me and making my life as beautiful as it is. A special appreciation goes to Sheena, my loyal library partner.

Contents

Abstract	i
Acknowledgements	ii
List of Figures	vi
List of Tables	viii
List of Abbreviations	ix
List of Symbols	xi
1 Introduction	1
1.1 Problem Description	1
1.2 Motivation	3
1.3 Objectives	4
1.4 Contributions	4
1.5 Thesis Organisation	5
2 Theoretical Background	6
2.1 Hybrid Electric Vehicles	6
2.1.1 Classifying HEVs	7
2.1.2 Control Strategies for HEVs	8
2.1.3 Simulation of HEVs	9
2.1.4 The Complexity of the Thermal State	10
2.2 Artificial Neural Networks	10
2.2.1 Long Short-Term Memory	10
2.3 Reinforcement Learning	12
2.3.1 Markov Decision Process	13
2.3.2 Deep Reinforcement Learning	13
2.3.3 Classifying RL Algorithms	14
2.3.4 Overview of Actor–Critic Architectures	15
2.3.5 Proximal Policy Optimisation (PPO)	17
2.3.6 Twin Delayed Deep Deterministic Policy Gradient (TD3)	17
2.3.7 Soft Actor-Critic (SAC)	19

3	Related Work	22
3.1	Previous Light-Duty CI-ICE 2027 Progress	22
3.1.1	Initial Modelling and Base Control Strategies	22
3.1.2	Model Predictive Control	23
3.1.3	Recurrent Deep Reinforcement Learning	23
3.1.4	Current Limitations and Open Challenges	24
3.2	State of the Art – Reinforcement-Learning-Based Hybrid-Electric-Vehicle Control . .	24
3.2.1	Continuous Control with Actor-Critic Architectures	24
3.2.2	Multi-Objective Staged Course Learning	25
3.2.3	Domain Knowledge Integration	25
4	Methodology	26
4.1	Overview — The Control Loop	26
4.2	Reinforcement Learning Problem Formulation	27
4.2.1	Markov Decision Process Framing	27
4.2.2	Observation Design	27
4.2.3	Action Design	28
4.2.4	Reward Design	28
4.2.5	Selection of Reinforcement-Learning Algorithms	29
4.3	Multi-Stage Course Learning Approach	29
4.3.1	Terminology and Motivation	29
4.3.2	Phase 1 — Exclusive Speed Tracking	30
4.3.3	Phase 2 — Multi-Objective Optimisation	31
4.4	Evaluation Methodology	32
5	Implementation	34
5.1	The Specific Vehicle LSTMs	34
5.2	Computational Efficiency and Deployment	36
5.3	The Reinforcement-Learning Environment	38
5.4	The Reinforcement-Learning Algorithms	42
5.5	Optuna Hyperparameter Optimisation	43
5.6	Training Infrastructure	43
5.7	Per-Phase Implementation Pipelines	44
5.7.1	Phase 1 Pipeline	44
5.7.2	Phase 2 Pipeline	44
6	Results	48
6.1	Phase 1	48
6.1.1	PPO	49
6.1.2	TD3	52
6.1.3	SAC	55
6.1.4	Summary	58
6.2	Phase 2	62
6.2.1	PPO	63
6.2.2	TD3	66
6.2.3	SAC	68
6.2.4	Summary	70

6.3	Generalisation Evaluation on the WLTC	76
6.3.1	Population-Level Picture (10 seeds)	76
6.3.2	Best-Seed Comparison	76
7	Discussion	81
7.1	Findings and Takeaways	81
7.1.1	Overall Outcome	81
7.1.2	Mechanistic and Algorithmic Insights	82
7.1.3	Policy Selection and WLTC Generalisation	83
7.2	Limitations of the Study	83
7.2.1	Scope and Generalisability	83
7.2.2	Experimental Design and Training Procedure	84
7.2.3	Reward and Control Formulation	85
7.2.4	Scope of the Control Objective	85
7.3	Sustainability Analysis	86
7.3.1	Environmental Dimension	86
7.3.2	Economic Dimension	87
7.3.3	Social Dimension	88
7.3.4	Ethical Implications	89
7.3.5	Relationship with the Sustainable Development Goals	90
8	Conclusion	91
8.1	Summary	91
8.2	Future Work	92
8.3	Final Remarks	92
A	Neural Network Interfaces	93
A.1	ICE - Inputs and Outputs	93
A.2	Drivetrain - Inputs and Outputs	94
A.3	Drivetrain Plus - Inputs and Outputs	94
B	Phase 1 - Hyperparameter Optimisation Details	95
B.1	Phase 1 - Optuna Search Spaces	96
B.2	Phase 1 - Optuna Trial Histories	96
B.3	Phase 1 - Best Hyperparameter Sets	98
C	Phase 2 - Reward Weight Optimisation Details	99
C.1	Phase 2 - Reward-Weight Trial Histories	99

List of Figures

1.1	Evolution of EMSs	3
2.1	Hybrid powertrain configurations and planetary gearbox	8
2.2	Topology of HEV control strategies	9
2.3	Feedforward neural network	11
2.4	LSTM cell structure	11
2.5	Agent–environment interaction	12
2.6	Taxonomy of RL algorithms	15
2.7	Actor–critic architecture	17
2.8	PPO pseudocode	18
2.9	TD3 pseudocode	20
2.10	SAC pseudocode	21
4.1	Closed-loop surrogate-environment interaction	27
4.2	Overall experimental pipeline of the study	30
4.3	Six-phase target-velocity trajectory	33
5.1	Cascaded surrogate LSTM architecture	35
5.2	WLTC speed up benchmark	37
5.3	Inference-speedup comparison	38
5.4	Gaussian speed-tracking reward term	40
5.5	Technical pipeline of Phase 1	45
5.6	Technical pipeline of Phase 2	47
6.1	PPO seed training in Phase 1	50
6.2	PPO speed tracking in Phase 1	51
6.3	Best PPO seed action summary in Phase 1	51
6.4	Best PPO seed evaluation results in Phase 1	52
6.5	TD3 seed training in Phase 1	53
6.6	TD3 speed tracking in Phase 1	53
6.7	Best TD3 seed action summary in Phase 1	54
6.8	Best TD3 seed evaluation results in Phase 1	55
6.9	SAC seed training in Phase 1	56
6.10	SAC speed tracking in Phase 1	56
6.11	Best SAC seed action summary in Phase 1	57

6.12	Best SAC seed evaluation results in Phase 1	58
6.13	Phase 1 training-progress comparison	59
6.14	Speed-tracking accuracy versus NOx across all Phase 1 seeds	60
6.15	Per-seed evaluation metrics in parallel coordinates	60
6.16	Learned engine operating points over the measured EA189 BSFC map	61
6.17	PPO seed training in Phase 2	63
6.18	PPO speed tracking in Phase 2	64
6.19	Best PPO seed action summary in Phase 2	65
6.20	Best PPO seed evaluation results in Phase 2	66
6.21	TD3 seed training in Phase 2	67
6.22	TD3 speed tracking in Phase 2	67
6.23	SAC seed training in Phase 2	68
6.24	SAC speed tracking in Phase 2	68
6.25	Best SAC seed action summary in Phase 2	69
6.26	Best SAC seed evaluation results in Phase 2	70
6.27	Phase 2 training-progress comparison	71
6.28	NO _x reduction across the training pipeline	72
6.29	Phase-1 vs Phase-2 Pareto picture coloured by charge drift	73
6.30	Per-seed Phase-2 evaluation metrics in parallel coordinates	73
6.31	NO _x rate versus engine torque	74
6.32	Phase-1/Phase-2 engine-load distributions	74
6.33	Phase-1 vs Phase-2 operating points over the measured NOx map	75
6.34	Phase-1 vs Phase-2 operating points over the measured BSFC map	75
6.35	WLTC speed-tracking comparison	77
6.36	WLTC speed-NOx Pareto comparison	77
6.37	WLTC phase-wise NOx comparison	78
6.38	Best SAC seed action summary on WLTC	79
6.39	Best SAC seed evaluation results on WLTC	80
B.1	PPO Optuna trials in Phase 1	97
B.2	TD3 Optuna trials in Phase 1	97
B.3	SAC Optuna trials in Phase 1	98
C.1	PPO reward-weight Optuna trials in Phase 2	99
C.2	TD3 reward-weight Optuna trials in Phase 2	100
C.3	SAC reward-weight Optuna trials in Phase 2	100

List of Tables

5.1	Surrogate LSTM architectures and hyperparameters	35
5.2	ICE model valid ranges	36
5.3	Drivetrain model valid ranges	36
5.4	Observation-vector components and bounds	39
5.5	Action-vector components and ranges	39
5.6	Reward weights per training phase	41
6.1	Phase 1 evaluation summary	49
6.2	Phase 1 best-seed comparison	61
6.3	Best Phase 2 Optuna trials	62
6.4	Phase 2 evaluation summary	63
6.5	Phase 1 to Phase 2 headline changes	71
6.6	WLTC 10-seed mean comparison	76
6.7	WLTC best-seed comparison	78
A.1	ICE neural-network inputs and outputs	93
A.2	Drivetrain neural-network inputs and outputs	94
A.3	Drivetrain Plus neural-network inputs and outputs	94
B.1	Phase-1 Optuna search spaces	96
B.2	Best Phase-1 hyperparameter configurations	98

List of Abbreviations and Acronyms

ACC	Adaptive Cruise Control
AI	Artificial Intelligence
ANN	Artificial Neural Network
BSFC	Brake-Specific Fuel Consumption
DP	Dynamic Programming
DDPG	Deep Deterministic Policy Gradient
DRL	Deep Reinforcement Learning
DOC	Diesel Oxidation Catalyst
DPF	Diesel Particulate Filter
EAT	Exhaust Gas Aftertreatment
ECMS	Equivalent Consumption Minimisation Strategy
EM	Electric Motor
EM1	Electric Motor 1
EM2	Electric Motor 2
EMS	Energy Management System
FVV	Forschungsvereinigung Verbrennungskraftmaschinen
HEV	Hybrid Electric Vehicle
HPO	Hyperparameter Optimisation
ICE	Internal Combustion Engine
IDEAI	Intelligent Data Science and Artificial Intelligence

IFS	Institute of Automotive Engineering Stuttgart
LDCI2027	Light-Duty CI-ICE 2027
LSTM	Long Short-Term Memory
MAE	Mean Absolute Error
MDP	Markov Decision Process
MLP	Multilayer Perceptron
MPC	Model Predictive Control
NN	Neural Network
NO_x	Nitrogen Oxides
POMDP	Partially Observable Markov Decision Process
PPO	Proximal Policy Optimisation
RL	Reinforcement Learning
RMSE	Root Mean Square Error
RNN	Recurrent Neural Network
SAC	Soft Actor-Critic
SCR	Selective Catalytic Reduction
SGD	Stochastic Gradient Descent
SOC	State of Charge
TD3	Twin Delayed Deep Deterministic Policy Gradient
TPE	Tree-structured Parzen Estimator
TRPO	Trust Region Policy Optimisation
TU	Technische Universität
UPC	Universitat Politècnica de Catalunya
WLTC	Worldwide Harmonised Light Vehicles Test Cycle

List of Symbols

Symbol	Description	Unit	Context
v_{target}	Reference vehicle velocity	km/h	RL observation
v_{Car}	Actual vehicle velocity	km/h	Drivetrain output
n_{ICE}	ICE speed	rpm	Control input / model input
m_{fuel}	Fuel mass	mg/cyl/cycle	Control input / ICE input
τ_{ICE}	ICE torque	Nm	ICE output / drivetrain input
τ_{EM2}	EM2 torque	Nm	Control input / drivetrain input
Brakemech	Mechanical brake command	%	Control input / drivetrain input
SoC	Battery state of charge	–	Drivetrain output / RL observation
$\dot{m}_{\text{NO}_x, \text{tp}}$	Tailpipe NOx mass-flow rate	g/s	ICE output / RL observation
$M_{\text{NO}_x, \text{tp}}^{\text{eval}}$	Cumulative tailpipe NOx mass over an evaluation cycle	g	Evaluation metric / Phase 2 objective
T_{amb}	Ambient temperature	K	ICE boundary input
p_{amb}	Ambient pressure	bar	ICE boundary input

Chapter 1

Introduction

More than ever, the transition towards increasingly sustainable technological systems has become one of the central engineering challenges of today. Global warming and the continued accumulation of greenhouse-gas (GHG) emissions require to reconsider how energy is produced, converted, and consumed. In this context, the transport sector plays a particularly important role since it accounts for almost one third of the European Union’s total GHG emissions in 2023 [1]. Improving the sustainability of transport is therefore not only a technological opportunity, but also an environmental responsibility.

While passenger cars are responsible for around 72% of Europe’s transport activity, measured in passenger kilometres, conventional combustion-engine vehicles still represented a market share of 31% in early 2026 and therefore remain a key target for improvement [1], [2]. At the same time, the transition towards alternative propulsion systems is already reshaping the automotive landscape. Fully electric vehicles provide one important pathway, but Hybrid Electric Vehicles (HEVs) offer a particularly relevant intermediate solution because they combine the flexibility and range of conventional powertrains with the efficiency benefits of electrification [3], [4]. In particular, hybrid architectures create new opportunities to reduce fuel consumption and emissions through more efficient coordination of the engine, electric machines, battery, and the aftertreatment system.

This thesis is situated within that transition. It focuses on the control of hybrid electric vehicles, where the main challenge is no longer only the engineering of the hardware itself, but also the design of operating strategies that exploit the system’s complexity in an efficient and robust manner. The highly non-linear system dynamics motivate the investigation of Artificial Intelligence (AI) based methods, such as Reinforcement Learning (RL), as a framework for developing control policies while targeting emission reduction, efficiency, and drivability [5], [6].

1.1 Problem Description

Since HEVs are already established on the market, efficient control strategies for such vehicles already exist. However, these conventional approaches exhibit several limitations that motivate the investigation of RL as an alternative control strategy.

Real-time execution versus global optimality. A fundamental problem in HEV control is the trade-off between achieving the best possible fuel economy and being able to execute the control

software in real time. Global optimisation methods, such as Dynamic Programming (DP), can compute the mathematical optimum for fuel efficiency, but they require complete prior knowledge of the driving cycle in advance. This requirement, together with their high computational burden, makes DP unsuitable for real-time control policies [3], [7]. RL addresses this limitation by combining the global optimisation perspective of DP with the real-time execution speed of standard controllers [8]. Provided that training is successful, RL can learn a parameterised control strategy offline and then use one-step predictions to execute near-optimal decisions in real time, with computational times averaging less than a millisecond per action [9].

Calibration and heuristic imprecision. Currently, the automotive industry relies mainly on rule-based Energy Management Systems (EMSs) because they are simple, understandable, and reliable [10], [11]. These control strategies rely on fixed thresholds and the subjective experience of engineers [12], [13]. As a result, rules must be calibrated meticulously for the specific hardware configuration and driving conditions [14]. RL addresses this rigidity through its self-learning and trial-and-error capabilities [15]. By directly interacting with the environment, RL can discover complex relationships autonomously and formulate policies without manual tuning or predefined heuristic rules.

Dependency on physical models and future forecasts. Other advanced real-time control strategies, such as Model Predictive Control (MPC) and the Equivalent Consumption Minimisation Strategy (ECMS), struggle with environmental uncertainties. ECMS requires extensive tuning of an “equivalence factor” to adapt to varying conditions, while MPC depends heavily on sufficiently accurate predictions of future conditions within its prediction horizon, which are generally not available in real-world scenarios [3], [16]. Model-free RL addresses this issue as a data-driven approach that does not require an explicit physical model or prior knowledge of the driving route in order to make decisions. Instead, it leverages its experiences to learn robust control strategies offline in advance.

Multi-objective conflicts. The multi-objective nature of HEV control makes all of the previously mentioned challenges considerably harder, because several competing objectives must be balanced simultaneously. Minimising fuel consumption, reducing tailpipe emissions, maintaining the battery’s State of Charge (SOC), and preserving component durability together create a highly complex and high-dimensional optimisation problem. This leads to the curse of dimensionality: as the number of relevant states, actions, and objectives increases, the control problem becomes more difficult to represent, calibrate, and optimise. Rule-based strategies become increasingly challenging to tune, local optimisation methods such as ECMS or MPC struggle to balance all objectives consistently under changing conditions, global optimisation methods such as DP become computationally unfeasible, and tabular RL requires discretisation of continuous variables, which introduces discretisation errors [3], [16], [17], [18].

Taken together, these considerations suggest that the difficulty in HEV control is not a single isolated weakness of existing methods, but the need to address all of these challenges simultaneously: real-time feasibility, calibration effort, uncertainty, and multi-objective complexity must be handled jointly if hybrid control strategies are to realise their full potential.

1.2 Motivation

The motivation of this thesis follows directly from the limitations outlined above. If conventional HEV control strategies struggle because real-time feasibility, calibration effort, uncertainty, and multi-objective complexity must be handled simultaneously, then Deep Reinforcement Learning (DRL) appears, at least in theory, to offer a suitable alternative. By combining data-driven learning with neural-network function approximation, DRL offers the potential to handle nonlinear dynamics, reduce reliance on manual calibration, avoid explicit physical modelling requirements, and address high-dimensional multi-objective optimisation problems more effectively than the previously discussed alternatives [15].

This broader methodological shift is reflected in the historical evolution of EMSs, where classical rule-based and optimisation-based approaches have increasingly been complemented by learning-based strategies. As shown in Figure 1.1, RL-based methods became a visible part of the EMS research landscape towards the end of the 2010s. The figure therefore positions this thesis within a broader development in HEV control rather than as an isolated application of DRL.

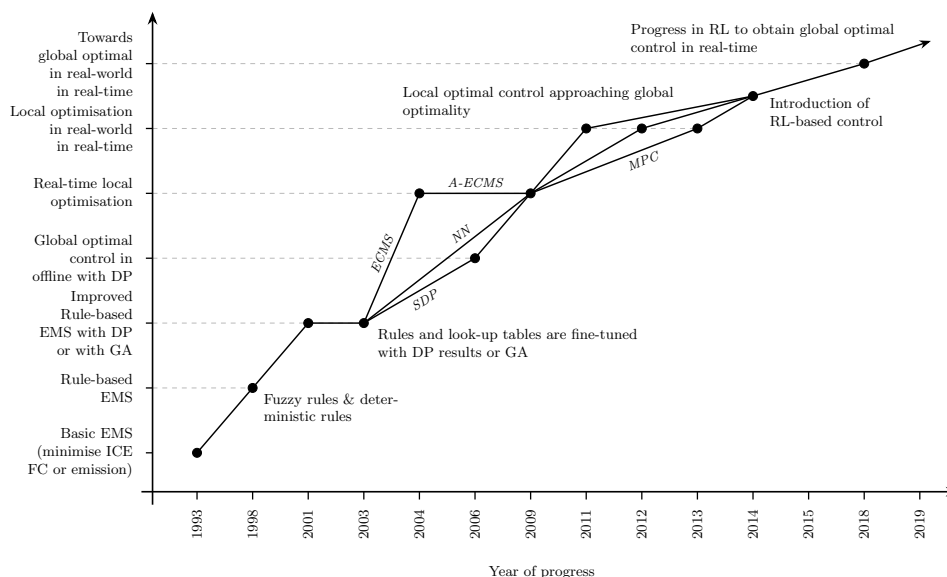


Figure 1.1: Tentative evolution of EMSs from 1993 to 2018; adapted from [19].

It is this promise of learning-based EMS design that initiated the broader research project of which this thesis forms a part.

This work is a standalone master’s thesis, but it is also embedded in the activities of the IDEAI (Intelligent Data Science and Artificial Intelligence (IDEAI)) research centre at the UPC (Universitat Politècnica de Catalunya (UPC)) within the CORNET’s funding project [20] Light-Duty CI-ICE 2027 (LDCI2027). The project brings together several institutions with complementary roles, including the German research association for combustion engines Forschungsvereinigung Verbrennungskraftmaschinen (FVV) e.V. [21], the UPC [22], the Institute of Automotive Engineering Stuttgart (IFS) [23], and the Technische Universität (TU) Berlin [24].

Within this collaboration, each participant has a clearly defined responsibility: TU Berlin is

responsible for the engine test bench in order to obtain real measurement data from sensor systems; IFS is responsible for modelling the engine and the power-split architecture; and the UPC is responsible for the control strategy. The common objective of the project is to reduce Nitrogen Oxides (NO_x) emissions so that light-duty hybrid vehicles can comply with the emission regulations imposed by the EURO guidelines [25], [26]. In this sense, the present thesis should be understood as one concrete contribution to a larger research effort that investigates whether DRL-based control can become a viable method for HEV operation.

At the same time, this thesis continues earlier work on Exhaust Gas Aftertreatment (EAT) before the turbine of a diesel engine and builds on previous achievements within the IDEAI research centre [27], [28]. The broader methodological direction therefore remains consistent: the development of intelligent control systems and suitable hybrid-electric-vehicle models using concepts from Artificial Intelligence (AI), Artificial Neural Networks (ANNs), and RL.

1.3 Objectives

This thesis pursues the following objectives and scope:

- **This thesis is** a proof-of-concept study of DRL-based control for the considered hybrid electric vehicle.
- **This thesis is** an assessment of the general feasibility of DRL for the given control task.
- **This thesis is** a comparative study of selected RL algorithms, including their strengths, weaknesses, and practical trade-offs.
- **This thesis is** an investigation of whether NO_x emission reduction can be improved in comparison with industry-standard strategies such as rule-based control.
- **This thesis is** a contribution to the broader research project, intended to support future scientific and industrial evaluation of RL-based HEV control.
- **This thesis is not** deployable production software for direct industrial use.
- **This thesis is not** autonomous-driving software or a driver-replacing system.

1.4 Contributions

This thesis contributes to the field of RL-based HEV control as follows:

1. **A surrogate-based DRL control framework.** The considered HEV control task is formulated as a DRL problem in which an agent interacts with a cascaded Long Short-Term Memory (LSTM)-based surrogate environment. The framework targets speed-trajectory tracking while explicitly accounting for tailpipe NO_x emissions and battery SOC.
2. **A curriculum-inspired training strategy.** A two-phase training procedure is proposed in which the agents first learn the fundamental speed-tracking task and are then fine-tuned with additional emission- and charge-sustaining objectives. This staged design separates basic controllability from the more challenging multi-objective optimisation problem.

3. **A comparative actor–critic study.** Three actor–critic RL algorithms, namely Proximal Policy Optimisation (PPO), Soft Actor-Critic (SAC), and Twin Delayed Deep Deterministic Policy Gradient (TD3), are implemented and compared using a common environment, evaluation protocol, and hyperparameter-optimisation workflow. This comparison highlights both algorithmic strengths and practical robustness limitations.
4. **A WLTC generalisation evaluation.** The learned controllers are evaluated on the standardised Worldwide Harmonised Light Vehicles Test Cycle (WLTC) profile against a rule-based reference controller. This evaluation provides a first indication of generalisation behaviour, places the learned controllers in relation to a more industry-standard solution, and reveals remaining trade-offs between NO_x , fuel consumption, speed tracking, and charge sustainment.
5. **Project-level design insights.** The thesis provides guidance for future development within the LDCI2027 project by documenting the influence of reward design, reward-weight selection, algorithm choice, and surrogate-model limitations on RL-based HEV control.

1.5 Thesis Organisation

The remainder of this thesis is organised as follows.

- Chapter 2 (Theoretical Background) introduces the technical foundations required for this thesis. It summarises relevant aspects of HEVs, ANNs, LSTM-based networks, and RL, with particular emphasis on actor–critic architectures.
- Chapter 3 (Related Work) reviews previous work within the LDCI2027 project and situates this thesis within the broader state of the art on applying RL to HEV control.
- Chapter 4 (Methodology) formulates the control task as an RL problem and describes the conceptual approach used to address the multi-objective control goal, including the staged training strategy and evaluation methodology.
- Chapter 5 (Implementation) provides the technical implementation details, including the LSTM surrogate models, the RL environment, the selected algorithms, the hyperparameter-optimisation procedure, and the training infrastructure.
- Chapter 6 (Results) presents and analyses the Phase 1 and Phase 2 results, focusing on speed tracking, NO_x emissions, SOC behaviour, operating-point shifts, and zero-shot WLTC generalisation.
- Chapter 7 (Discussion) interprets the results, discusses the main findings and limitations, and reflects on their implications for RL-based HEV control and sustainability.
- Chapter 8 (Conclusion) summarises the thesis, states the main conclusions, and outlines promising directions for future research.

Chapter 2 is intentionally selective: it does not aim to provide an exhaustive overview of all intersecting fields, but instead synthesise the concepts and findings most directly connected to the objectives of this thesis.

Chapter 2

Theoretical Background

This chapter provides the theoretical foundation required to understand the contribution of this thesis. It first introduces the automotive context by defining Hybrid Electric Vehicles (HEVs), describing the main powertrain topologies, and outlining energy-management strategies with a focus on online learning-based control. The discussion then turns to simulation-based development and motivates the use of data-driven surrogate models as a computationally efficient alternative to high-fidelity physical vehicle simulations.

Building on this, the chapter introduces the machine-learning concepts needed later in the thesis. Artificial Neural Networks (ANNs) are presented as general nonlinear function approximators, followed by Long Short-Term Memorys (LSTMs) as a special type of ANNs. The remainder of the chapter establishes the reinforcement-learning background: the agent–environment interaction, exploration and exploitation, Markov Decision Processes (MDPs) and Partially Observable Markov Decision Processes (POMDPs), Deep Reinforcement Learning (DRL), and the actor–critic family of methods. The chapter concludes with the three specific algorithms used in this work, namely Proximal Policy Optimisation (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC).

2.1 Hybrid Electric Vehicles

HEVs are vehicles that use at least two distinct energy-transforming units and energy reservoirs for propulsion [29]. Typically, those are represented by a chemical component (fuel) and an electrical component (battery). Whereas a conventional vehicle relies entirely on an Internal Combustion Engine (ICE) for propulsion, an HEV integrates an Electric Motor (EM) into the powertrain [30].

The main purpose of hybridisation is to introduce an additional degree of freedom in how propulsion power is generated and distributed. This can reduce fuel consumption and generally improve vehicle efficiency [7]. King and Nelson [31] identified several mechanisms through which HEVs can reduce fuel consumption. The most important mechanisms are:

- **Eliminating idle fuel use.** An HEV can switch the engine off when no traction power is required and restart it rapidly when needed, thereby avoiding fuel consumption during engine-off standstill phases. This function is not exclusive to HEVs, since start-stop systems are also

common in conventional vehicles, but hybridisation generally facilitates smoother and more frequent engine-off operation.

- **Recuperating braking energy.** During braking, a conventional vehicle wastes kinetic energy as heat, whereas an HEV can recover part of this energy and store it in a bidirectional energy storage system.
- **Downsizing the engine.** Since conventional vehicle engines are sized for peak demand but are often operated at partial load, HEVs can use electric machines to cover peak demand while allowing the ICE to operate more frequently in higher-efficiency regions.
- **Improving engine operation.** In general, the additional degrees of freedom provided by electric integration enable the engine to operate in more efficient regions and can even facilitate electric-only driving. The extent of this benefit depends on the powertrain topology: series and power-split hybrids can decouple the ICE operating point from the vehicle speed to a greater degree, whereas parallel hybrids retain a stronger mechanical link between engine speed and wheel speed [11].

Because improved fuel economy generally also reduces emissions, lower emissions are a further benefit of HEVs. However, although the mechanisms described above primarily target better fuel economy, fuel-consumption minimisation is not the main objective of this thesis. Instead, this work exploits the same additional control freedom to reduce tailpipe Nitrogen Oxides (NO_x) emissions while maintaining speed tracking and charge-sustaining operation.

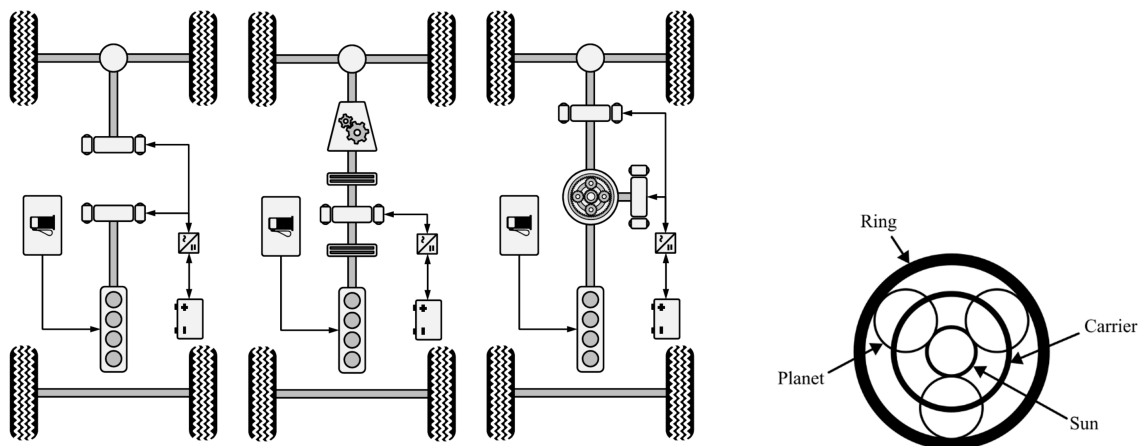
2.1.1 Classifying HEVs

HEVs can be classified in several ways, such as by degree of hybridisation, by the nature of the power source, or by architecture and powertrain configuration [30]. The most relevant distinction for this thesis is the latter. The elementary hybrid powertrain topologies include series hybrids, parallel hybrids, and power-split hybrids [32] (a schematic overview is provided in Figure 2.1a):

- **Series hybrid.** In a series hybrid system, the ICE drives an electric generator instead of driving the wheels directly. In turn, the generator charges the battery and powers an electric motor that propels the vehicle. The electric motor is therefore the only unit directly coupled to the wheels [3], [33].
- **Parallel hybrid.** In a parallel hybrid system, the ICE and electric motor are both connected to a mechanical transmission in parallel. Both can therefore convert power into wheel motion [3].
- **Power-split hybrid.** Also known as *combined hybrid*, *complex* or *series-parallel* [3], [30], [33], these systems represent features of both, series and parallel hybrids. At the cost of higher complexity, this system can transfer power to the wheels mechanically, electrically, or through a combination of both paths. The central component is usually a planetary gearbox that mechanically couples the ICE with the carrier gear, a first Electric Motor 1 (EM1) to the sun gear and the final driveshaft, and a second Electric Motor 2 (EM2) to the ring gear. This layout splits the power produced by the ICE into a direct mechanical path to the wheels and an electrical path to charge the battery through a generator while enabling the ICE speed to be independent of the vehicle speed [5].

EM1 functions as this generator when exerting negative torque on the sun gear [34]. EM1 also controls the gear ratio and enables the continuously variable transmission. EM2 can provide additional torque on the ring gear or independently drive the vehicle electrically, depending on the exact architecture. Both electric motors can recuperate energy during braking [5]. A simple illustration of a planetary gearbox is depicted in Figure 2.1b.

This high degree of freedom makes power-split hybrids efficient and adaptable, but it also increases component count, control complexity, manufacturing cost and vehicle mass [5], [32].



(a) Overview of the main hybrid powertrain configurations: series (left), parallel (middle), and power-split (right); reprinted from [5]; original figures by [35].

(b) Illustration of a planetary gearbox; reprinted from [5].

Figure 2.1: Hybrid powertrain configurations (left) and an illustration of a planetary gearbox (right).

2.1.2 Control Strategies for HEVs

Following the topology reported by Enang and Bannister [3], HEV energy management strategies can first be divided into *offline* and *online* approaches. This top-level distinction reflects whether the control policy is computed mainly before vehicle operation using a known drive cycle, or whether decisions are generated in real time while the vehicle is running. The individual strategy families and subcategories are summarised in Figure 2.2.

Offline energy management strategies. Offline methods assume prior knowledge of the driving mission and therefore solve the energy management problem globally over a complete cycle. Their main purpose is to provide a performance benchmark and to quantify the best achievable fuel economy or emission performance under ideal information. However, their computational burden and their dependence on future driving information make them unsuitable for direct real-time implementation in production vehicles.

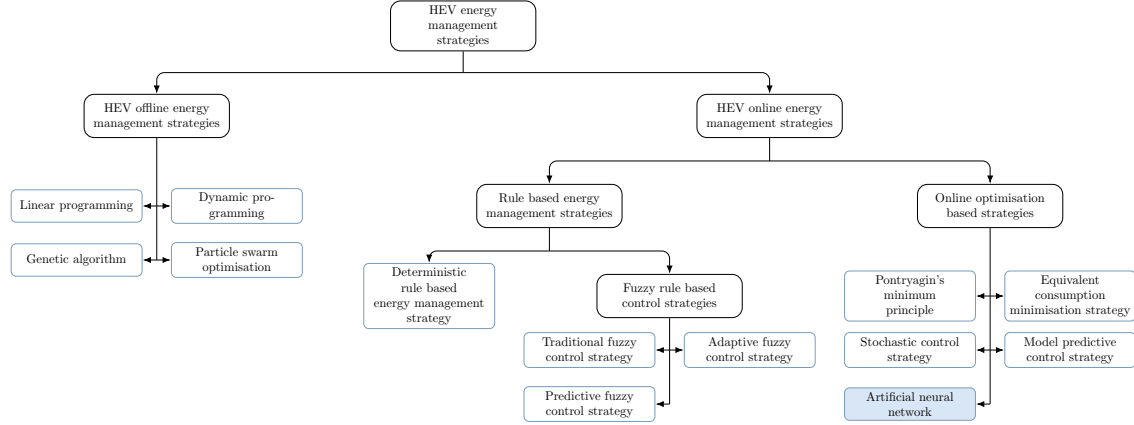


Figure 2.2: Topology of HEV control strategies. Artificial neural networks, which are used in this thesis through DRL are coloured; reprinted from [3].

Online energy management strategies. Online strategies are designed for real-time use and therefore rely only on currently available data. For this thesis, the relevant point is the learning-based branch of the online taxonomy shown in Figure 2.2. In particular, DRL is attractive because it can learn policies from interaction data, can be applied to nonlinear black-box environments, and is suited to complex real-time optimisation tasks [17], [36].

2.1.3 Simulation of HEVs

As discussed above, DRL represents a promising control approach for the energy management of HEVs. As a control method that relies on ANNs, it is inherently data-driven. Consequently, the availability of high-quality data is a fundamental prerequisite. Such data can be obtained either from measurements or from simulation of the vehicle dynamics. For simulation, two principal approaches exist: highly accurate physics-based simulations (often referred as digital twins), and surrogate models that approximate the original system with lower accuracy but higher sample efficiency [37]. In engineering, Gamma Technologies' GT-SUITE [38] is widely used as a “0D/1D” simulator for modelling engines and drivetrains and enables the simulation of combustion, fluid flow, heat transfer, electrical components, and overall vehicle dynamics. However, because it must solve complex thermodynamic and fluid-dynamic equations, such simulations are typically slower than real time. For RL-based control strategies that rely on a vast amount of data, integrating such a simulator directly into the training loop is therefore not feasible. A promising alternative is data-driven modelling with ANNs, especially LSTMs, which are well suited for capturing the temporal dependencies that arise from drivetrain dynamics and thermally dependent emission aftertreatment (see subsection 2.1.4). These models usually run much faster than high-fidelity physics-based simulations, but at the cost of accuracy. Nevertheless, because RL depends on large amounts of data, LSTM-based surrogate models provide a practical solution for this type of control problem; the central challenge is to find an appropriate trade-off between modelling accuracy and sample efficiency.

2.1.4 The Complexity of the Thermal State

Reducing tailpipe emissions in HEVs is more complex than minimising instantaneous fuel use or engine-out emissions. Diesel aftertreatment systems such as the Diesel Oxidation Catalyst (DOC), Diesel Particulate Filter (DPF), and Selective Catalytic Reduction (SCR) process exhaust gases downstream of the ICE, but their effectiveness depends strongly on their thermal state. Below the light-off temperature, the catalyst is not yet sufficiently active and emissions generated by the engine may pass through the aftertreatment system with limited conversion. Consequently, an operating strategy that keeps engine load and exhaust temperature low at every instant is not necessarily optimal over a complete driving cycle. Temporarily operating the engine at a higher load can heat the aftertreatment system more rapidly and may reduce cumulative tailpipe emissions later in the cycle [32].

This thesis does not treat aftertreatment control as a separate research objective. The short discussion is included to make the full control problem visible: the underlying vehicle model includes aftertreatment-related quantities such as SCR, DOC, and DPF temperatures (see Appendix A), and these temperatures depend on previous operation. This delayed thermal behaviour motivates the use of LSTMs as surrogate plant models (see subsection 2.1.3 and subsection 2.2.1) and helps explain why the real control problem is more accurately understood as a POMDP (see subsection 3.1.4 and subsection 4.2.1). For reasons of experimental feasibility, however, this thesis does not address the hidden thermal state with a recurrent Reinforcement Learning (RL) agent. This is further explained in chapter 4.

2.2 Artificial Neural Networks

ANNs, or Neural Network (NN)s, are machine-learning models that learn relationships between inputs and outputs from data. They were inspired by biological brains and consist of connected computational units, called neurons, that are arranged in layers. Each neuron combines incoming values with trainable weights and a bias and then applies a non-linear activation function. By stacking many such units, neural networks can approximate complex, non-linear relationships without manually defining all relevant features in advance [39].

A typical feedforward network passes information from an input layer through one or more hidden layers to an output layer, as illustrated in Figure 2.3. During training, the network parameters are adjusted so that the predicted output becomes closer to a desired target. This is done by minimising a loss function, usually with mini-batch gradient-based optimisation and backpropagation. In practice, this means that prediction errors are propagated backwards through the network to determine how the weights should be updated. This flexibility makes ANNs suitable for prediction, function approximation and control tasks, but it also means that their performance depends strongly on the quality and coverage of the training data [39].

2.2.1 Long Short-Term Memory

LSTMs are a special type of Recurrent Neural Network (RNN) introduced by Hochreiter and Schmidhuber [40]. Unlike feedforward networks, which process each input independently, recurrent networks maintain an internal state that is updated over time. This makes them suitable for sequential data, where the current output depends not only on the current input but also on previous operating conditions [39].

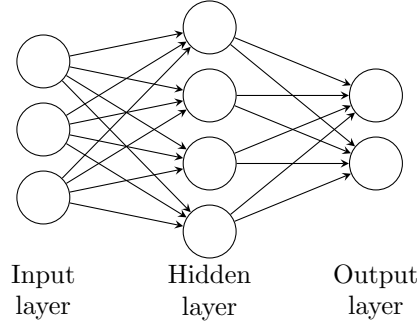


Figure 2.3: Simplified feedforward neural network with an input layer, one hidden layer and an output layer.

The main idea of an LSTM is to extend the recurrent state with a memory cell. As shown in Figure 2.4, this memory is regulated by gates: a forget gate determines which previous information should be retained, an input gate controls how much new information is written, and an output gate determines which part of the memory is exposed to the next layer or time step. This gated structure helps LSTMs preserve relevant temporal information over longer horizons than conventional recurrent networks [39], [40].

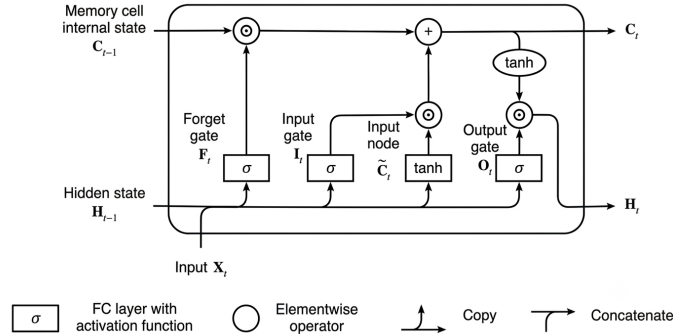


Figure 2.4: Structure of an LSTM cell showing the memory path and the gates that regulate the flow of temporal information; adapted from [41].

In the context of this thesis, LSTMs are relevant because the hybrid powertrain behaviour cannot be described reliably from the instantaneous input alone. Specifically the internal temperatures of the system inherit a temporal lag which makes them highly dependent on previous operating points. LSTM-based surrogate models can therefore capture dynamic relationships in the simulated vehicle while remaining much faster to evaluate than high-fidelity physical simulations (see subsection 2.1.3). In this work, the LSTMs are used for the surrogate environment, whereas the RL agents themselves are implemented in their non-recurrent version (see chapter 4).

2.3 Reinforcement Learning

Besides supervised and unsupervised learning, RL is commonly representing the third pillar of machine learning. In contrast to the former two, which rely on pre-collected datasets, RL is based on sequential interaction between a learning agent and an environment [15].

To formalise RL, several elements are necessary:

- **Agent.** The agent is the decision-making entity in reinforcement learning. It observes the available information from the environment, selects actions, and improves its behaviour over time based on the feedback it receives.
- **Environment.** The environment represents everything external to the agent with which it interacts. It responds to the agent's actions by changing its state and by returning new observations and rewards.
- **Policy.** A policy defines the agent's behaviour by specifying which action should be chosen in a given situation. It can be deterministic or stochastic and represents the object that the agent ultimately tries to optimise.
- **Reward signal.** The reward signal is a scalar feedback value that evaluates the immediate consequence of an action. It defines the learning objective by indicating which behaviours are desirable and which should be avoided.
- **Value function.** The value function estimates how beneficial a state or state-action pair is in the long term. In this way, it supplements the immediate reward by capturing the expected cumulative future return.
- **Model of the environment.** A model describes how the environment evolves and what rewards are likely to result from actions. Such a model is optional since model-free RL exists; the algorithms considered in this thesis do not require an explicit model of the transition dynamics.

At each time step, the agent observes the current situation, selects an action, and receives feedback from the environment in the form of a reward. The objective of the agent is to learn a policy that maximises the expected cumulative reward over time. An illustration of the RL interaction loop is depicted in Figure 2.5 [15].

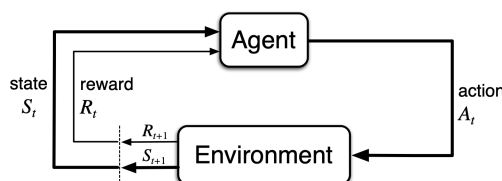


Figure 2.5: Agent–environment interaction in a Markov decision process; reprinted from [15].

Exploration versus exploitation. The central challenge in RL is the trade-off between exploration and exploitation. On the one hand, the agent must exploit behaviours that already appear promising in order to maximise cumulative reward. On the other hand, it must continue to explore alternative actions so that potentially better solutions are not overlooked too early. The difficulty is that an agent cannot focus exclusively on only one of these two objectives without impairing learning performance. This dilemma represents the *fundamental challenge* in RL [15].

2.3.1 Markov Decision Process

The standard mathematical framework used to formulate reinforcement-learning problems is the Markov Decision Process, MDP. It can be described as a sequential decision problem by the tuple $(\mathcal{S}, \mathcal{A}, P, R)$, where $\mathcal{S}$ denotes the state space, $\mathcal{A}$ the action space, $P(s_{t+1} \mid s_t, a_t)$ the state-transition probability, and $R(s_t, a_t, s_{t+1})$ the reward function [42]. At each time step t , the agent is in state $s_t \in \mathcal{S}$, chooses an action $a_t \in \mathcal{A}$, transitions to a new state s_{t+1} according to the dynamics P , and receives a scalar reward. Different authors use slightly different conventions for this tuple: some include a discount factor γ [43], which weights the relevance of future rewards, while others define an MDP through a four-argument probability function [15]. The defining property of an MDP is the *Markov property*, which states that the transition probabilities fully encode the environment’s dynamics. Consequently, the probability of the next state depends only on the current state–action pair (s_t, a_t) and not on the history of previous states and actions [15]:

$$P(s_{t+1} \mid s_t, a_t, s_{t-1}, a_{t-1}, \dots, s_0, a_0) = P(s_{t+1} \mid s_t, a_t). \quad (2.1)$$

In many real-world applications, however, the true system state is not fully observable. Instead, the agent only has access to incomplete, noisy, or indirect observations. Such problems are more appropriately described by a partially observable MDP, POMDP. In this case, the agent receives an observation o_t rather than the full state s_t , so the Markov property does not necessarily hold for the observation alone. This makes the decision problem more difficult, because the agent must infer relevant information about the hidden state from the observation history [15].

The distinction between MDPs and POMDPs is highly relevant in practice. Many control problems are modelled as MDPs for simplicity, even though the available measurements do not fully describe the underlying system. In these cases, a suitable state representation, history window, or recurrent model can help approximate the missing information. This is one reason why sequence models such as LSTMs can be useful in reinforcement learning settings: they help encode temporal context when the current observation is insufficient to characterise the environment completely [15], [42], [43].

2.3.2 Deep Reinforcement Learning

Foundational RL methods are often introduced in tabular form. In this setting, value functions, policies, or both are represented explicitly as tables whose entries correspond to individual states or state–action pairs. Such formulations are the most basic representation of RL, but they become impractical as soon as the control problem grows in complexity [15].

The main limitation is scalability. If the state space or action space is continuous, it must first be discretised before a table can be used. This introduces approximation errors and, depending on the granularity of the discretisation, can also lead to an enormous number of table entries. The

same issue appears in high-dimensional problems where the number of possible state combinations grows combinatorially, making RL prohibitively expensive or unfeasible [15], [18].

A solution is to replace explicit tables with function approximators. Instead of storing a separate value for every individual state or action, the agent learns a parametric function that maps observations or state-action pairs to value estimates, policies, or both. This allows the agent to generalise from observed samples to unseen states and actions, and to operate in spaces that are too large for explicit enumeration. Among the available function approximators, NNs have proven especially powerful (see section 2.2). In RL, NNs can therefore be used to approximate value functions, policies, or complete actor-critic architectures [15].

DRL is exactly this combination of RL with deep NN function approximation. It makes RL applicable to high-dimensional, continuous, and nonlinear problems that would otherwise be intractable. In the context of this thesis, this step is essential because the considered HEV control task involves continuous control actions, nonlinear vehicle and aftertreatment dynamics, and observations whose relevance can depend on temporal context. A neural-network policy or critic can operate directly on such continuous representations and can be trained from simulated interaction data without manually enumerating the state-action space [15], [18]. In the following, RL and DRL are used interchangeably because the deep variant is the only form used in this work.

2.3.3 Classifying RL Algorithms

The following taxonomy is intentionally non-exhaustive. Its purpose is not to cover every family of RL algorithms, but to introduce the distinction that leads to the actor-critic methods used in this thesis. A useful starting point is the pair of classical algorithmic viewpoints of *value iteration* and *policy iteration*. Although modern DRL algorithms are usually formulated with neural-network function approximators rather than explicit tables, these two perspectives still provide the conceptual basis for classifying many RL methods [15].

Value-based methods. In value-based methods, the central object of learning is a value function, for example a state-value function $V(s)$ or an action-value function $Q(s, a)$. The policy is obtained only indirectly from these estimates by selecting the action with the highest predicted return. Learning therefore focuses on evaluating the long-term consequences of actions first, while decision-making follows from this evaluation step [15].

Policy-based methods. In policy-based methods, the policy itself is the object of optimisation. Instead of deriving behaviour from value estimates, a parameterised policy $\pi_\theta(a \mid s)$ is adjusted directly so that the expected discounted return

$$G_t = \sum_{k=0}^{\infty} \gamma^k r_{t+k+1} \quad (2.2)$$

is maximised. This perspective is particularly attractive in continuous-action problems, because it avoids the need to search over all possible actions, and it naturally permits stochastic policies. For this reason, direct policy optimisation plays a central role in modern continuous-control DRL [15], [18].

Actor–critic methods. These arise as a synthesis of these two families. The actor represents the policy and selects actions, while the critic estimates a value function that assesses the quality of states, actions, or the current policy. The critic thereby provides a training signal for improving the actor, whereas the actor generates the behaviour from which the critic learns. Actor–critic approaches can thus be interpreted as combining the direct optimisation of policy-based methods with the evaluative structure of value-based methods [15], [44]. This combination is especially important in DRL, where it often improves data efficiency and stabilises learning in high-dimensional and continuous domains.

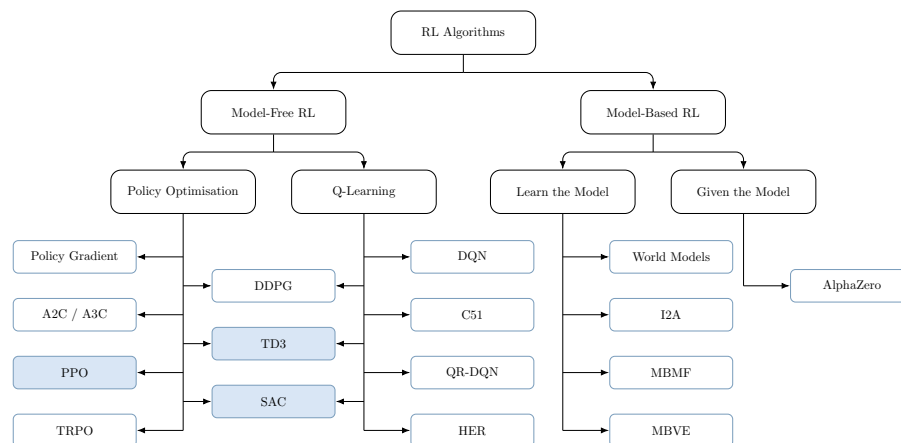


Figure 2.6: A non-exhaustive, but useful taxonomy of algorithms in modern RL; algorithms used in this thesis are coloured; adapted from [45].

This taxonomy is useful for positioning the algorithms discussed later, since all three methods considered in this work are actor–critic algorithms and therefore inherit characteristics from both direct policy optimisation and value-function learning (Figure 2.6).

2.3.4 Overview of Actor–Critic Architectures

Building on the taxonomy above, actor–critic architectures combine value-based and policy-based learning within a single framework. They learn a parameterised policy $\pi_{\theta}(a \mid s)$ and a value function, such as $V(s)$ or $Q(s, a)$, at the same time [15].

This combination addresses several limitations of the two individual paradigms by using two interacting sets of neural-network parameters:

- **The actor.** Represents the policy parametrisation, θ . It determines the agent’s behaviour by mapping states to actions. The actor’s weights are updated in the direction suggested by the critic so that the expected return is increased.
- **The critic.** Represents the value-function parametrisation ω . It evaluates the actions taken by the actor and provides a learning signal for improving the current policy.

Actor–critic methods address key limitations of purely value-based and purely policy-based reinforcement learning by combining their main strengths within a single architecture. A simple illustration of an actor–critic architecture is depicted in Figure 2.7.

Limitations of value-based methods. Purely value-based methods are often comparatively sample-efficient, but they are less natural for continuous action spaces and for learning genuinely stochastic policies. In such settings, selecting an action by maximising a value function over all possible actions becomes difficult or even intractable. Actor-based parameterisations avoid this problem by learning the parameters of a policy distribution directly, for example the mean and standard deviation of a Gaussian distribution [15], [18].

Limitations of policy-based methods. Purely policy-based methods are well suited to continuous control because they optimise a parameterised policy directly. However, they often suffer from high-variance gradient estimates because they rely on sampled trajectory returns. In the classical REINFORCE algorithm, for example, the return is estimated with Monte Carlo sampling. This means that the complete realised return of a trajectory is used as the learning signal, so an update may have to wait until the end of an episode and can vary strongly between sampled trajectories. Although such estimates are unbiased, they can therefore lead to slow and unstable learning. Actor-critic methods reduce this variance by introducing a critic that bootstraps value estimates and enables temporal-difference learning. Bootstrapping means that the current value estimate is updated from an immediate reward plus the critic’s estimate of the next state, for example through a target of the form $r_{t+1} + \gamma V(s_{t+1})$. This introduces some bias, but it provides stepwise feedback on the actor’s decisions and usually yields a lower-variance training signal [15]. Modern actor-critic methods often refine this idea further by using the advantage function, $A(s, a) = Q(s, a) - V(s)$, so that the critic evaluates how much better an action is compared with the baseline value of the state. This usually yields a more stable and informative learning signal for policy updates [46].

Data efficiency via off-policy learning. A further advantage of many actor-critic architectures is that they can be combined with off-policy learning. Whereas basic on-policy methods can only learn from data generated by their current target policy, off-policy methods improve a *target policy* while using a different *behaviour policy* to generate data [15]. This can substantially improve data efficiency because collected experience remains useful even after the policy has been updated.

In practice, this is commonly realised with an experience replay buffer, which stores interactions with the environment as transition tuples of the form $(S_t, A_t, R_{t+1}, S_{t+1})$. This idea was introduced by Lin [47] under the term *experience replay*. By sampling from this buffer, the agent can reuse past experience in mini-batch updates rather than updating only from the most recent transition. This improves computational efficiency and leads to more stable gradient estimates. In addition, random sampling from the replay buffer breaks the strong temporal correlations that are present in consecutive samples from a trajectory, which can further stabilise learning [18]. For the present work, this distinction is important because PPO is an on-policy method, whereas TD3 and SAC are off-policy methods that can exploit replay buffers.

The three algorithms used in this thesis all belong to the actor-critic family. They differ along two main axes: PPO is on-policy with a stochastic policy, TD3 is off-policy with a deterministic policy, and SAC is off-policy with a stochastic policy regularised by an entropy term. The selection rationale is given in subsection 4.2.5; the present subsections only summarise their algorithmic mechanics.

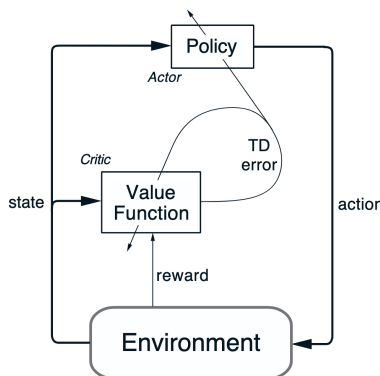


Figure 2.7: The actor–critic architecture; reprinted from [48].

2.3.5 Proximal Policy Optimisation (PPO)

PPO [46] is an on-policy actor–critic method that has become one of the standard baselines for continuous control and is applicable to both discrete and continuous action spaces. Like its predecessor Trust Region Policy Optimisation (TRPO) [49], it is motivated by the question of how to take the largest useful improvement step on the current policy without moving so far that performance collapses [50]. Whereas TRPO enforces this with a hard trust-region constraint solved by a second-order method, PPO obtains a comparable effect using only first-order information through a *clipped surrogate objective* [46], [50].

The probability ratio between the new and the old policy is clipped to a narrow interval $[1 - \epsilon, 1 + \epsilon]$, and the objective is taken as the minimum of the unclipped and the clipped term. This minimum turns the objective into a pessimistic (lower) bound: once the ratio leaves the interval on the side that would otherwise increase the objective, the term saturates and contributes no further gradient, which removes the incentive to move the policy far from its previous version. It is important to note that clipping only *discourages* large updates rather than strictly forbidding them. For this reason practical implementations add further safeguards, such as early stopping once the mean Kullback–Leibler divergence between the new and old policy exceeds a threshold [46], [50].

PPO naturally supports multiple parallel actors: N environments each collect a fixed-length rollout, advantages are estimated from the collected data, and the resulting buffer is then optimised for K epochs of minibatch Stochastic Gradient Descent (SGD) or with Adam [46], [51]. After this optimisation pass the rollout is discarded, since PPO is strictly on-policy and cannot reuse trajectories generated by an earlier version of the policy. The clipping mechanism makes PPO more stable and considerably simpler to implement than TRPO, at the price of substantially lower sample efficiency than off-policy alternatives such as SAC and TD3 [46]. Figure 2.8 reproduces the algorithm in pseudocode.

2.3.6 Twin Delayed Deep Deterministic Policy Gradient (TD3)

TD3 [52] is an off-policy actor–critic algorithm developed to address two coupled failure modes of its predecessor Deep Deterministic Policy Gradient (DDPG) [18]: the systematic overestimation of state–action values caused by function-approximation error, and the high-variance, error-

Algorithm 1 PPO-Clip

-
- 1: Input: initial policy parameters θ_0 , initial value function parameters ϕ_0
 - 2: **for** $k = 0, 1, 2, \dots$ **do**
 - 3: Collect set of trajectories $\mathcal{D}_k = \{\tau_i\}$ by running policy $\pi_k = \pi(\theta_k)$ in the environment.
 - 4: Compute rewards-to-go $\hat{R}_t$.
 - 5: Compute advantage estimates, $\hat{A}_t$ (using any method of advantage estimation) based on the current value function V_{ϕ_k} .
 - 6: Update the policy by maximizing the PPO-Clip objective:

$$\theta_{k+1} = \arg \max_{\theta} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \min \left(\frac{\pi_{\theta}(a_t|s_t)}{\pi_{\theta_k}(a_t|s_t)} A^{\pi_{\theta_k}}(s_t, a_t), \quad g(\epsilon, A^{\pi_{\theta_k}}(s_t, a_t)) \right),$$

typically via stochastic gradient ascent with Adam.

- 7: Fit value function by regression on mean-squared error:

$$\phi_{k+1} = \arg \min_{\phi} \frac{1}{|\mathcal{D}_k|T} \sum_{\tau \in \mathcal{D}_k} \sum_{t=0}^T \left(V_{\phi}(s_t) - \hat{R}_t \right)^2,$$

typically via some gradient descent algorithm.

- 8: **end for**
-

Figure 2.8: PPO pseudocode; reprinted from [50].

accumulating value estimates that make DDPG fragile and highly sensitive to hyperparameters [52], [53]. Like DDPG, TD3 learns a deterministic policy; for exploration it adds uncorrelated zero-mean Gaussian noise to the actions during data collection, the authors having found that the Ornstein–Uhlenbeck noise originally used by DDPG offered no measurable benefit [52]. TD3 introduces three modifications relative to DDPG, the first two of which give the algorithm its name:

- First, *clipped double-Q learning* (hence “twin”) maintains two independently initialised critics and forms a single temporal-difference target from the *minimum* of their two target predictions, which directly limits the overestimation bias that arises when a critic is bootstrapped from its own, possibly over-optimistic, value estimate [52], [53]. Favouring the smaller estimate may introduce a slight underestimation, but this is far less harmful, since underestimated values are not propagated through the policy update (as low-valued actions are avoided by the policy) [52].
- Second, *delayed policy updates* (hence “delayed”) update the actor and the target networks less frequently than the critics. The original paper recommends one policy update for every two critic updates ($d = 2$), so that the critics can settle to lower-variance estimates before each policy-improvement step, reducing the actor-side volatility that builds up through the coupling of actor and critic [52], [53].
- Third, *target-policy smoothing* adds small clipped Gaussian noise to the target action before it is evaluated by the critics, enforcing the intuition that similar actions should have similar values and preventing the deterministic policy from exploiting outlier peaks in the learned Q-function [52], [53].

Figure 2.9 reproduces the algorithm in pseudocode.

2.3.7 Soft Actor-Critic (SAC)

SAC [54] is an off-policy actor–critic algorithm that combines the sample efficiency of DDPG-style methods with the exploration benefits of a stochastic, entropy-regularised policy; it was developed roughly concurrently with TD3. Unlike DDPG [18] and TD3 [52], which use a deterministic policy and rely on externally added exploration noise, SAC learns a stochastic policy directly and therefore explores by sampling from its current policy. In continuous action spaces a categorical distribution such as a softmax is not applicable, so SAC parameterises the policy as a Gaussian whose samples are mapped to a bounded interval by a hyperbolic-tangent (“squashing”) transformation. Training such a stochastic actor with low-variance gradients is enabled by the reparameterisation trick, which expresses the sampled action as a deterministic function of the state, the policy parameters and an independent noise variable, so that the policy gradient can be propagated through the sampling step [54], [55].

The defining feature of SAC is its *maximum-entropy objective*: instead of maximising the expected return alone, it maximises a trade-off between the expected return and the entropy of the policy, weighted by a temperature coefficient α . This entropy regularisation acts as a built-in exploration mechanism and is known to reduce the tendency of off-policy actor–critic methods to converge prematurely to local optima [55]. Like TD3, SAC employs the clipped double-Q setup to mitigate overestimation bias. However, the next-state actions used in the target are sampled from the *current* policy rather than a separate target policy, and because that policy is stochastic, the noise inherent in sampling already provides an effect similar to TD3’s explicit target-policy smoothing, so no separate smoothing step is required. At evaluation time the policy is typically made deterministic by using the mean action, which usually improves performance over the stochastic policy [54], [55]. Figure 2.10 reproduces the algorithm in pseudocode.

Algorithm 1 Twin Delayed DDPG

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi_1, \phi_2$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\theta_{\text{targ}} \leftarrow \theta, \phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$ 
3: repeat
4:   Observe state  $s$  and select action  $a = \text{clip}(\mu_\theta(s) + \epsilon, a_{\text{Low}}, a_{\text{High}})$ , where  $\epsilon \sim \mathcal{N}$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for  $j$  in range(however many updates) do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute target actions
          
$$a'(s') = \text{clip}(\mu_{\theta_{\text{targ}}}(s') + \text{clip}(\epsilon, -c, c), a_{\text{Low}}, a_{\text{High}}), \quad \epsilon \sim \mathcal{N}(0, \sigma)$$

13:      Compute targets
          
$$y(r, s', d) = r + \gamma(1 - d) \min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', a'(s'))$$

14:      Update Q-functions by one step of gradient descent using
          
$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$

15:      if  $j \bmod \text{policy\_delay} = 0$  then
16:        Update policy by one step of gradient ascent using
          
$$\nabla_{\theta} \frac{1}{|B|} \sum_{s \in B} Q_{\phi_1}(s, \mu_\theta(s))$$

17:        Update target networks with
          
$$\begin{aligned} \phi_{\text{targ},i} &\leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \\ \theta_{\text{targ}} &\leftarrow \rho \theta_{\text{targ}} + (1 - \rho) \theta \end{aligned} \quad \text{for } i = 1, 2$$

18:      end if
19:    end for
20:  end if
21: until convergence

```

Figure 2.9: TD3 pseudocode; reprinted from [53].

Algorithm 1 Soft Actor-Critic

```

1: Input: initial policy parameters  $\theta$ , Q-function parameters  $\phi_1, \phi_2$ , empty replay buffer  $\mathcal{D}$ 
2: Set target parameters equal to main parameters  $\phi_{\text{targ},1} \leftarrow \phi_1, \phi_{\text{targ},2} \leftarrow \phi_2$ 
3: repeat
4:   Observe state  $s$  and select action  $a \sim \pi_\theta(\cdot|s)$ 
5:   Execute  $a$  in the environment
6:   Observe next state  $s'$ , reward  $r$ , and done signal  $d$  to indicate whether  $s'$  is terminal
7:   Store  $(s, a, r, s', d)$  in replay buffer  $\mathcal{D}$ 
8:   If  $s'$  is terminal, reset environment state.
9:   if it's time to update then
10:    for  $j$  in range(however many updates) do
11:      Randomly sample a batch of transitions,  $B = \{(s, a, r, s', d)\}$  from  $\mathcal{D}$ 
12:      Compute targets for the Q functions:


$$y(r, s', d) = r + \gamma(1 - d) \left( \min_{i=1,2} Q_{\phi_{\text{targ},i}}(s', \tilde{a}') - \alpha \log \pi_\theta(\tilde{a}'|s') \right), \quad \tilde{a}' \sim \pi_\theta(\cdot|s')$$


13:      Update Q-functions by one step of gradient descent using


$$\nabla_{\phi_i} \frac{1}{|B|} \sum_{(s,a,r,s',d) \in B} (Q_{\phi_i}(s, a) - y(r, s', d))^2 \quad \text{for } i = 1, 2$$


14:      Update policy by one step of gradient ascent using


$$\nabla_\theta \frac{1}{|B|} \sum_{s \in B} \left( \min_{i=1,2} Q_{\phi_i}(s, \tilde{a}_\theta(s)) - \alpha \log \pi_\theta(\tilde{a}_\theta(s)|s) \right),$$


      where  $\tilde{a}_\theta(s)$  is a sample from  $\pi_\theta(\cdot|s)$  which is differentiable wrt  $\theta$  via the
      reparametrization trick.

15:      Update target networks with


$$\phi_{\text{targ},i} \leftarrow \rho \phi_{\text{targ},i} + (1 - \rho) \phi_i \quad \text{for } i = 1, 2$$


16:    end for
17:  end if
18: until convergence

```

Figure 2.10: SAC pseudocode; reprinted from [55].

Chapter 3

Related Work

This chapter summarises previous work related to this thesis. Specifically, it covers previous achievements and findings of the Light-Duty CI-ICE 2027 (LDCI2027) project from a control perspective, as well as general research on Reinforcement Learning (RL)-based Hybrid Electric Vehicle (HEV) control strategies.

3.1 Previous Light-Duty CI-ICE 2027 Progress

The LDCI2027 project focuses on developing advanced operating strategies to minimise Nitrogen Oxides (NO_x) emissions in light-duty hybrid diesel vehicles, using Artificial Intelligence (AI) and RL to support compliance with strict emission regulations such as Euro 6 and Euro 7 [25], [26]. Previous progress within this project can be divided into two development phases: the initial parametrisation and testing of control algorithms in MATLAB/Simulink, and the subsequent transition to advanced recurrent Deep Reinforcement Learning (DRL) architectures in Python to address partial observability and computational bottlenecks.

3.1.1 Initial Modelling and Base Control Strategies

Early work in the project focused on replacing computationally demanding physics-based simulations with data-driven surrogate models [28]. The authors used Long Short-Term Memory (LSTM) Neural Network (NN)s provided by the Institute of Automotive Engineering Stuttgart (IFS) to predict the vehicle’s dynamic behaviour, particularly the battery’s State of Charge (SOC) and tailpipe NO_x emissions [27].

To control these models, the initial methodologies used pre-processing frameworks based on lookup tables and brake-specific fuel consumption curves. This allowed the controller to map high-level actions, such as the power-split factor, to specific engine parameters, including fuel injection mass [27].

Using this environment, two control strategies were initially tested [27]:

- **Nonlinear Model Predictive Control (NLMPCC).** An NLMPCC was implemented to balance SOC tracking and NO_x minimisation. However, the simulations showed that although the controller could maintain the SOC reference, it struggled significantly with emissions, resulting in NO_x levels nearly five times higher than the target constraints.

- **Soft Actor-Critic (SAC) Agent.** An early RL approach was tested using a SAC algorithm. This implementation highlighted the critical challenge of multi-objective reward scaling. The penalty for NO_x emissions was found to be orders of magnitude larger than the SOC reward, which strongly biased the agent. The SAC algorithm ultimately failed to converge within the tested episodes, demonstrating the need for more robust training frameworks and improved reward engineering.

3.1.2 Model Predictive Control

Recognising the limitations of the initial implementations, later progress shifted towards a Python-based framework to develop an incremental NN-based controller using supervised learning. This phase introduced several methodological improvements [28]:

- **Incremental action prediction.** Instead of predicting absolute control values, the new controller was redesigned to predict incremental variations (Δ s) using stateful LSTMs with a multilayer perceptron head. This approach simplified learning, prevented gradient explosion, and provided more stable control.
- **Differentiable physical constraints.** A major challenge was that the Internal Combustion Engine (ICE) model is physically valid only at engine speeds above 900rpm; below this threshold, the fuel mass must be strictly zero. To work with this restriction, the authors implemented “soft activation layers” in the form of sigmoid gates. This allowed the network to learn differentially how to shut off fuel injection when the engine speed dropped below the threshold.

3.1.3 Recurrent Deep Reinforcement Learning

Despite the improvements achieved with the supervised predictive controller, performance degraded at high speeds and under extreme operating conditions [28]. Furthermore, a paradigm shift was required: because the internal memory states of the surrogate LSTMs cannot be measured in a real vehicle, they must remain hidden from the controller [28]. These hidden states encode thermal dynamics that influence aftertreatment effectiveness, including the light-off behaviour introduced in subsection 2.1.4. This strict reliance on measurable physical variables transformed the problem into a Partially Observable Markov Decision Process (POMDP) [28].

To address this issue, the project shifted entirely to a DRL framework, specifically selecting a recurrent Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm. This algorithm was chosen because its deterministic policy enables precise setpoint tracking, its off-policy nature improves sample efficiency, and its twin-critic design stabilises continuous control [28]:

- **Agent architecture.** The architecture used separate LSTMs for the actor and the critics to prevent representation bottlenecks and to ensure that the critics evaluated temporal dependencies independently of the policy.
- **Overlapping trajectories.** The researchers implemented an overlapping-window replay buffer with a “burn-in” phase to warm up the hidden states before backpropagation. This was necessary because recurrent sampling from the replay buffer must preserve temporal dependencies. During burn-in, a fixed number of initial steps are executed only in forward mode, without propagating gradients back through them.

- **Distributed learning via Ray [56].** Data collection was significantly accelerated using a distributed actor–learner architecture, with environment simulations running in parallel on CPUs while a centralised GPU learner optimised the policy.

3.1.4 Current Limitations and Open Challenges

Although the established theoretical pipeline and system architecture are advanced and modular, the latest recurrent TD3 implementation was unable to learn a robust strategy that fully stabilises the vehicle speed at the target value [28]. The author hypothesises that this is primarily due to a computational sampling bottleneck: simulating the environment with the surrogate LSTMs restricts data collection, leaving too few interactions for DRL convergence in continuous spaces [28]. Additionally, potential implementation bugs in the recurrent state propagation or burn-in logic also remain an open challenge [28].

The two issues identified in subsection 3.1.1, together with the open challenge of the sampling bottleneck, motivate the methodology of this thesis. The emissions shortfall is addressed through the Phase 2 NO_x penalty and charge-sustaining reward design; the reward-scaling and convergence problem is addressed through reward normalisation, staged training, and reward-weight optimisation; and the sampling bottleneck is addressed by using a newer version of the surrogate LSTM models (see chapter 4 and section 5.1).

3.2 State of the Art – Reinforcement-Learning-Based Hybrid-Electric-Vehicle Control

The state of the art in RL-based HEV control is broad, covering different objectives, challenges, approaches, and algorithms [57], [58], [59]. Many studies address individual aspects of this multi-objective problem, such as avoiding local minima through exploration [36], handling hybrid action spaces with separate networks for engine-throttle control and gear shifting [9], or predicting vehicle velocity when future route knowledge is unavailable [60]. Because these use cases do not fully align with the present thesis, they mainly provide methodological components that can be adapted here. The most relevant studies are discussed in the following.

3.2.1 Continuous Control with Actor-Critic Architectures

Traditional value-based RL methods, such as Q-learning, struggle with the high-dimensional and continuous control spaces inherent in HEV energy management. These methods rely on evaluating discrete action bins, which makes them susceptible to discretisation errors and the curse of dimensionality when applied to continuous variables. To overcome these limitations, the state of the art increasingly relies on actor–critic architectures, as described in subsection 2.3.4 [9], [14].

Recent HEV literature shows a clear transition towards state-of-the-art actor–critic algorithms, primarily Deep Deterministic Policy Gradient (DDPG), SAC, and TD3. For example, Tresca et al. [17] implemented a SAC agent to manage a plug-in HEV, highlighting the algorithm’s entropy maximisation mechanism, which naturally enhances training stability and exploration. To prevent the neural networks from overfitting to a single driving route, they introduced a multi-cycle training strategy across 72 different mission profiles, showing that training over an ensemble of cycles can substantially improve controller generalisation and robustness under unseen real-world conditions. Similarly, Zhou et al. [14] addressed the inherent instability and overestimation bias of standard

DDPG agents by employing an improved TD3 algorithm. Their TD3-based Energy Management System (EMS) effectively smoothed the continuous torque commands, leading to faster convergence and more stable global performance than traditional value-based and policy-based methods.

3.2.2 Multi-Objective Staged Course Learning

Formulating an EMS for intelligent HEVs requires balancing multiple conflicting objectives simultaneously, such as minimising fuel consumption, reducing tailpipe emissions, maintaining battery SOC, and ensuring passenger comfort [6]. Aggregating these competing goals into a single static reward function can destabilise the RL process. The agent may receive interfering reward signals, leading to premature convergence or trapping the policy in local optima, thereby preventing optimisation of the intended objective.

To address the complexities of high-dimensional multi-objective control, Wang and Wang [6] proposed a multi-stage “course learning” mechanism integrated within a TD3 framework for managing both Adaptive Cruise Control (ACC) and EMS. Drawing inspiration from progressive human learning, this approach breaks the complex control problem down into sequential sub-stages with different focuses. In the initial training stage, the agent is strictly guided to satisfy primary safety prerequisites, receiving rewards exclusively for maintaining a safe car-following distance and basic speed tracking. Once the agent successfully learns to keep the speed-tracking error within an acceptable boundary, the training automatically transitions to a secondary stage that refines the control-action quality. In this subsequent phase, variable virtual goals are introduced and energy-saving rewards are amplified, allowing the agent to optimise secondary objectives, such as fuel economy and power-source durability, while retaining the foundational safety rules.

3.2.3 Domain Knowledge Integration

While model-free DRL algorithms offer powerful self-learning capabilities, their initial trial-and-error exploration phases can generate irrational, physically infeasible, or otherwise unfavourable control actions. Relying purely on the neural network to autonomously discover and represent the physical constraints of the ICE, electric motors, and battery pack is sample-inefficient and may not sufficiently cover the real control problem, especially when component-health-preserving measures are not correctly encoded in the RL loop. Consequently, recent research trends focus on actively embedding domain knowledge and heuristic rules directly into the DRL loop to constrain the action space.

Zhou et al. [14] designed a heuristic rule-based local controller that acts as a dynamic action-space regulator within their TD3-based EMS. By computing the instantaneous allowable engine and motor torques before neural-network exploration, the local controller filters out infeasible or irrelevant torque allocations, reducing the search space and providing a “warm start” that accelerates convergence. Furthermore, Tang et al. [9] showed that DRL algorithms generally struggle to learn intermittent and discontinuous control commands, such as gear shifting or mode switching, because DRL networks output continuous actions at every time step. To address this limitation, they embedded a rule-based engine start–stop strategy into their Double DRL framework. By assigning the intermittent decision of turning the engine on or off to a deterministic rule-based system, the continuous RL agent could focus on optimising the throttle, resulting in fuel-economy improvements over traditional Dynamic Programming (DP) benchmarks.

Chapter 4

Methodology

This chapter presents the methodological framework used to develop Reinforcement Learning (RL)-based control policies for reducing Nitrogen Oxides (NO_x) emissions in the considered hybrid electric vehicle. The goal is to track a target speed trajectory closely while minimising NO_x emissions and maintaining the battery’s State of Charge (SOC).

The chapter first introduces the closed-loop interaction between the agent and the Long Short-Term Memory (LSTM)-based surrogate environment, then formulates the control task as an approximate Markov Decision Process (MDP) and motivates the selected observation, action, and reward design at a conceptual level. It subsequently describes the curriculum-inspired two-phase training procedure, in which speed-tracking feasibility is established before emission- and SOC-related objectives are introduced. Finally, it defines the evaluation logic used to compare agents and to establish the baselines that later optimisation stages must improve upon.

This chapter is closely linked to the implementation details described in chapter 5. It explains the methodological *what* and *why*, allowing the overall approach to be followed and reproduced at a conceptual level. The following chapter, by contrast, addresses *how exactly* the methodology was implemented and therefore supports reproducibility at an engineering level.

4.1 Overview — The Control Loop

The overall control loop consists of an RL agent interacting with a surrogate Hybrid Electric Vehicle (HEV) environment, as shown in Figure 4.1. The environment is composed of two cascaded LSTM models: an Internal Combustion Engine (ICE) surrogate, which predicts engine torque and emissions, and a drivetrain surrogate, which uses the predicted torque together with the remaining actuator commands to predict vehicle speed and SOC.

Earlier work within the Light-Duty CI-ICE 2027 (LDCI2027) project used a recurrent Twin Delayed Deep Deterministic Policy Gradient (TD3) algorithm as the RL agent and correctly framed the control problem as a Partially Observable Markov Decision Process (POMDP); however, it was not able to stabilise the vehicle speed at a single constant target value (see chapter 3). This thesis therefore adopts a simplified formulation to obtain stable control behaviour, while explicitly acknowledging that the underlying POMDP is not handled optimally: it uses non-recurrent agents, approximates the partially observable problem as an MDP, and extends the control objective from maintaining one fixed speed to tracking a complete speed trajectory.

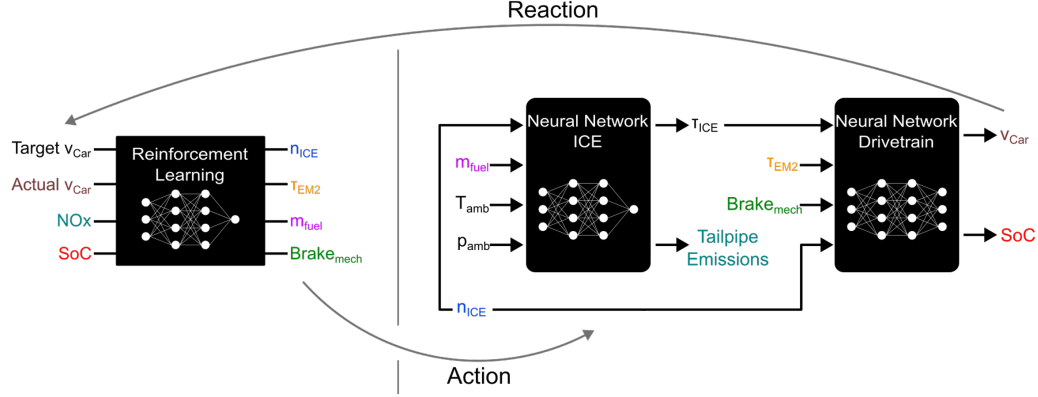


Figure 4.1: Closed-loop interaction between the RL agent and the surrogate environment.

4.2 Reinforcement Learning Problem Formulation

Again, the task is formulated as a RL problem in which an agent interacts with an environment represented by two LSTM surrogate models representing the ICE and the drivetrain. As discussed in subsection 2.1.3, these models replace more expensive physical simulations while preserving the temporal structure required for control development.

4.2.1 Markov Decision Process Framing

The problem formulation is inherently partially observable. The plant models contain internal states that capture thermal engine and aftertreatment dynamics, but these states would not be directly measurable in a deployed vehicle with the assumed sensor setup. The control problem is therefore more accurately described as a partially observable sequential decision problem than as a fully observable one.

At the current stage of development, however, these thermal effects are represented only in simplified form to reduce complexity. A more rigorous treatment of the hidden internal state would require recurrent RL algorithms, which can use LSTMs or other recurrent neural networks to infer latent plant dynamics. This thesis does not take that step. Although the LSTM surrogates expose several aftertreatment-related variables, only one thermal value is included in the agent’s observation vector: the temperature of Selective Catalytic Reduction (SCR)¹. This variable serves as a partial proxy for the otherwise hidden thermal state and allows the problem to be approximated as an MDP for the present implementation, even though the real system is more accurately characterised as a POMDP.

4.2.2 Observation Design

The observation supplied to the agent is intentionally extensive, but it remains restricted to quantities that could plausibly be measured on a real vehicle with a realistic sensor setup. Consequently, the internal recurrent states of the plant models and most aftertreatment temperatures are excluded, even though the surrogate networks would expose them, because they would not be available at deployment time. Within this constraint, the observation comprises four conceptual groups:

- **Tracking information.** The current vehicle speed and its error relative to the reference trajectory, which define the primary tracking objective.
- **Energy state.** The battery SOC and its deviation from the initial value, which provide the information required for charge-sustaining operation.
- **Powertrain and emission state.** Instantaneous quantities such as ICE speed, ICE torque, injected fuel mass and tailpipe NO_x , which characterise the current engine operating point and its immediate emission consequence.
- **Actuation history.** The most recent actuator commands and a normalised timer counting the steps since the last engine on/off transition, which provide limited temporal context for the otherwise memoryless policy.

A single thermal variable, the wall temperature of the SCR before the turbine, is included as a deliberate exception. It serves as a partial proxy for the otherwise hidden aftertreatment thermal state and is the only concession used to approximate the underlying POMDP as the tractable MDP formulation adopted in this thesis (subsection 4.2.1). Other potentially useful variables that would not be measurable on a real vehicle are excluded by design. The exact composition of the observation vector, the physical bounds of each component and the per-component normalisation applied before the policy network are reported in section 5.3.

4.2.3 Action Design

The action design is based on the requirements specified by the LDCI2027 project [28]. At the conceptual level, the controller must output four control quantities (see Figure 4.1):

- ICE speed (n_{ICE}),
- Electric Motor 2 (EM2) torque (τ_{EM2}),
- fuel mass (m_{fuel}), and
- mechanical brake percentage (**Brakemech**).

These quantities describe the required control interfaces, but they do not fully capture the input restrictions imposed by the plant LSTMs. In particular, the engine has two distinct operating regimes: an engine-off state and an engine-on state with a valid ICE speed and fuel mass interval. An explicit engine-off case therefore has to be represented in the action design. While the earlier LDCI2027 work incorporated sigmoid soft-gate layers for this purpose [28] (see chapter 3), this thesis uses a single continuous engine command as the corresponding agent action. Internally, this command is then mapped either to the engine-off case or to a valid engine-on ICE speed and fuel mass range.

4.2.4 Reward Design

The reward is the channel through which the control objectives are communicated to the agent. It combines a primary speed-tracking term with penalty terms for the secondary objectives, namely

emissions and SOC deviation, as well as a small number of shaping terms for mechanically undesirable behaviour such as unnecessary braking and rapid engine on/off switching. The speed-tracking reward is maximal at zero tracking error and decays smoothly as the error increases.

Because the objectives are measured on very different physical scales (e.g. km/h vs grams), each term is normalised to a comparable range before weighting. This keeps the relative weights interpretable and prevents any single objective from overwhelming the others. Such behaviour was observed in an earlier iteration of the project, where an unnormalised emission penalty dominated the SOC reward and prevented the agent from converging (chapter 3). The shaping terms are kept small because they are intended to regularise implausible actuation and guide learning, rather than to express primary objectives in their own right.

Since the RL reward is a proxy for the true objectives rather than the objective itself, agents are selected and compared using directly measured deployment metrics (section 4.4), not accumulated reward. Enabling or disabling individual terms through their weights also realises the staged learning approach: Phase 1 keeps only the speed-tracking term active, whereas Phase 2 additionally activates the emission and SOC terms (subsection 4.3.2, subsection 4.3.3). The exact reward formulation, normalisation factors and per-phase weights are given in section 5.3.

4.2.5 Selection of Reinforcement-Learning Algorithms

The final selection of RL algorithms was guided by two considerations:

- First, a simple and widely used baseline algorithm was needed to verify that the training pipeline and environment interaction worked as intended. Proximal Policy Optimisation (PPO) was selected for this role because it is comparatively easy to understand, well documented and commonly implemented.
- Second, more capable off-policy algorithms were required for the main continuous-control experiments. Soft Actor-Critic (SAC) and TD3 were selected because they are sample-efficient actor-critic methods and have been used successfully in related HEV control studies: [17] used SAC, while [14] used TD3. They also align with the prior LDCI2027 work, in which SAC was used in [27] and (recurrent) TD3 was used in [28]. These algorithms therefore provide the more advanced candidates for the nonlinear control task considered in this thesis.

4.3 Multi-Stage Course Learning Approach

The experimental strategy follows a phase-wise procedure that is conceptually close to curriculum learning [61]. It separates basic controllability from later multi-objective optimisation and thereby makes the validation process more interpretable.

4.3.1 Terminology and Motivation

Training a Deep Reinforcement Learning (DRL) agent on conflicting multi-objective goals simultaneously, as is the case in controlling an HEV, is a difficult problem and can lead to local optima and unstable training. Learning such complex and nonlinear control policies therefore requires additional guidance to stabilise training. Optimising several variables at once increases the complexity of the problem, according to the curse of dimensionality (see section 1.1), and contributed to the

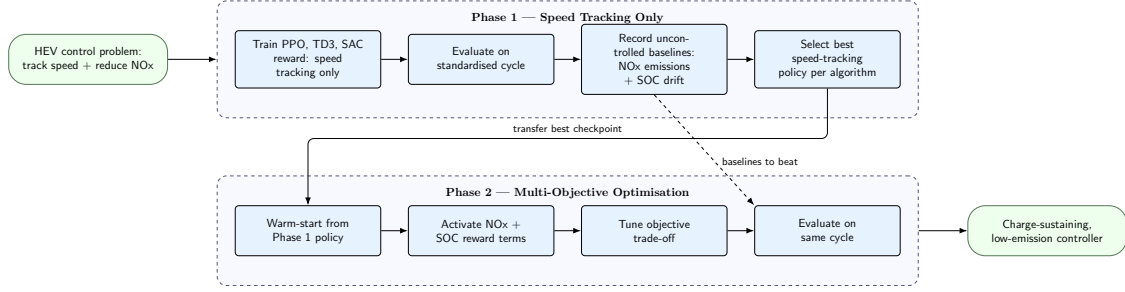


Figure 4.2: Overall experimental pipeline of the study.

difficulty of balancing reward weights in the prior LDCI2027 work [27]. This thesis addresses this issue by implementing a *multi-stage course-learning approach*. Although this term is adopted from Wang and Wang [6], it describes the broader concept of curriculum learning in machine learning [61], where the difficulty of the learning task is increased as the model’s competence improves. Sutton and Barto [15] however refer to this technique as *shaping*. Derived from animal psychology, shaping involves “changing the reward signal as learning proceeds” [15].

In this thesis, the term *shaping* follows Ibrahim et al. [62] and should not be confused with Sutton and Barto’s use of the term. Here, shaping is understood in the context of reward shaping: reward engineering defines the reward function itself, whereas reward shaping further refines or augments the reward signal to guide learning more effectively [62]. The concrete reward design used in this work is described in subsection 4.2.4 and implemented in section 5.3.

In the first phase, the reward function is designed to prioritise the fundamental prerequisite of speed tracking. Once baseline control competence has been established, the trained models are reused and the curriculum advances to a second stage, in which additional reward terms for SOC deviation and NO_x emissions are activated to optimise the energy-management strategy. This simplifies the learning process by addressing the objectives sequentially rather than all at once, thereby reducing the complexity of the optimisation problem.

4.3.2 Phase 1 — Exclusive Speed Tracking

The agents are trained with a single active objective - speed tracking. In this stage, PPO, TD3 and SAC are each expected to learn how to follow the desired speed profile as accurately as possible, while effects on battery SOC and emissions are deliberately ignored in the reward. The results are used to record the corresponding SOC variation and NO_x emission levels, establishing empirical reference bounds for the later optimisation stage.

The two questions Phase 1 answers. The role of Phase 1 in the overall thesis goal of learning a speed-tracking and emission-reducing controller for an HEV is twofold:

1. **Feasibility.** Demonstrate that each candidate RL algorithm can learn the speed-tracking task at all in the LSTM-surrogate environment, thereby ruling out environment-side training issues before any reward extension is attempted.
2. **Unconstrained baselines.** Quantify the cumulative NO_x emissions and the SOC drift that emerge as side effects of speed-only optimisation. These two quantities are the targets

that Phase 2 must improve upon and define the reference against which any multi-objective formulation will be benchmarked.

4.3.3 Phase 2 — Multi-Objective Optimisation

Phase 2 fine-tunes the best Phase 1 checkpoint (the one with the lowest RMSE in speed-tracking), selected from the ten seed runs, under an augmented multi-objective reward. In addition to speed tracking, the reward activates NO_x and SOC terms. The objective is therefore not merely to follow the speed trajectory, but to do so with lower tailpipe NO_x emissions while avoiding the trivial strategy of depleting the battery. Phase 2 addresses this directly by steering the policy towards charge-sustaining, low-emission operation without discarding the tracking behaviour learned in Phase 1.

Fine-tuning from the Phase 1 checkpoint. Rather than training each agent from scratch under the full multi-objective reward, Phase 2 initialises every agent from its Phase 1 best-seed checkpoint and fine-tunes it under the augmented reward. Conceptually, this transfers the already-learned speed-tracking behaviour into the second stage, so that training only has to reshape the policy towards the additional objectives instead of re-learning controllability from zero. This warm-start is the core of the curriculum-inspired staged approach: basic controllability is acquired first and the harder multi-objective trade-off is introduced only once that foundation is in place.

Reward augmentation. The augmented reward retains the Phase 1 speed-tracking term and adds two penalties. The first penalises tailpipe NO_x , reflecting the central objective of the thesis. The second penalises deviations of the battery SOC from its initial value, enforcing charge-sustaining behaviour. The exact reward formulation and weights are reported in section 5.3.

The main difficulty in Phase 2 is balancing the conflicting objectives of speed tracking, emission reduction and charge sustainment. Their relative importance is controlled by the reward weights. Two strategies for selecting these weights are considered. The first is a manual grid search over the emission and SOC weights, with one agent fine-tuned for each combination. This approach is simple and interpretable, but it evaluates only a small number of fixed combinations at substantial computational cost.

Tuning reward weights. The grid search is therefore replaced by an automated Optuna search using the sampling and pruning strategy already employed in Phase 1. Each trial fine-tunes an agent with a sampled weight pair and evaluates it using a *composite score* that reflects the intended behaviour: minimise cumulative tailpipe NO_x , while applying increasing penalties when speed-tracking error or battery drift exceed acceptable tolerances. This converts the multi-objective trade-off into a constrained scalar objective and focuses the search on weight regions that reduce emissions without sacrificing tracking or charge sustainment. The exact weight ranges, penalty tolerances and search budget are given in subsection 5.7.2.

Overall, Phase 2 measures how far optimised multi-objective policy improves over the baselines established in Phase 1.

4.4 Evaluation Methodology

The evaluation of both, Phase 1 and Phase 2 follows deliberate protocols:

True objective versus reward proxy for Phase 1. As discussed above, the reward signal is only a proxy for the real task objective. Although cumulative reward is commonly used to evaluate RL agents [15], it can be misleading when the reward function is imperfectly designed. This is particularly relevant in Phase 1, where the target objective is explicitly defined as speed tracking only. The best Phase 1 agent is therefore not selected by the highest cumulative reward, but by the lowest Root Mean Square Error (RMSE) between the target speed trajectory and the realised vehicle speed. The RMSE is defined as

$$\text{RMSE}_v = \sqrt{\frac{1}{N} \sum_{t=1}^N (v_{\text{target},t} - v_{\text{Car},t})^2}, \quad (4.1)$$

where N is the number of evaluated time steps, $v_{\text{target},t}$ is the target velocity at time step t , and $v_{\text{Car},t}$ is the corresponding realised vehicle velocity.

Performance metrics. Phase 1 therefore uses speed-tracking RMSE [km/h] as its primary performance metric. It also records cumulative NO_x emissions [g/evaluation cycle], battery SOC drift [decimal fraction], and cumulative fuel consumption [g/evaluation cycle] as diagnostic quantities. The SOC drift is reported as the absolute difference between the initial and final SOC values. Speed-tracking Mean Absolute Error (MAE) [km/h] is included as an additional tracking metric: while MAE weights all absolute speed errors linearly, RMSE penalises larger deviations more strongly because the errors are squared before averaging. Consequently, RMSE is used as the main Phase 1 selection criterion, whereas Phase 2 combines speed-tracking RMSE, cumulative NO_x, and SOC drift into a composite scalar score. The exact formulation is given in subsection 5.7.2.

Statistical reliability. To account for the stochastic nature of DRL, each selected configuration is trained and evaluated with multiple random seeds. The resulting metrics are reported as mean $\pm$ standard deviation across seeds, which makes it possible to distinguish systematic algorithm behaviour from random-seed variance. In addition to these aggregate statistics, the best-performing seed is retained as the checkpoint for the subsequent phase, since Phase 2 requires a single policy to be fine-tuned.

Evaluation cycle. Each trained policy is evaluated on the standardised six-phase speed trajectory shown in Figure 4.3. Inference is performed deterministically to ensure reproducibility and comparability. The speed steps are defined heuristically to assess acceleration and deceleration at different intensities. The implementation can also be applied to other trajectories, including synthetic cycles, standardised real-world driving cycles, and constant-speed scenarios.

Final WLTC comparison. While phase 1 and phase 2 both evaluate their hyperparameter or reward tuning experiments on the same evaluation cycle (Figure 4.3), a final experiment is conducted which switches the evaluation cycle from the synthetic one to a real-world standardised Worldwide Harmonised Light Vehicles Test Cycle (WLTC) [63]. The WLTC is a globally standardised driving

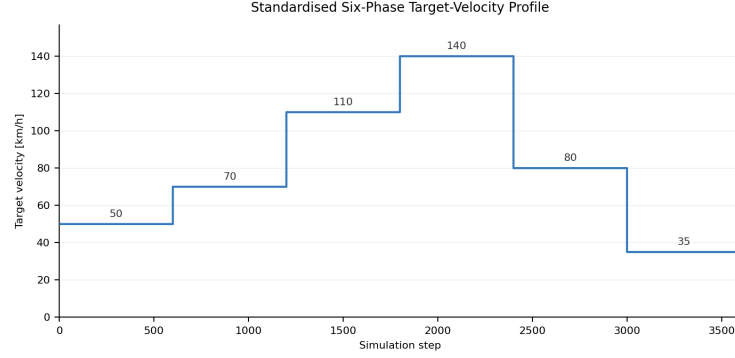


Figure 4.3: Standardised six-phase target-velocity trajectory used as the default evaluation benchmark.

profile used to measure automotive fuel consumption, tailpipe emissions, and electric vehicle range. Derived from real-world driving data, it consists of distinct low, medium, high, and extra-high speed phases that mimic diverse urban, suburban, rural, and motorway driving conditions.

This final comparison is used to bridge the gap simulation to reality gap a bit more by assessing the learned controller in a more realistic scenario instead exclusively on synthetic cycles. It assesses the capability of the final models to generalise to situation that haven't been part of their training and compares the effectivity of them, in terms of speed tracking, NO_x reduction and SOC sustainment compared to a rule-based solution developed by Frey et al. [5].

Chapter 5

Implementation

This chapter describes the implementation of the experimental framework used to train and evaluate the reinforcement-learning controllers. It first introduces the Long Short-Term Memory (LSTM)-based vehicle-model stack, including its interfaces, validity ranges, recurrent-state initialisation, and deployment optimisations that make large-scale Reinforcement Learning (RL) training computationally feasible. The chapter then details the custom Gymnasium [64] environment that connects the surrogate models to the learning agents by defining the observation space, action mapping, reward function, safety mechanisms, and episode structure.

The remaining sections describe the concrete RL algorithm implementations, the hyperparameter-optimisation procedure, the training infrastructure, and the two-stage experimental pipeline. In this way, the chapter provides the engineering-level details required to reproduce the Phase 1 speed-tracking experiments and the Phase 2 emission- and State of Charge (SOC)-aware fine-tuning experiments.

5.1 The Specific Vehicle LSTMs

The vehicle-model stack used in this work consists of recurrent LSTM surrogate models provided by Frey et al. [5]. These models represent the EA189 Internal Combustion Engine (ICE) with a customised aftertreatment setup and the drivetrain. The LSTM network training configurations are described in Table 5.1.

Parameter	ICE	Drivetrain	Drivetrain Plus
Layers	3	3	3
Neurons per layer	256	256	256
Sequence length [s]	600	1800	1800
Batch size	1	1	1
Optimiser	AdamW	AdamW	AdamW
Initial learning rate	1e-3	1e-3	1e-3
Scaler	Min-max-scaler	Min-max-scaler	Min-max-scaler
Loss function	MAE	MAE	MAE

Table 5.1: Neural network architectures and training hyperparameters of the LSTM surrogate models used in this work.

The underlying numerical plant models were implemented in GT-Suite [38] to generate synthetic data for training the LSTM surrogates. These are characterised by the following properties:

Cascaded architecture. The final model stack consists of three neural networks: an *ICE* network, a *Drivetrain* network, and the optional *Drivetrain Plus* network, as illustrated in Figure 5.1. The ICE network receives engine speed (n_{ICE}), fuel mass (m_{fuel}), ambient temperature (T_{amb}), and ambient pressure (p_{amb}) as inputs and predicts engine torque (τ_{ICE}) as the transfer output, together with emissions- and aftertreatment-related variables.

The drivetrain network receives n_{ICE} , τ_{ICE} , EM2 torque (τ_{EM2}), and mechanical braking (**Brakemech**) as inputs and predicts vehicle speed (v_{Car}) and battery state of charge (*SoC*). For RL purposes, v_{Car} and *SoC* are the key plant outputs required to evaluate tracking performance, energy usage, and the feasibility of the control action. Ambient conditions are assumed to remain constant over one complete driving cycle of 1800 seconds, with valid ranges of 250–310 K for T_{amb} and 0.8–1.1 bar for p_{amb} .

An exhaustive overview of the network interfaces of the ICE, Drivetrain, and Drivetrain Plus neural networks is provided in Appendix A; Table A.1, Table A.2, and Table A.3.

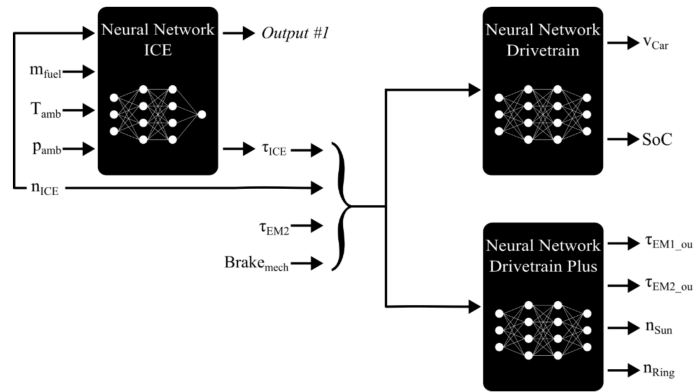


Figure 5.1: Final version of the cascaded surrogate LSTM architecture; reprinted from [5].

Validity ranges. The latest models represent cooldown behaviour during engine-off phases by including stationary cooldown cycles in the training set. Consequently, the ICE model has two distinct valid operating regimes: engine-off operation with $n_{ICE} = 0$ rpm and $m_{fuel} = 0$, and engine-on operation in the range $900 \text{ rpm} \leq n_{ICE} \leq 4000 \text{ rpm}$ with $3 \text{ mg} \leq m_{fuel} \leq 70 \text{ mg}$. The intermediate region between engine-off and engine-on conditions is explicitly invalid, meaning operation between 0 and 900 rpm or between 0 and 3 mg fuel mass is not supported and may produce unphysical behaviour. The operating ranges of the ICE and drivetrain models are summarised in Table 5.2 and Table 5.3.

Parameter	Range
Speed (n_{ICE})	Engine off: 0 rpm; engine on: 900–4000 rpm
Fuel mass (m_{fuel})	Engine off: 0 mg; engine on: 3–70 mg
Ambient temperature (T_{amb})	250–310 K
Ambient pressure (p_{amb})	0.8–1.1 bar

Table 5.2: Valid ranges of the ICE model.

Parameter	Range
State of charge (SoC)	0–1
Engine speed (n_{ICE})	0–4500 rpm
EM2 torque (τ_{EM2})	–421 to 421 Nm
ICE torque (τ_{ICE})	–50 to 300 Nm
Mechanical brake ($Brakemech$)	0–100%

Table 5.3: Valid ranges of the drivetrain model.

Initialisation of the LSTMs. All recurrent models use a *two-part* LSTM scheme, with a dedicated initialisation model and an inference model, to prevent recurrent inference from starting with arbitrary hidden states. By default, these states are initialised to zero, which can result in a warm-up phase with reduced prediction accuracy. Frey et al. [5] followed Hagenbucher et al. [65] to address this issue by the *two-parted* intitialisation. The initial timestep is handled by a separate initialisation model that maps physically meaningful initial conditions to the internal LSTM state required by the inference model.

Drivetrain Plus network. The vehicle-model stack includes an additional *Drivetrain Plus* network, whose interface is summarised in Appendix A; Table A.3. It uses the same inputs as the standard drivetrain model but predicts additional internal drivetrain variables. It is not required for the present RL control loop, but it allows for richer state diagnostics and more interpretable post-analysis of drivetrain operation.

5.2 Computational Efficiency and Deployment

The vehicle-model stack provides a computationally efficient surrogate environment for RL experiments, but achieving this throughput required particular deployment optimisations. Since RL

training depends directly on the rate of plant-model inference, these inference-performance gains are indispensable for the present work.

Model execution mode. Inference time was a major bottleneck in previous project iterations (subsection 3.1.4). The main improvement was achieved by utilising the `tf.function` decorator, which enables TensorFlow to compile the Python code into a highly optimised computation graph. Instead of relying on Python’s default eager execution to evaluate operations step by step, this approach traces the function once and reuses the pre-compiled graph for subsequent calls. This bypasses the Python interpreter overhead and allows for graph-level optimisations, significantly accelerating inference times [66]. On the Worldwide Harmonised Light Vehicles Test Cycle (WLTC), this reduced the total rollout time from 104.50 s to 4.79 s in a local MacBook benchmark (Figure 5.2).

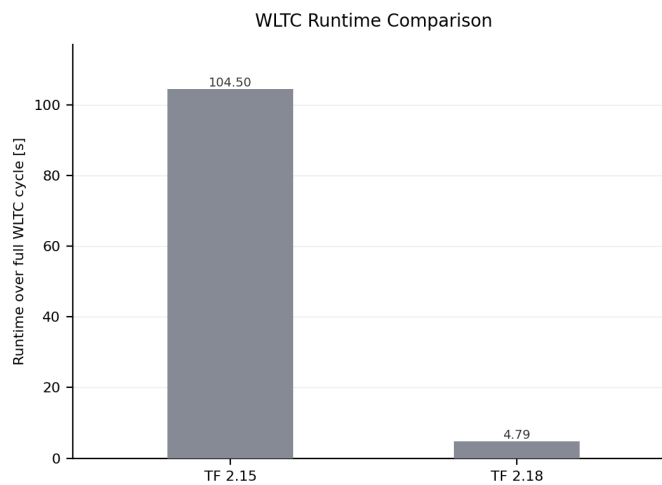
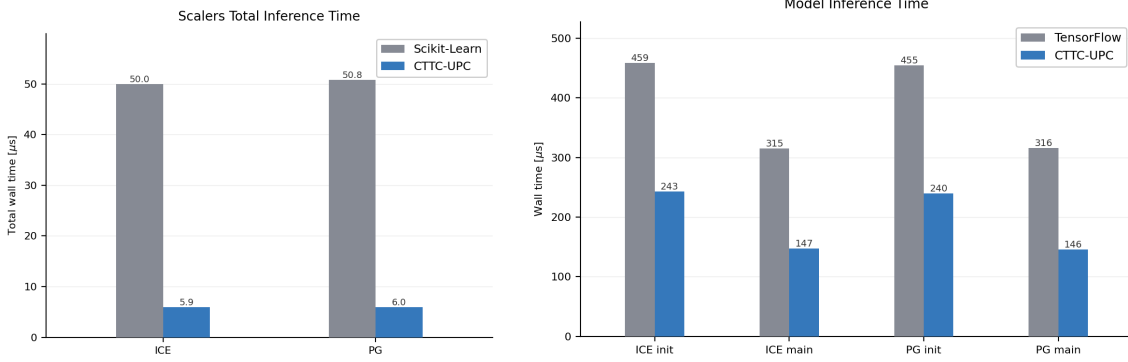


Figure 5.2: Total runtime over the full WLTC cycle executed on a 2023 Macbook Pro with a speed up of more than 20 $\times$.

Model file switch. Following the recommendations of researchers at CTTC-UPC [67], the models were converted from TensorFlow to a language- and framework-agnostic format to improve portability across execution environments. Since no automatic conversion was available for the models used here, CTTC-UPC implemented ad hoc export routines and developed a high-level library that emulates the TensorFlow interface, enabling seamless model replacement. This produced additional performance gains which, although smaller than the graph-mode improvement described above, still contribute meaningfully to efficient RL training (Figure 5.3).



(a) Total scaler overhead per control step, obtained by summing the three scaler operations per inference call: forward input transformation, inverse output transformation, and forward output transformation. The speed up corresponds to approximately $8.5\times$.

(b) Forward-pass latency for the initialisation and inference model, yielding approximately $2\times$ speed ups, respectively.

Figure 5.3: Wall-clock inference time of the original Scikit-Learn/TensorFlow implementations versus the optimised CTTC-UPC [67] counterparts.

5.3 The Reinforcement-Learning Environment

After the surrogate LSTMs have been validated, they cannot simply be connected directly to the RL agent. Instead, a custom Gymnasium [64] environment is required to wrap the cascaded LSTM stack and define the essential elements of RL: observations, actions, and rewards.

Observation space. The agent observes a 12-dimensional vector that instantiates the four conceptual groups defined in subsection 4.2.2. Its components, their physical units and the per-component bounds used for the subsequent normalisation are listed in Table 5.4. The bounds are chosen to comfortably enclose the operating range (based on the requirements defined by [5]), supported by the surrogate networks (section 5.1) and the limit values exposed by the action interface.

#	Symbol	Description	Unit	Low	High
1	v_{Car}	Vehicle speed	km/h	-5	150
2	Δv	Speed error w.r.t. target	km/h	-155	155
3	SOC	Battery state of charge	-	0	1
4	τ_{ICE}	ICE torque	Nm	-50	300
5	$\dot{m}_{NO_x, tp}$	Tailpipe NOx mass-flow rate	g/s	0	0.4
6	n_{ICE}	ICE speed	rpm	0	4000
7	m_{fuel}	Injected fuel mass	mg	0	70
8	ΔSOC	SOC deviation from episode-initial	-	-1	1
9	$\bar{t}_{eng}$	Normalised engine-switch timer	-	0	1
10	τ_{EM2}^-	Last EM2 torque	Nm	-421	421
11	u_{brk}^-	Last mechanical brake percentage	%	0	100
12	$T_{wall, SCR1}$	SCR1 wall temperature	K	298	1240

Table 5.4: Components of the observation vector, with their physical bounds used for min-max normalisation. The thermal proxy $T_{wall, SCR1}$ is the single compromise that turns the underlying POMDP into the tractable MDP approximation of subsection 4.2.1.

After clipping each component to [low, high], the observation is min-max-normalised to $[0, 1]$ and supplied to the policy network as a $\mathbb{R}^{12}$ vector. The engine-switch timer is itself a normalised quantity in $[0, 1]$ that counts the elapsed steps since the last engine on/off transition and equals 1 once the engine switch cooldown described below has elapsed.

Action space. The agent acts through a four-dimensional continuous Box action in $[-1, 1]^4$. Each component is rescaled per step to its physical range before being passed to the plant, as listed in Table 5.5.

#	Symbol	Description	Agent range	Physical range
1	a_{ICE}	ICE command (on/off + rpm)	$[-1, 1]$	$\{\text{off}\} \cup [900, 4000]$ rpm
2	τ_{EM2}	EM2 torque	$[-1, 1]$	$[-421, 421]$ Nm
3	m_{fuel}	Fuel mass per cycle	$[-1, 1]$	$[3, 70]$ mg
4	u_{brk}	Mechanical brake	$[-1, 1]$	$[0, 100]$ %

Table 5.5: Components of the action vector, with the agent-side normalised range and the physical range after rescaling.

The ICE command implements the design choice introduced in subsection 4.2.3: it encodes both the engine on/off decision and the rpm setpoint through the piecewise map

$$(n_{ICE}, m_{fuel}) = \begin{cases} (0, 0), & a_{ICE} \leq 0, \\ (900 + a_{ICE} (4000 - 900), m_{fuel, scaled}), & a_{ICE} > 0, \end{cases} \quad (5.1)$$

where $m_{fuel, scaled}$ is the fuel command after the agent-to-physical rescaling of Table 5.5. The negative half of the ICE command therefore corresponds to engine-off with fuel mass forced to zero, while the positive half spans the LSTM-valid engine speed range $[900, 4000]$ rpm. The intermediate invalid regions of the ICE surrogate (0-900 rpm and 0-3 mg) are unreachable by design.

Reward function. The reward emitted each step comprises four conceptual contributions: a Gaussian speed-tracking reward, an emission penalty, a charge-sustaining SOC penalty, and a mechanical shaping penalty that regularises engine flickering and braking (which proved necessary in order to stabilise early training),

$$R_t = \underbrace{W_{\text{speed}} \cdot \exp\left(-\frac{1}{2} \left(\frac{\Delta v_t}{\sigma_v}\right)^2\right)}_{\text{Speed Tracking}} - \underbrace{(W_{\text{emi}} \cdot \tilde{e}_{\text{NO}_x,t})}_{\text{Emissions}} - \underbrace{(W_{\text{soc}^2} \cdot (\Delta \text{SOC}_t)^2)}_{\text{SOC Management}} - P_{\text{mech},t}, \quad (5.2)$$

where $\Delta v_t = v_{\text{ref},t} - v_t$ is the instantaneous speed error, $\Delta \text{SOC}_t = \text{SOC}_t - \text{SOC}_0$ the deviation from the episode-initial SOC, and $\tilde{e}_{\text{NO}_x,t} = \min(\dot{m}_{\text{NO}_x,\text{tp},t}/c_{\text{NO}_x,\text{rate}}, 1)$ the clipped normalised tailpipe Nitrogen Oxides (NO_x) mass-flow rate. The mechanical shaping penalty is

$$P_{\text{mech},t} = W_{\text{brake}} \cdot \frac{u_{\text{brk},t}}{c_{\text{brk}}} + W_{\text{flick}} \cdot [\text{engine}_t = \text{on} \wedge \text{engine}_{t-1} = \text{off}], \quad (5.3)$$

and the fixed normalisation constants take the values $c_{\text{NO}_x,\text{rate}} = 0.4$ g/s and $c_{\text{brk}} = 100\%$ (based on the requirements defined by [5]). The flicker penalty is applied once whenever the engine transitions from off to on and, paired with the engine-switch cooldown introduced above, discourages high-frequency engine flickering. The speed-tracking scale $\sigma_v = 10$ km/h contributes to the reward signal as shown in Figure 5.4.

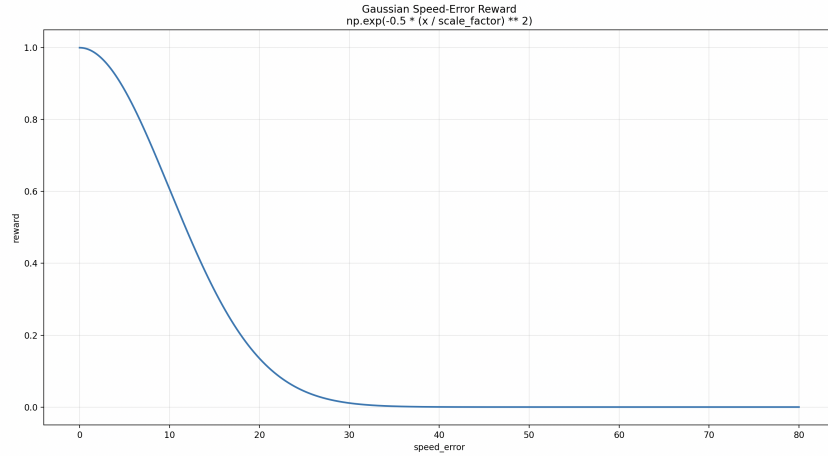


Figure 5.4: Gaussian speed-tracking reward scaling for $\sigma_v = 10$ km/h. The reward reaches its maximum at zero speed error and decays smoothly as the absolute tracking error increases.

Two design choices in Equation 5.2 deserve emphasis:

- First, the speed term is a dense, bounded bell in $[0, 1]$ centred at zero error rather than a quadratic or absolute penalty: this keeps the tracking contribution from saturating the return at large transient errors while preserving a strong gradient close to the reference.

- Second, the emission term is clipped at 1 with the $\min()$ operator inside $\tilde{e}_{\text{NO}_x,t}$, which caps the per-step penalty whenever the surrogate model produces an anomalously large tailpipe NO_x mass-flow prediction. This ensures stability against single outliers that might disrupt a smooth reward signal. The squared SOC term penalises depletion and over-charging symmetrically around the episode-initial SOC while penalising higher deviation stronger than minor in order to ensure that the agent is still able to use the electric motor to some extent. This term implements the charge-sustaining objective motivated in subsection 4.2.4.

The per-phase weights are listed in Table 5.6. Phase 1 activates only the speed term and the mechanical shaping penalty, so that the agent first learns speed tracking in isolation, as motivated in subsection 4.3.2. Phase 2 additionally activates the emission and SOC terms; their weights are first explored on a coarse 3×3 grid over $(W_{\text{emi}}, W_{\text{soc}^2})$ and subsequently refined by an Optuna-based search over the continuous ranges given in the last column of Table 5.6. The search procedure itself is described in subsection 5.7.2.

Weight	Phase 1	Phase 2 grid	Phase 2 Optuna range
W_{speed}	1.0	1.0	1.0
W_{emi}	0.0	{0.25, 0.5, 1.0}	[0.25, 1.25]
W_{soc^2}	0.0	{50, 150, 400}	[25, 250]
W_{brake}	0.25	0.25	0.25
W_{flick}	0.25	0.25	0.25

Table 5.6: Reward weights per training phase. Phase 1 keeps the emission and SOC terms inactive to isolate speed tracking. Phase 2 activates them and optimises their weights. Default values are stored in `config_rewards.py`.

With the Phase 1 weights of Table 5.6, the reward consists of the Gaussian speed-tracking term with $W_{\text{speed}} = 1.0$ and $\sigma_v = 10$ km/h, together with the two mechanical shaping penalties ($W_{\text{brake}} = 0.25$ and $W_{\text{flick}} = 0.25$), while the emission and SOC penalties are disabled ($W_{\text{emi}} = W_{\text{soc}^2} = 0$). The maximum per-step reward is therefore 1.0, achieved at zero speed error ($\Delta v_t = 0$), so the theoretical reward ceiling over a 1,800 s episode sampled at 2 Hz (3,600 simulation steps) is 3,600. How closely the trained policies approach this ceiling is discussed in section 6.1.

Safety mechanisms. Three additional mechanisms enforce that the surrogate networks remain inside their validated operating domain and that mechanically implausible actuation is suppressed.

- **Hard SOC clip.** The drivetrain output SOC_t is hard-clipped to $[0.0001, 0.9999]$ before being written back to the observation. This prevents the recurrent state of the drivetrain LSTM from drifting into the unphysical boundary regions $\{0, 1\}$, where the surrogate’s prediction is not validated. These restrictions are not inherent artefacts of the LSTMs; rather, they follow from how the surrogates were trained and were communicated orally by Frey et al. [5] during preparation for this thesis.
- **Action safety filter.** When SOC_t is within 0.02 of either boundary, the Electric Motor 2 (EM2) torque is overridden to zero if its sign would worsen the boundary violation: charging ($\tau_{\text{EM}2} < 0$) is blocked at $\text{SOC}_t \geq 0.98$, and discharging ($\tau_{\text{EM}2} > 0$) is blocked at $\text{SOC}_t \leq 0.02$. This guarantees that the agent cannot drive the battery model beyond its validated window

even before the hard clip takes effect. Although this measure is partly redundant with the hard SOC clip, it provides an additional safety layer.

- **Engine switch cooldown.** The engine on/off state must remain unchanged for at least six consecutive simulation steps (3 s at 2 Hz). When the agent requests a transition before this window has elapsed, the requested state is overridden by the previous one and the engine-on default is set to the lower valid bounds $(n_{ICE}, m_{fuel}) = (900 \text{ rpm}, 3 \text{ mg})$. Whenever the engine is off, both n_{ICE} and m_{fuel} are forced to zero regardless of the agent’s commands. Following the conceptualisation of Alshiekh et al., this can be considered a post-posed shield: the shield is placed after the learning agent, monitors the selected action, and substitutes a safe action only if the agent’s choice violates the safety specification [68]. This ensures that the engine state is not toggled more frequently than physically feasible and avoids unrealistic engine wear.

Episode structure. All episodes are simulated at the surrogate sampling rate of 2 Hz and span $N = 3,600$ steps, corresponding to 1,800 s of vehicle operation. The target-speed schedule is configured differently for training and evaluation:

- **Training.** The environment is constructed with `random_target=True`. Each episode is split into six 600-step segments (300 s each) and a new target speed is drawn uniformly from $[0, 150]$ km/h at the start of every segment. Exposing the policy to many such randomised schedules spans a broad range of step transitions and steady-speed operating points.
- **Evaluation.** The environment is constructed with `eval_mode=True` and uses the fixed deterministic schedule shown in Figure 4.3. The six segments are each held for 600 steps, supporting the deterministic and reproducible benchmarking discussed in section 4.4.

On every `reset()` call, the environment also re-initialises the recurrent state of both plant LSTMs through the two-part initialisation scheme described in section 5.1. The initialisation network is supplied with physically meaningful initial conditions (zero emissions, ambient temperatures at 298 K, no ICE torque, vehicle at rest, $\text{SOC}_0 = 0.7$ unless overridden by the cycle file), which are mapped to hidden states compatible with the inference network. The observable-state attributes (last actions, last speed, last SOC, last NO_x and last SCR1 temperature) are reset consistently with these initial conditions.

5.4 The Reinforcement-Learning Algorithms

The three reinforcement-learning algorithms selected in subsection 4.2.5, namely Proximal Policy Optimisation (PPO), Twin Delayed Deep Deterministic Policy Gradient (TD3), and Soft Actor-Critic (SAC), are implemented through the Stable-Baselines3 (SB3) library [69]. The actor and critic networks are multi-layer perceptrons (SB3’s `MlpPolicy`); their hidden-layer widths and depths are not fixed manually but are treated as hyperparameters in the Phase 1 search. PPO uses a shared feature extractor for actor and critic, whereas SAC and TD3 use separate actor and twin-critic networks. Each agent observes the 12-dimensional observation space, acts on the 4-dimensional action space defined in section 5.3, and is trained with the vectorised environment and normalisation wrappers described in section 5.6.

Apart from selecting the algorithm families and defining the heuristic hyperparameter-search spaces, no final agent configuration was chosen manually. Network widths, learning rates, batch sizes, update frequencies, exploration/noise parameters, and algorithm-specific options are therefore treated as outcomes of the Phase 1 hyperparameter optimisation described in section 5.5. The algorithmic mechanics are introduced in chapter 2, the selection rationale is given in subsection 4.2.5, and the resulting per-algorithm configurations are reported in Appendix B (Table B.2).

5.5 Optuna Hyperparameter Optimisation

Each Phase-1 algorithm is tuned through an independent Optuna study that minimises the deterministic speed-tracking Root Mean Square Error (RMSE) on the evaluation cycle (section 4.4). Optimising the measured task metric rather than the shaped reward avoids selecting configurations that exploit the reward design, so called reward hacking (subsection 4.2.4) [70]. The Phase 2 reward search, however, minimises the composite score J (Equation 5.4). The remaining setup is kept consistent.

Sampler, pruner and storage. The studies use a Tree-structured Parzen Estimator (TPE) sampler (`TPESampler`) with ten random startup trials before the Bayesian phase, and a `MedianPruner` with a warm-up of 2,000,000 timesteps, a reporting interval of 500,000 timesteps and five startup trials. After the warm-up, an evaluation callback runs five deterministic episodes at every reporting interval, computes the pooled RMSE and reports it to the pruner. Trials whose intermediate RMSE is worse than the median of completed trials at the same step are pruned. The study state is persisted to an append-only `JournalStorage` log file, which is crash-safe and can be shared across SLURM-array workers (section 5.6) without a separate database service.

Budget and search spaces. Each trial trains a candidate agent for a total of 4,000,000 timesteps in the vectorised training environment described in section 5.6. A budget of 20 trials per study is allocated for each of the three algorithms, although the actual number of completed trials differs, as explained in section 7.2. The algorithm-specific search spaces are listed in Appendix B, Table B.1.

Outputs. Each study persists two artefacts: `best_params.json` as the resolved best-trial parameters, and `all_trials.csv` as the per-job record. The selected best hyperparameters per algorithm are represented in Appendix B, Table B.2.

5.6 Training Infrastructure

Cluster execution. All training runs are dispatched as SLURM array jobs. Each array worker instantiates the same Optuna study via the shared `JournalStorage` (section 5.5), so multiple Phase-1 trials proceed in parallel without coordination beyond the shared journal log.

Environment vectorisation. At runtime, the custom `EmissionControlEnv` is wrapped in an `SB3 SubprocVecEnv`, which executes n_{envs} instances of the environment in parallel worker processes. n_{envs} is set to 20 during hyperparameter search and multi-seed validation. The seed of each child environment is derived from the run identifier to avoid correlated initial conditions across workers while preserving full reproducibility.

Observation and reward normalisation. The environment is wrapped in `VecNormalize`, which maintains running statistics of observations and rewards. Observations are always normalised (based on the min-max values in Table 5.4); reward normalisation is enabled only for PPO, since the off-policy algorithms would be destabilised by rescaling of past experiences when sampling from their replay buffers. The running statistics are persisted alongside each model checkpoint as `vec_normalize.pkl` so that any downstream evaluation or fine-tuning can reload them and reproduce the same observation distribution.

Checkpointing and provenance. Training is monitored by SB3’s `CheckpointCallback` and a custom callback that produces `vec_normalize.pkl` with a frequency of 100,000 timesteps. At the end of each run, the final model, the final normaliser, the evaluation metrics (`evaluation_metrics.json`) and a record of the training config (`train_config.json`) are written to the run directory. The record of the training config stores the algorithm, the random seed, the active reward weights, the training duration and the override-source paths, which allows downstream analyses.

5.7 Per-Phase Implementation Pipelines

5.7.1 Phase 1 Pipeline

Phase 1 fixes the reward to the speed-only objective (Table 5.6) and is realised as a three-stage pipeline. The stages are decoupled into separate launchers so that they can be dispatched independently as SLURM array jobs:

1. **Hyperparameter optimisation.** One Optuna study is launched per algorithm over the search spaces in Table B.1, minimising deterministic speed-tracking RMSE. Each trial trains a fresh agent for 4,000,000 timesteps on 20 parallel environments, evaluates it on the deterministic six-phase cycle every 500,000 timesteps, and reports the result to a median pruner. The selected hyperparameters and trial history are saved as `best_params.json` and `all_trials.csv`.
2. **Multi-seed validation.** The winning `best_params.json` is trained 10 times with seeds $\{0, \dots, 9\}$, one seed per array task. Each seed run executes 4,000,000 timesteps, checkpoints the model and `vec_normalize.pkl` every 100,000 steps, and writes the final model, normaliser, `train_config.json`, and deterministic `evaluation_metrics.json`.
3. **Best-seed selection.** The per-seed evaluation files are ranked by RMSE_v so the lowest speed-tracking RMSE is selected and ties are broken by lower cumulative NO_x . The corresponding model checkpoint and `vec_normalize.pkl` form the bridge to Phase 2.

5.7.2 Phase 2 Pipeline

Phase 2 reuses the Phase 1 best-seed checkpoint as a fine-tuning warm-start (subsection 4.3.3) and activates the additional NO_x and squared-SOC reward terms from Table 5.6. It is implemented as a three-stage pipeline whose runs can again be dispatched independently as SLURM array jobs:

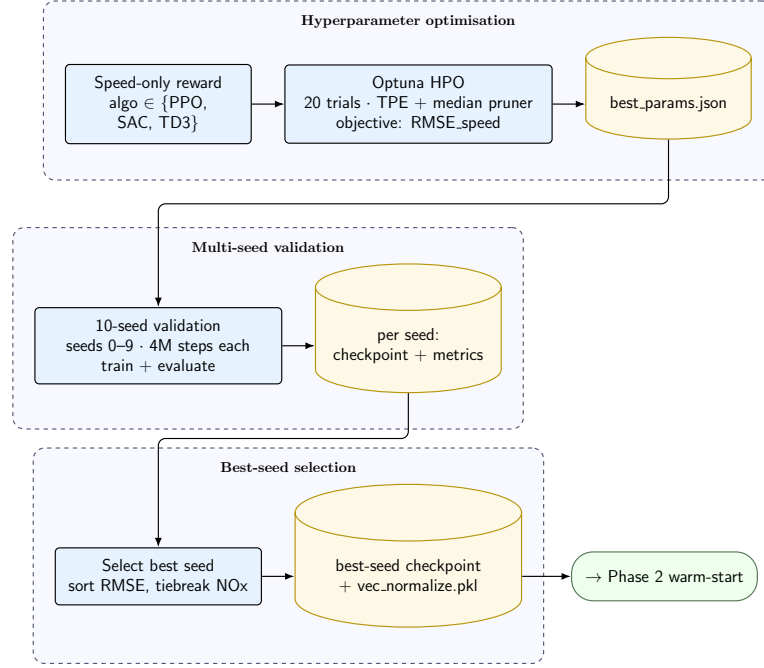


Figure 5.5: Technical pipeline of Phase 1.

1. **Warm-start fine-tune.** Each Phase 2 run loads the Phase 1 `final.zip` checkpoint and its `vec_normalize.pkl` normaliser into a freshly constructed Phase 2 environment. The Phase 1 hyperparameters are reapplied, while `learning_starts` is deliberately omitted from the reload arguments to avoid a second random-action warm-up.
2. **Reward-weight study.** The emission and SOC weights are first explored with a 3×3 grid, $(W_{\text{emi}}, W_{\text{soc}^2}) \in \{0.25, 0.5, 1.0\} \times \{50, 150, 400\}$, with one grid cell per SLURM array task. Each cell fine-tunes the Phase 1 checkpoint for 4,000,000 timesteps, evaluates on the deterministic six-phase cycle, and saves per-cell artefacts. Because this grid samples only nine fixed combinations, it is followed by a per-algorithm Optuna search over $W_{\text{emi}} \in [0.25, 1.25]$ and $W_{\text{soc}^2} \in [25, 250]$, using the same TPE sampler, median pruner and `JournalStorage` setup as Phase 1. Each Optuna trial samples one weight pair, fine-tunes for 4,000,000 timesteps, evaluates every 500,000 timesteps, and minimises the constrained score in Equation 5.4.
3. **Multi-seed validation.** The selected weight pair from `best_params_phase2.json` is validated across seeds $\{0, \dots, 9\}$, one seed per array task. Each seed continues from the same Phase 1 checkpoint, trains for 4,000,000 timesteps over 20 parallel environments, and writes the final model, `vec_normalize.pkl`, `train.config.json`, and deterministic `evaluation_metrics.json`.

Composite score for multi-objective optimisation. The Optuna reward-weight score used in Stage 2 is

$$J = M_{\text{NO}_x, \text{tp}}^{\text{eval}} + \lambda_{\text{RMSE}} \max(0, \text{RMSE}_v - r_{\text{RMSE}})^2 + \lambda_{\text{SOC}} \max(0, \max_t |\Delta \text{SOC}_t| - r_{\text{SOC}})^2, \quad (5.4)$$

where $M_{\text{NO}_x, \text{tp}}^{\text{eval}}$ is the cumulative tailpipe NO_x mass in grams over the deterministic evaluation cycle. It is obtained from the instantaneous surrogate output $\dot{m}_{\text{NO}_x, \text{tp}, t}$ as

$$M_{\text{NO}_x, \text{tp}}^{\text{eval}} = \sum_{t=0}^{T-1} \dot{m}_{\text{NO}_x, \text{tp}, t} \Delta t, \quad (5.5)$$

and corresponds to the evaluation metric stored as `total_nox_g`, not the instantaneous g/s LSTM output. The term RMSE_v is the evaluation speed-tracking RMSE in km/h, and $\max_t |\Delta \text{SOC}_t|$ is the peak SOC drift over the cycle. The fixed constants are $\lambda_{\text{RMSE}} = 20$, $\lambda_{\text{SOC}} = 1000$, $r_{\text{RMSE}} = 5$ km/h, and $r_{\text{SOC}} = 0.05$; lower scores are better.

The composite score combines three design elements:

- **Scalarisation.** The simultaneously considered objectives are transformed into a single scalar objective that Optuna can minimise [71].
- **ϵ -constraint formulation.** Cumulative tailpipe NO_x remains the primary objective, while speed-tracking error and SOC drift are treated as constraints with tolerances r_{RMSE} and r_{SOC} [72].
- **Quadratic penalties.** Constraint violations outside these tolerances are penalised quadratically, so feasible deviations are ignored while larger violations become increasingly costly.

Together, these elements make cumulative tailpipe NO_x the primary objective and treat speed tracking and charge sustainment as soft constraints. Within the tolerances, the penalties vanish; outside them, they grow quadratically. The tolerances are set relative to Phase 1: its best seeds achieve 3.1–3.5 km/h RMSE (section 6.1), so $r_{\text{RMSE}} = 5$ km/h leaves a moderate tracking margin. In contrast, $r_{\text{SOC}} = 0.05$ is far below the ~ 0.30 saturation drift observed in Phase 1 and therefore discourages the battery-exploitation behaviour that motivates Phase 2 (subsection 4.3.3).

The two coefficients λ_{RMSE} and λ_{SOC} exist only to place the three terms on a common scale. The cumulative NO_x values being minimised are on the order of a few grams, so clear constraint violations must add comparable or larger costs; otherwise, the NO_x term would dominate and Optuna could accept poor tracking or excessive SOC drift. The difficulty is that the two penalised quantities are numerically very different: a speed-error excess is measured in km/h and squares to values of order 1 to 10, whereas a SOC-drift excess is a small fraction and squares to values of order 10^{-3} to 10^{-2} . A shared multiplier would therefore render the SOC term negligible. The choice $\lambda_{\text{RMSE}} = 20$ and $\lambda_{\text{SOC}} = 1000$ offsets this scale gap: at a clear violation each term becomes large enough to steer the search away from infeasible weights. For example, a 2 km/h tracking excess adds $20 \cdot 2^2 = 80$ units, while a 0.10 SOC-drift excess adds $1000 \cdot 0.1^2 = 10$ units. The larger SOC coefficient merely compensates for its far smaller squared scale; it does not make SOC the more important constraint. The asymmetry in the resulting magnitudes (80 versus 10) is intentional: RMSE acts as a hard wall against catastrophic forgetting, while the softer SOC term allows more flexibility.

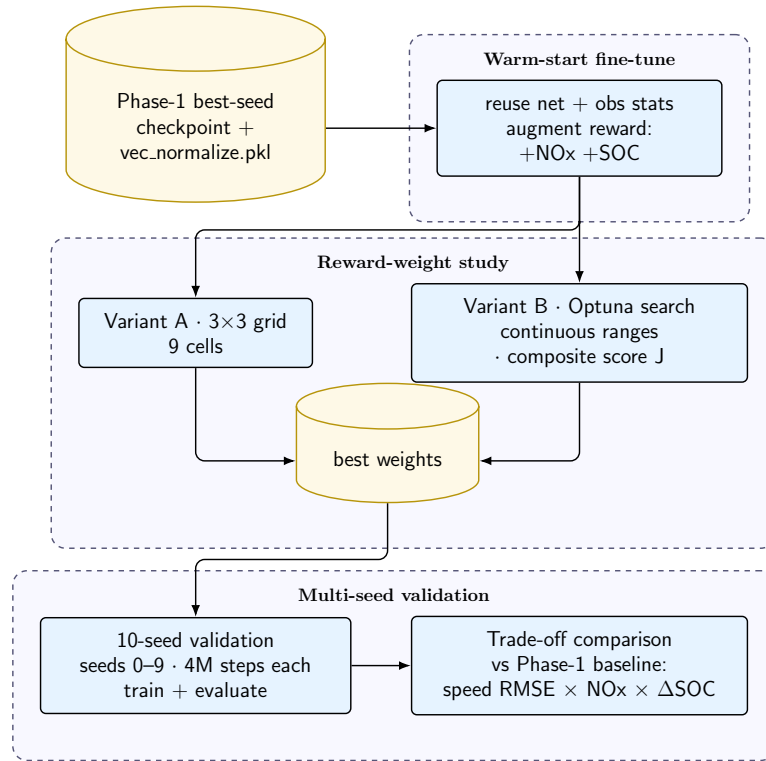


Figure 5.6: Technical pipeline of Phase 2.

Chapter 6

Results

This chapter presents the experimental results and analyses the findings from the two-stage training and evaluation pipeline. It is divided into Phase 1, which comprises speed-tracking-only experiments used to establish baseline performance with respect to emissions and battery State of Charge (SOC) behaviour, and Phase 2, in which additional reward terms for emission reduction and SOC maintenance are activated and compared against the Phase 1 baselines. Each phase analysis first provides a summary of the results, then discusses the outcomes at the algorithm level, and concludes by summarising the overall findings. The chapter further examines the resulting operating-point shifts on the measured Nitrogen Oxides (NO_x) and Brake-Specific Fuel Consumption (BSFC) maps and concludes with a zero-shot Worldwide Harmonised Light Vehicles Test Cycle (WLTC) comparison against the rule-based benchmark controller to assess generalisation beyond the staircase evaluation cycle.

6.1 Phase 1

The outcome of Phase 1 includes a set of trial runs orchestrated by Optuna to identify a single hyperparameter combination for each Reinforcement Learning (RL) agent that is best suited for the current objective: speed tracking. For each algorithm, the best hyperparameter configuration was selected based on the lowest Root Mean Square Error (RMSE) between the actual speed and the target speed trajectory on the evaluation cycle, rather than on the highest episodic reward. This allows to optimise towards the real objective (speed tracking) instead of optimising towards a proxy (reward function which encodes speed tracking). The best configuration was subsequently re-trained 10 times with different seeds and identical hyperparameters in order to evaluate training variance. The resulting training- and evaluation-metric distributions are reported as mean $\pm$ standard deviation over the 10 seeds. The model with the lowest evaluation RMSE among the 10 seeds is retained and represents the starting checkpoint for Phase 2.

The Phase 1 Optuna trial histories are only treated as selection diagnostics and are therefore collected in section B.2. The main text focuses on the selected configurations, their seed-wise retraining behaviour, and the resulting evaluation performance.

The aggregated Phase 1 evaluation statistics across all 10 seeds per algorithm are reported in Table 6.1. Three values stand out:

- **Velocity.** All algorithms achieve a RMSE under 5km/h and a MAE under approximately 2.2km/h, confirming that the speed-tracking sub-task is solvable by all three approaches.
- **Nitrogen Oxides.** The mean NO_x emission per evaluation cycle (without an NO_x penalty in the reward) is between 48 and 95 g, corresponding to roughly 1,200–2,300 mg/km when normalised by the cycle distance. This is 15–30 $\times$ above the Euro 6 limit of 80 mg/km of NO_x and constitutes the upper bound that Phase 2 must reduce.
- **State of Charge.** The mean ΔSOC is +0.15–+0.25. Most seeds saturate the battery at the upper SOC clip ($\text{SOC}_{\text{final}} = 0.9999$), while a small subset of seeds depletes the battery. Both endpoints yield similar speed rewards but very different powertrain behaviour.

Metric (evaluation cycle)	PPO	TD3	SAC
Cumulative reward	3,428 $\pm$ 142	3,486 $\pm$ 44	3,503 $\pm$ 57
RMSE speed [km/h]	4.19 $\pm$ 0.98	3.81 $\pm$ 0.29	3.45 $\pm$ 0.43
Mean Absolute Error (MAE) speed [km/h]	2.19 $\pm$ 1.24	1.90 $\pm$ 0.58	1.54 $\pm$ 0.75
Cumulative NO_x [g]	48.3 $\pm$ 29.0	95.4 $\pm$ 100.8	63.8 $\pm$ 59.8
Cumulative fuel [g]	79.6 $\pm$ 27.0	122.6 $\pm$ 56.9	113.5 $\pm$ 38.5
ΔSOC	+0.247 $\pm$ 0.157	+0.146 $\pm$ 0.260	+0.191 $\pm$ 0.280
Best-seed RMSE [km/h]	3.16	3.51	3.11
Worst-seed RMSE [km/h]	6.31	4.41	4.58
Mean wall-clock per seed [h]	8.05	13.03	15.28

Table 6.1: Phase 1 evaluation metrics across the 10 seeds of the best Optuna hyperparameter configuration per algorithm. Bold entries mark the best of the three algorithms per metric. Cumulative NO_x , fuel and ΔSOC are reported as the empirical baselines that Phase 2 must improve. Wall-clock time refers to a 4M-timestep training run on the cluster at $n_{\text{envs}} = 20$.

The following sections present the results for each algorithm in Phase 1 and Phase 2, respectively. The Optuna hyperparameter and reward searches are summarised through their most important findings for each selected configuration, while the complete histories of the 20 valid runs per search are shown in section B.2 and section C.1. The main text focuses on the key aspects of the selected configuration, analyses the stability of its 10-seed validation, and then examines the control strategy of the single best seed per algorithm in terms of speed-tracking performance, NO_x emissions, and SOC evolution.

6.1.1 PPO

Hyperparameter optimisation trials. Of the 70 sampled Optuna trials, only 17 ran to completion, while the remaining 53 were terminated early by the median pruner. Optuna ultimately concentrated the search on a narrow learning-rate band of approximately $10^{-5} - 3 \times 10^{-5}$, with the best-performing configuration at 1.28×10^{-5} . Median pruning was therefore important for allocating computing resources to promising trajectories rather than continuing trials that were unlikely to recover. Since trials with higher learning rates were consistently pruned, the result suggests that PPO requires an exceptionally low learning rate in this environment. Most non-pruned runs converged near the theoretical reward maximum of 3,600, with the steepest learning progress between roughly 500,000 and 1,500,000 steps and a plateau reached around 3,000,000 steps. The selected

configuration combines this unusually small learning rate with $n_{\text{epochs}} = 11$ over 2,048 environment steps per update and a [256, 256] Multilayer Perceptron (MLP), producing slow but stable policy updates. The Optuna trial history for the final 20 trial runs is provided in Figure B.1.

Seed runs. The 10 seed runs for this hyperparameter configuration are shown in Figure 6.1. Although some seeds learn faster than others, all 10 converge to cumulative training rewards between 3,351 and 3,438. Since the hyperparameter configuration is identical across runs, these differences are attributable to random initialisation. This is the most stable seed-wise behaviour of the three algorithms: PPO learns more slowly, but its policies remain stable throughout the 4,000,000 training steps.

Overall, the high episodic rewards achieved across all 10 seeds yield the average speed-tracking trajectory shown in Figure 6.2. The mean RMSE of 4.19 ± 0.98 km/h and MAE of 2.19 ± 1.24 km/h confirm that PPO can track the reference, although its per-seed dispersion is the largest of the three algorithms. The best individual seed reaches 3.16 km/h RMSE, comparable to the best SAC and TD3 seeds, while a single outlier (seed 8) settles at 6.31 km/h RMSE with 135 g of fuel and 110 g of NO_x .

The remaining aggregated evaluation metrics are reported in Table 6.1. The mean cumulative fuel consumption is 79.6 g and the mean cumulative NO_x is 48.3 g over the evaluation cycle, while the mean $\Delta\text{SOC} = +0.247$ reflects the strong tendency of PPO seeds to charge the battery up to the upper clip at 0.9999. The implications of these values for Phase 2 are discussed in subsection 6.1.4.

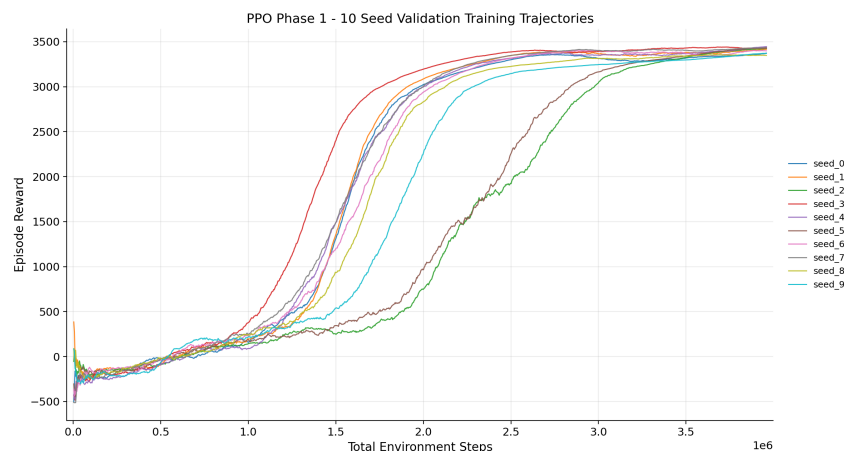


Figure 6.1: Training progress across different PPO seeds in Phase 1; curves are smoothed via moving average over 80 episodes.

The best seed. The best PPO checkpoint is seed 3. Its action summary in Figure 6.3 shows a high-rpm strategy. The engine is almost exclusively on, while the Internal Combustion Engine (ICE) speed is concentrated at 4,000 rpm basically across the full target-speed range. The Electric Motor 2 (EM2) torque remains close to zero with only mild positive torque in the 0–80 Nm range, the fuel command is tightly concentrated around 17–23 mg, and the brake is essentially unused. This fixed operating point produces the cleanest best-seed policy of all three algorithms: 5.57 g cumulative

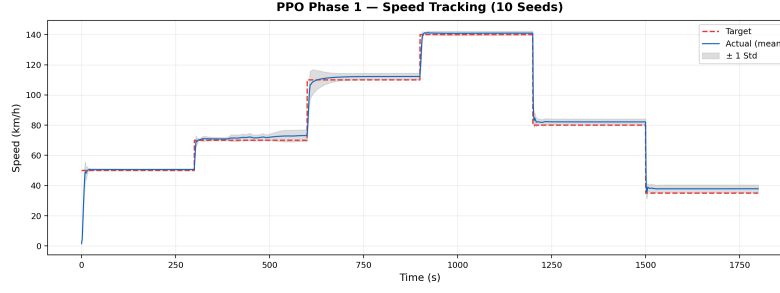


Figure 6.2: Evaluation speed-tracking trajectory for PPO in Phase 1.

NO_x and 74.46 g fuel at 3.16 km/h RMSE. The result also shows that high engine speed alone is not the dominant NO_x driver; the integrated fuel seems more decisive. The resulting velocity, SOC, and tailpipe NO_x emissions are depicted in Figure 6.4.

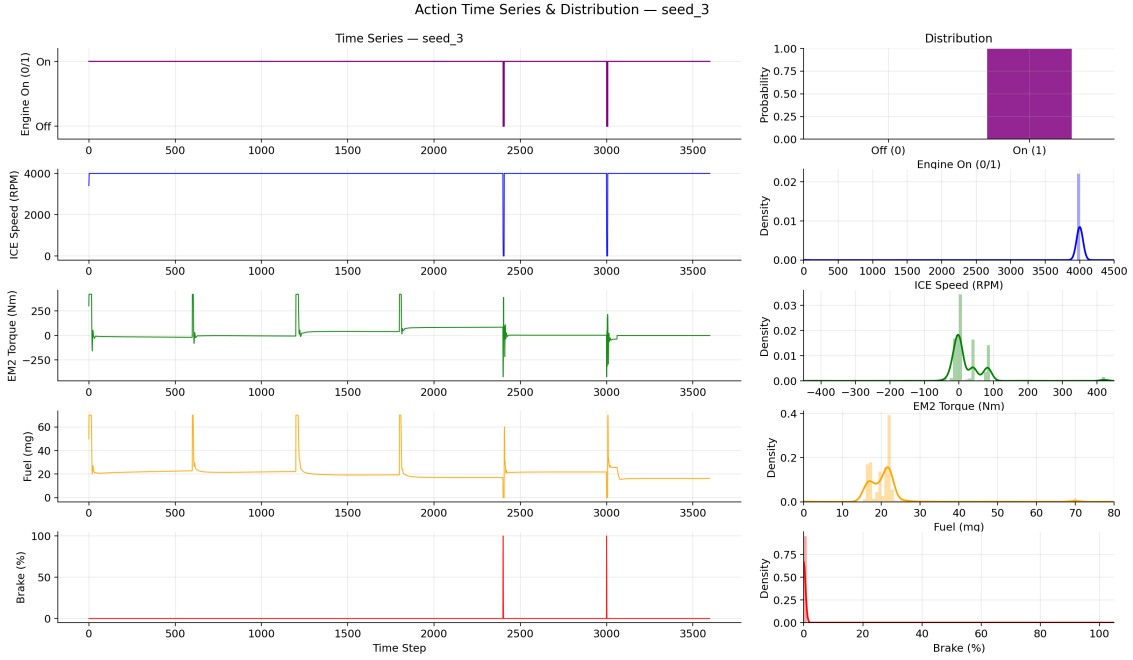


Figure 6.3: Action summary for the best PPO checkpoint, seed 3, in Phase 1. The left panels show the PPO agent’s actions during the evaluation cycle; the right panels show the corresponding densities.

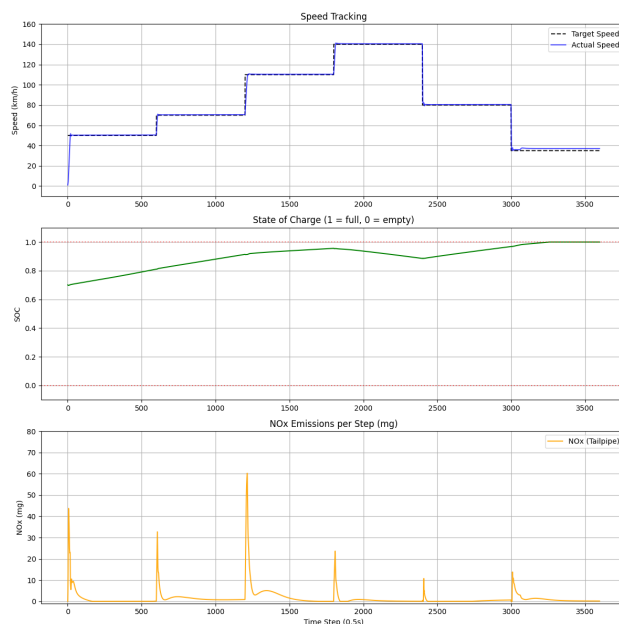


Figure 6.4: Evaluation results for the best PPO checkpoint, seed 3, in Phase 1. The panels show speed tracking, the SOC trajectory, and tailpipe NO_x emissions over the evaluation cycle.

6.1.2 TD3

Hyperparameter optimisation trials. TD3’s Optuna study comprised 45 sampled trials, of which 30 completed, 8 were pruned, and the remaining 7 failed; the corresponding trial history is shown in Figure B.2. The best configuration (Table B.2 in Appendix B) features a learning rate of 1.79×10^{-4} , a batch size of 1,024, a [128, 128] MLP, and most notably a `policy_delay` of 1. Setting the policy delay to 1 effectively switches off the delayed actor-update mechanism that distinguishes Twin Delayed Deep Deterministic Policy Gradient (TD3) from its predecessor Deep Deterministic Policy Gradient (DDPG). The twin critics with clipped Gaussian target-policy smoothing ($\sigma_{\text{target}} = 0.116$, $\text{clip} = 0.7$) and the exploration noise ($\sigma_{\text{action}} = 0.144$) remain active. In this environment, the delayed actor update is therefore not what stabilises learning, an observation that is discussed further in section 7.1.

Seed runs. The seed-wise learning curves (Figure 6.5) exhibit a fast initial climb to their plateau within the first 500,000 steps. A single seed was not able to learn that fast initially and could only find a policy to cumulate the amount of reward at approximately 1,500,000 steps. Different to Proximal Policy Optimisation (PPO) however, is that the runs did not keep their stability once they reached the plateau. Instead, some of them drift off to a worse policy and then recover again or end with a slightly lower episode reward.

TD3 produces the tightest RMSE distribution of the three algorithms (3.81 ± 0.29 km/h, Figure 6.6) but at the same time the widest energy-side distribution (95.4 ± 100.8 g NO_x , 122.6 ± 56.9 g fuel). The mean ΔSOC of $+0.146$ hides a bimodal underlying distribution: seeds 0, 6, and 8 deplete

the battery ($\Delta\text{SOC} = -0.12, -0.49, -0.03$ respectively), whereas the remaining seven seeds saturate it. Both groups can track the reference speed similarly well, which explains why the RMSE distribution remains narrow while the energy- and emission-side distributions are much wider.

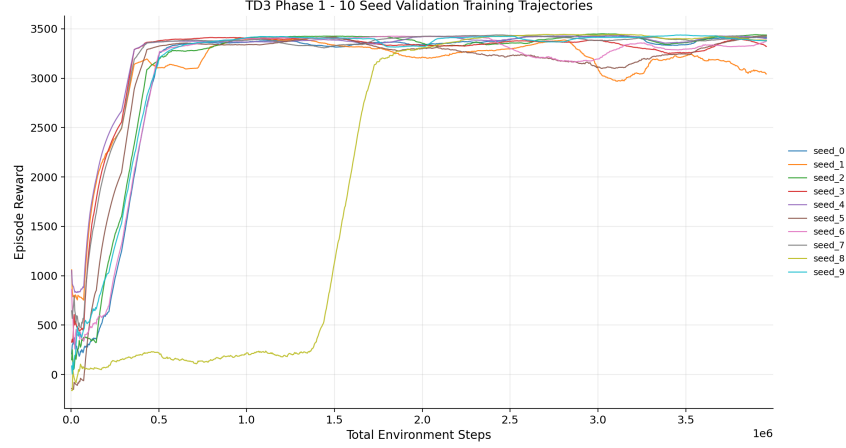


Figure 6.5: Training progress across different TD3 seeds in Phase 1; curves are smoothed via moving average over 80 episodes.

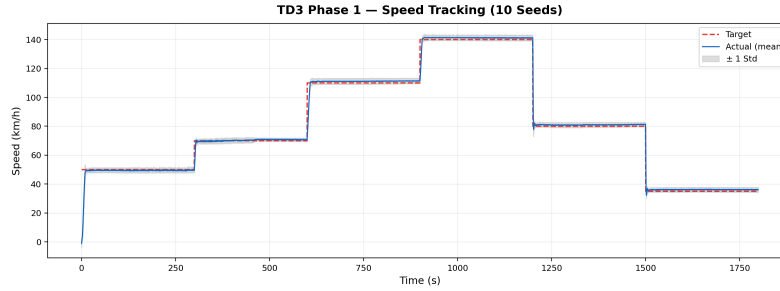


Figure 6.6: Evaluation speed-tracking trajectory for TD3 in Phase 1.

The best seed. The best TD3 checkpoint is seed 1. Its action summary in Figure 6.7 shows a lower-rpm policy than PPO, combined with a bimodal fuel distribution and highly volatile EM2 usage. The engine is again almost exclusively on, while the ICE speed is concentrated mainly around 900–1,300 rpm and 2,000–2,500 rpm. The 4,000 rpm region is used only when the target speed remains in the evaluation cycle increases. During steady cruising periods, where the target speed remains constant, the EM2 torque is approximately 0 Nm. In the first staircase segment (up to step 600), it is exclusively negative at around -165 Nm. In the fourth segment, at the highest target velocity of 140 km/h, the EM2 torque becomes highly volatile and alternates between approximately $+330$ Nm and -250 Nm every other time step (0.5 s). Fuel injection is clearly bimodal, with one cluster around 10–20 mg and another around 50–70 mg. This high-fuel cluster gives TD3 the worst

best-seed fuel and emission values: 163.94 g fuel and 73.42 g cumulative NO_x at 3.52 km/h RMSE. The resulting velocity, SOC, and tailpipe NO_x emissions are depicted in Figure 6.8.

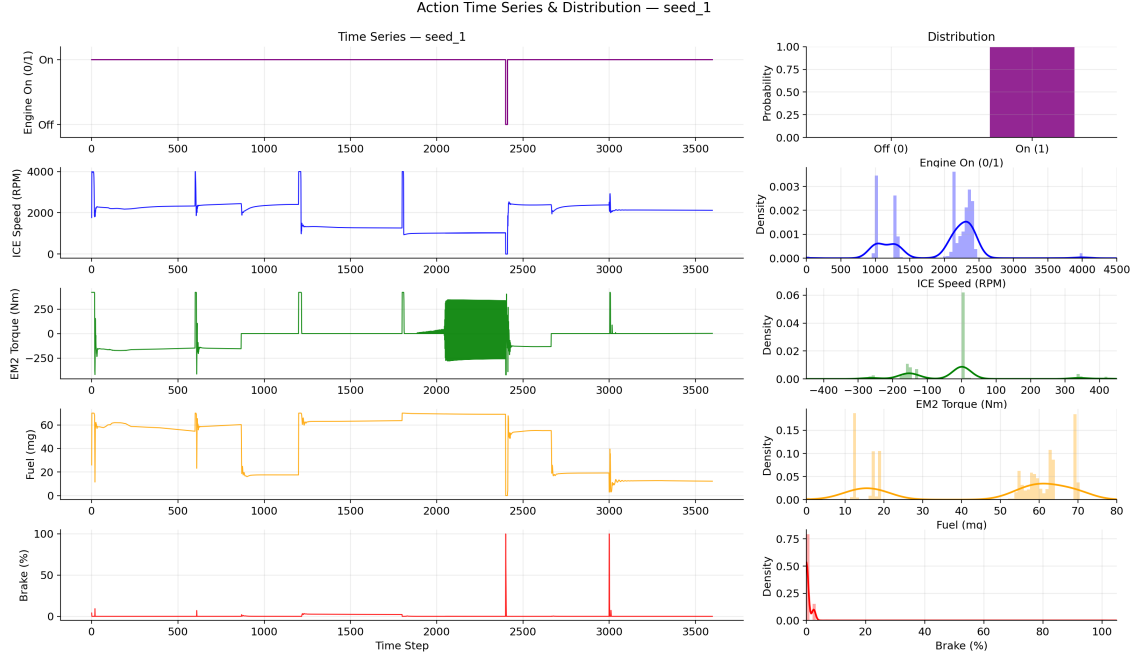


Figure 6.7: Action summary for the best TD3 checkpoint, seed 1, in Phase 1. The left panels show the TD3 agent’s actions during the evaluation cycle; the right panels show the corresponding densities.

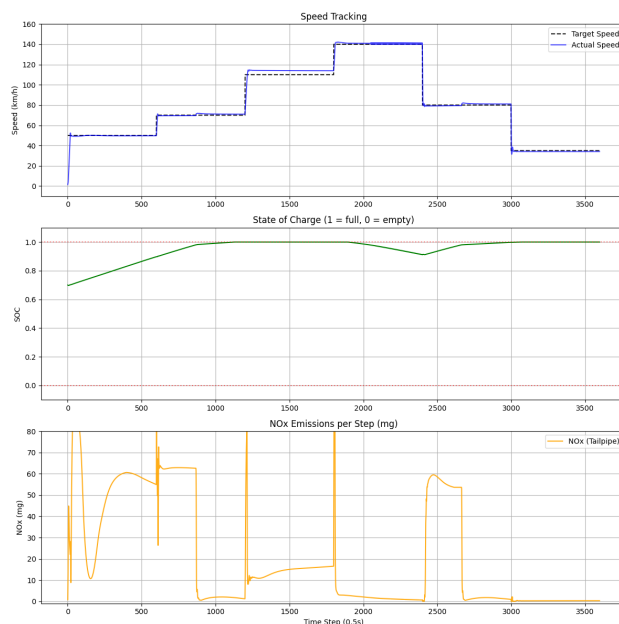


Figure 6.8: Evaluation results for the best TD3 checkpoint, seed 1, in Phase 1. The panels show speed tracking, the SOC trajectory, and tailpipe NO_x emissions over the evaluation cycle.

6.1.3 SAC

Hyperparameter optimisation trials. Of SAC’s 44 Optuna trials, 20 completed successfully, 12 were pruned, and 12 failed; the corresponding trial history is shown in Figure B.3. The best configuration uses a learning rate of 9.20×10^{-4} , a batch size of 512, a three-layer [128, 128, 128] MLP, $\tau = 0.0080$, and automatic entropy-coefficient tuning (`ent_coef = "auto"`); the full configuration is documented in Table B.2 in Appendix B.

Seed runs. The seed-wise learning curves (Figure 6.9) reach $\approx 3,500$ within 300,000 steps and remain close to that level for the remainder of training, apart from two temporary dips around 3,300,000 steps. SAC is more stable than TD3 because its reward trajectories are tighter, and it learns substantially faster than PPO; the two observed reward dips recover successfully and therefore do not indicate lasting instability. The resulting evaluation RMSE (3.45 ± 0.43 km/h) has both the lowest mean and the lowest per-seed variance of the three algorithms (see Table 6.1), making SAC the most reliable speed tracker in Phase 1.

The best seed. The best SAC checkpoint is seed 3 and is also the lowest-RMSE policy of Phase 1 across all algorithms. Its action summary in Figure 6.11 shows a more modulating strategy than PPO. The engine is active for most of the cycle, but it is switched off during the final staircase segment, making SAC the only best-seed policy that actively turns the engine off for a non-trivial part of the evaluation cycle. The ICE speed therefore contains a 0 rpm mode and a broad active range from approximately 2,400 to 4,000 rpm. The EM2 torque is mostly close to zero, with a nega-

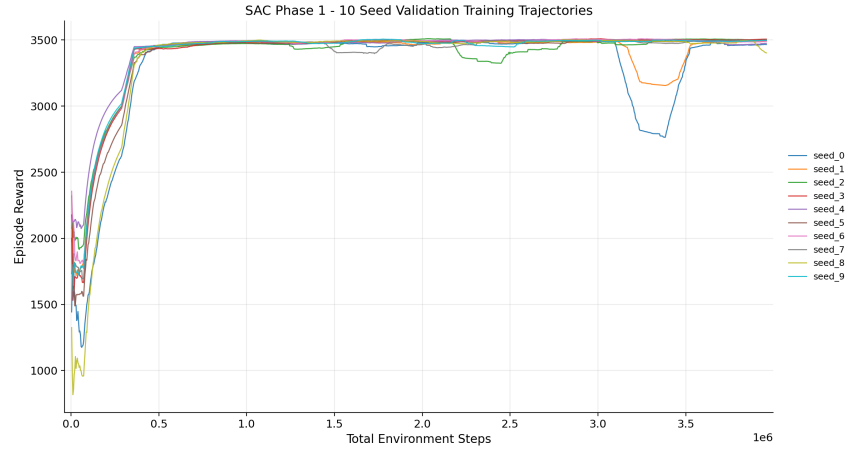


Figure 6.9: Training progress across different SAC seeds in Phase 1; curves are smoothed via moving average over 80 episodes.

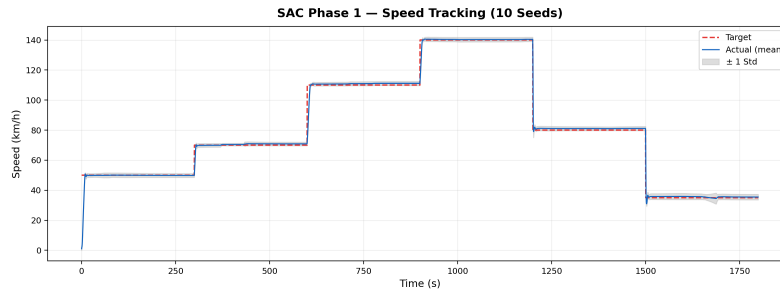


Figure 6.10: Evaluation speed-tracking trajectory for SAC in Phase 1.

tive presence at around -50 to -100 Nm and a smaller positive band between 0 and approximately $+70$ Nm. In the engine-off segment, the agent alternates between these positive torque values every other step to propel the vehicle while the ICE is off. Fuel injection is zero when the engine is off, due to the engine-switch cooldown through the post-posed action shield described in section 5.3; otherwise, it is concentrated around 30–40 mg up to 60 mg. This learned behaviour yields the best speed tracking (3.11 km/h RMSE and 0.89 km/h MAE), but it comes at higher cumulative fuel (109.31 g) and NO_x (48.94 g) than the best PPO seed. The resulting velocity, SOC, and tailpipe NO_x emissions are depicted in Figure 6.12.

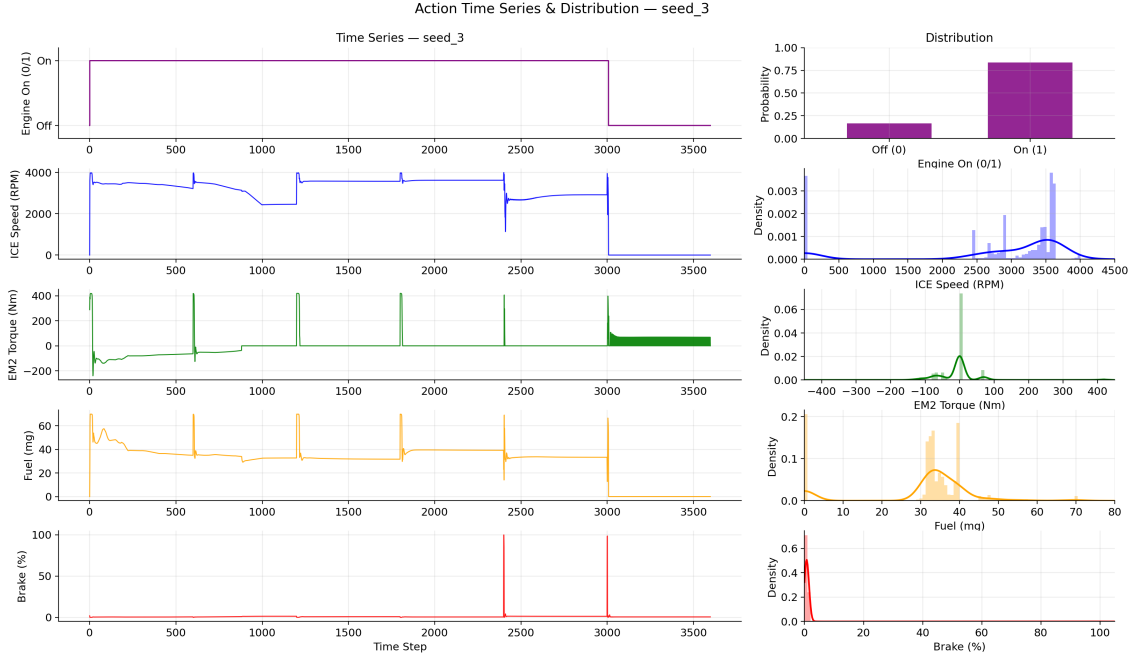


Figure 6.11: Action summary for the best SAC checkpoint, seed 3, in Phase 1. The left panels show the SAC agent’s actions during the evaluation cycle; the right panels show the corresponding densities.

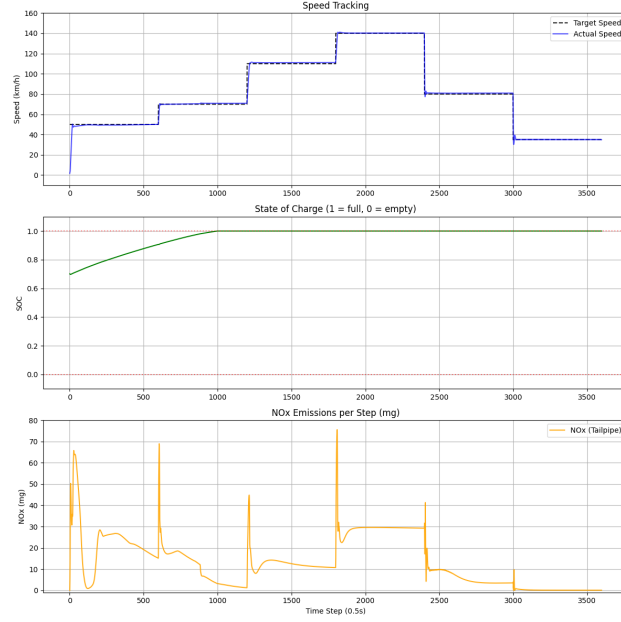


Figure 6.12: Evaluation results for the best SAC checkpoint, seed 3, in Phase 1. The panels show speed tracking, the SOC trajectory, and tailpipe NO_x emissions over the evaluation cycle.

6.1.4 Summary

Training convergence. The three algorithms reach essentially indistinguishable best-seed speed-tracking quality: the minimum-RMSE seeds of PPO, SAC and TD3 fall within 3.11–3.51 km/h. The remaining error is dominated by the unavoidable transient overshoot at each step change in the staircase evaluation cycle, while the steady-state tracking on each plateau is much tighter. Figure 6.13 also shows the typical sample-efficiency difference between the algorithms: SAC and TD3 reach their reward plateau earlier than PPO, whereas PPO converges more slowly under the same environment-interaction budget. By the end of training, however, all three algorithms operate close to the theoretical reward ceiling, so the central Phase 1 finding is not whether speed tracking can be learned, but which uncontrolled side effects appear once only speed tracking is rewarded.

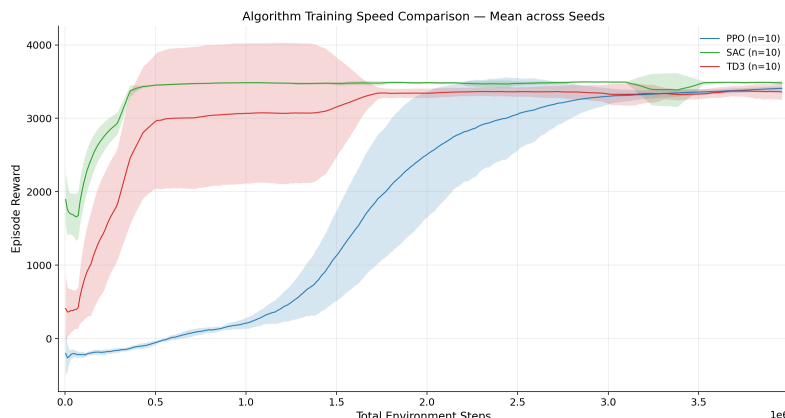


Figure 6.13: Comparison of seed-averaged training progress across all Phase 1 algorithms. Curves are smoothed with a moving average over 80 episodes, and shaded bands represent $\pm$ one standard deviation across the 10 seed curves at each x-grid point.

Equal tracking but divergent powertrain use. To look behind the seed-averaged tracking quality, three cross-algorithm visualisations (Figure 6.14–Figure 6.16) dissect how the 30 policies differ despite near-identical reward. The Pareto view in Figure 6.14 plots speed-tracking accuracy against cumulative NO_x . Speed tracking is tightly clustered (RMSE 3.1–6.3 km/h), whereas cumulative NO_x spans more than two orders of magnitude (2.6–354 g). The non-dominated low- NO_x front is occupied almost exclusively by battery-depleting policies — for example SAC seed 8 attains only 2.6 g NO_x but at $\Delta\text{SOC} = -0.64$ — while the majority of seeds saturate the battery upward ($\Delta\text{SOC} \approx +0.30$) and form a distinctly higher- NO_x band. The parallel-coordinate view in Figure 6.15 confirms the same degeneracy across every metric at once: reward, RMSE and MAE stay nearly flat from seed to seed, whereas fuel, NO_x and ΔSOC fan out strongly. Equal speed-tracking reward therefore masks fundamentally different energy-management strategies, and a low NO_x value in Phase 1 is frequently the by-product of draining the unconstrained battery rather than of a charge-sustaining clean-control policy. This degeneracy is the direct motivation for the explicit SOC and NO_x reward terms introduced in Phase 2.

Mechanistic origin in the engine map. Where the Pareto and parallel-coordinate views establish *that* the policies diverge, the engine-map occupancy in Figure 6.16 explains *where* that divergence originates. Overlaying the pooled engine-on operating points on the measured EA189 BSFC map reveals three distinct regimes: PPO and TD3 concentrate the engine near the 4,000 rpm ceiling whereas SAC spreads more broadly across low-to-mid engine speeds. None of the algorithms consistently utilises the ICE at the most efficient measured operating point, which is consistent with the elevated fuel total. Efficiency was never rewarded, so each policy settles wherever speed tracking is convenient. The simulator additionally caps ICE speed at 4,000 rpm, whereas the measured map extends to 4,500 rpm, leaving an unused efficiency margin. This margin is genuine headroom for Phase 2, but it must be realised by operating the engine more efficiently, not by allowing the controller to lower emissions through battery depletion.

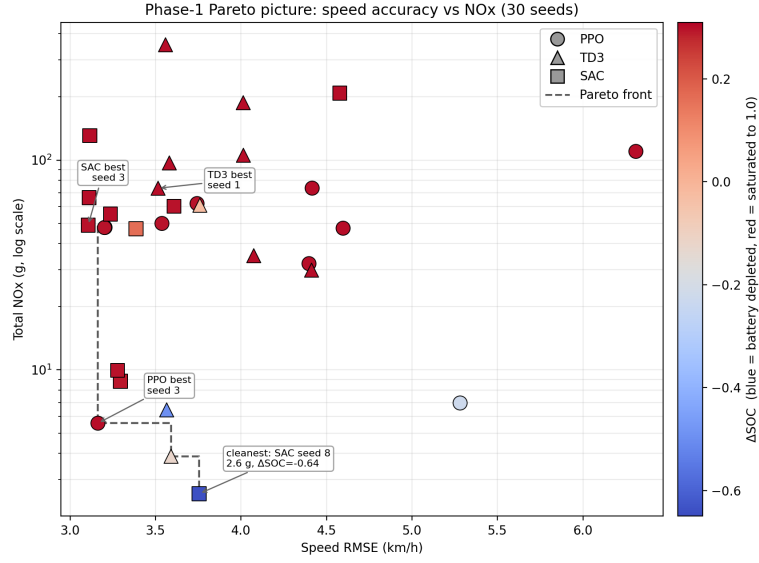


Figure 6.14: Speed-tracking accuracy versus NO_x across all 30 Phase 1 seeds on the deterministic staircase evaluation cycle. Marker shape encodes the algorithm, fill colour encodes the net battery-state change ΔSOC , and the dashed step line denotes the non-dominated front for jointly minimising RMSE and NO_x .

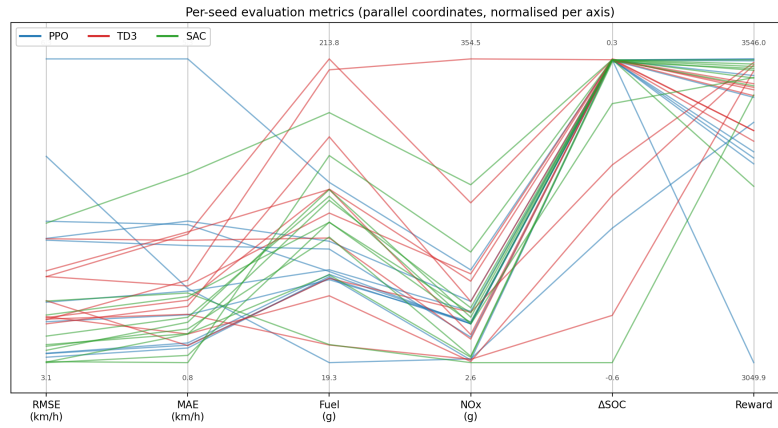


Figure 6.15: Per-seed evaluation metrics in parallel coordinates. Each polyline represents one seed, each axis is normalised independently to its own min-max range with the true extrema printed at the axis ends, and colour encodes the algorithm.

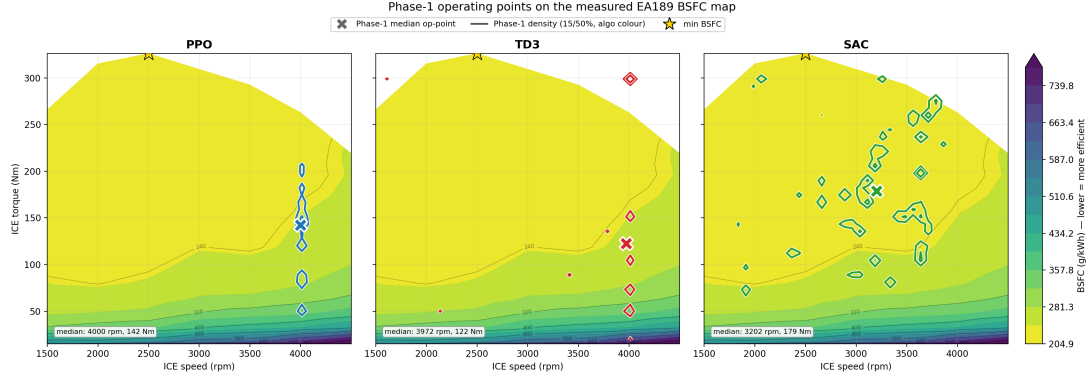


Figure 6.16: Learned engine operating points over the measured EA189 brake-specific fuel-consumption (BSFC) map. Filled contours show the measured BSFC map, labelled black lines show iso-BSFC contours, X markers show median operating points, the gold star marks the most efficient measured point, and coloured density contours show the engine-on, positive-torque operating regions pooled over the 10 seeds of each algorithm. The two coloured contours outline where each policy spent most of its time: the inner (50%) line encloses its most-frequented rpm–torque region, the outer (15%) line a wider surrounding band.

Best-seed baselines and the SOC bias. The checkpoint transferred to Phase 2 is selected by the lowest evaluation RMSE within each algorithm, so the relevant reference is the concrete best-seed operating point in Table 6.2 rather than only the seed-averaged values in Table 6.1. All three selected checkpoints saturate the battery upward with $\Delta\text{SOC} \approx +0.30$; none is charge-sustaining before the Phase 2 reward terms are introduced, which makes the battery-exploitation bias identified above an explicit property of the transferred policies, not only of the seed population. The three baselines nonetheless differ strongly in remaining headroom: PPO seed 3 is already unusually clean, SAC seed 3 is the best speed tracker but carries substantially higher NO_x , and TD3 seed 1 leaves the largest fuel- and emission-reduction margin.

Algorithm	Seed	RMSE (km/h)	MAE (km/h)	NO_x (g)	Fuel (g)	ΔSOC
PPO	3	3.163	1.001	5.57	74.46	+0.299
TD3	1	3.515	1.621	73.42	163.94	+0.300
SAC	3	3.106	0.891	48.94	109.31	+0.299

Table 6.2: Phase 1 best seeds on the deterministic evaluation cycle. The selected seeds are the checkpoints with the lowest evaluation RMSE per algorithm. Bold entries mark the best value in each metric. These specific seeds are used solely as the starting points for Phase 2; they are not the statistical baseline for evaluating whether Phase 2 improves speed tracking, reduces NO_x , and limits SOC deviation. That comparison should use the 10-seed averages reported in Table 6.1.

6.2 Phase 2

Phase 2 evaluates whether the best Phase 1 speed-tracking checkpoints can be converted into charge-sustaining, low- NO_x controllers without a meaningful loss of tracking accuracy. The fine-tuning protocol follows subsection 4.3.3 and subsection 5.7.2; the Phase 1 best-seed metrics in Table 6.2 define the concrete per-agent starting checkpoints. Because each agent is then re-trained over 10 seeds from that single checkpoint, the resulting Phase 2 10-seed statistics are compared like-for-like against the Phase 1 10-seed means of Table 6.1; the best-seed metrics fix only the starting points and are not the statistical baseline for the comparison.

Augmented reward. Phase 2 uses the reward formulation of Equation 5.2 with the emission and quadratic SOC terms active. The results below therefore vary only the two multi-objective weights ($W_{\text{emi}}, W_{\text{soc}^2}$); the speed-tracking and mechanical shaping terms remain fixed as specified in Table 5.6.

Weight selection by Optuna. The two active Phase 2 weights are selected with the constrained Optuna objective defined in Equation 5.4, which minimises cumulative NO_x inside the admissible tracking and charge-drift region and penalises violations outside it. Each agent was tuned over 20 valid trials; the selected trials, scores and evaluation metrics are listed in Table 6.3. The reward-weight trial histories are treated as selection diagnostics and are therefore collected in section C.1. The selected configuration was then re-trained over 10 seeds, and all reported statistics are the mean $\pm$ standard deviation over those seeds.

Algorithm	Trial	W_{emi}	W_{soc}	Score J	RMSE (km/h)	NO_x (g)	$\max \Delta \text{SOC} $
PPO	trial 1	1.185	92.86	4.111	3.16	4.11	0.040
TD3	trial 4	1.010	64.50	3.950	4.01	2.68	0.086
SAC	trial 4	0.495	167.01	2.436	3.68	2.44	0.013

Table 6.3: Best Phase 2 Optuna trial per algorithm. The Optuna score is the constrained scalar objective from Equation 5.4; lower is better and bold marks the best score. RMSE, NO_x and $\max |\Delta \text{SOC}|$ are deterministic single-run evaluation metrics of the selected trial before 10-seed retraining.

Initial reward drop. Because the fine-tune starts from a Phase 1 checkpoint whose reward normalisation and value estimates were based on the speed-only reward, the activation of the emission and SOC penalties causes a steep drop in episodic reward in the beginning of the training before the policy re-stabilises. This is visible as the early dip in every Phase 2 learning curve of the reward-weight search.

The aggregated Phase 2 evaluation statistics across the 10 seeds per algorithm are reported in Table 6.4. For TD3 a second column reports the statistics with its two divergent seeds removed; the reason for this split is detailed in subsection 6.2.2.

Metric (evaluation cycle)	PPO	TD3 (all 10)	TD3 (8/10)	SAC
Cumulative reward	$3,553.16 \pm 2.81$	$3,360.37 \pm 227.04$	$3,461.38 \pm 68.14$	$3,518.40 \pm 44.33$
RMSE speed (km/h)	3.13 ± 0.07	7.41 ± 9.94	3.96 ± 0.47	3.33 ± 0.33
MAE speed (km/h)	0.80 ± 0.08	3.66 ± 3.75	2.21 ± 0.58	1.38 ± 0.81
Cumulative NO _x (g)	4.94 ± 0.72	10.01 ± 4.85	8.65 ± 4.43	4.51 ± 1.46
NO _x intensity (mg/km)	122.42	252.42	212.80	111.03
Cumulative fuel (g)	52.76 ± 1.25	79.79 ± 31.40	79.46 ± 35.60	63.99 ± 20.17
$ \Delta\text{SOC} $	0.029 ± 0.004	0.061 ± 0.032	0.069 ± 0.032	0.010 ± 0.008
max $ \Delta\text{SOC} $	0.038 ± 0.009	0.137 ± 0.058	0.123 ± 0.048	0.021 ± 0.009

Table 6.4: Phase 2 evaluation metrics across the 10 seeds of the best reward-weight configuration per algorithm. Bold marks the best value among the stable candidates (PPO and SAC); TD3 is excluded from the bold comparison due to its instability. The two TD3 columns report all 10 seeds and the statistics with the two divergent seeds (1 and 3) removed, respectively. NO_x intensity is the cumulative NO_x normalised by the ≈ 40 km evaluation cycle distance. Cumulative reward row is diagnostic only; since reward weights differ per algorithm, reward is not directly comparable across them.

6.2.1 PPO

Reward-weight search. PPO’s Optuna study ran 20 valid trials (16 completed, 4 pruned) and settled on the highest emission weight of the three agents, $W_{\text{emi}} = 1.185$, with a moderate $W_{\text{soc}^2} = 92.86$ (Table 6.3). The corresponding reward-weight trial history is provided in Figure C.1. Overall, this produced the agent with the tightest speed-tracking ability with a RMSE of 3.16.

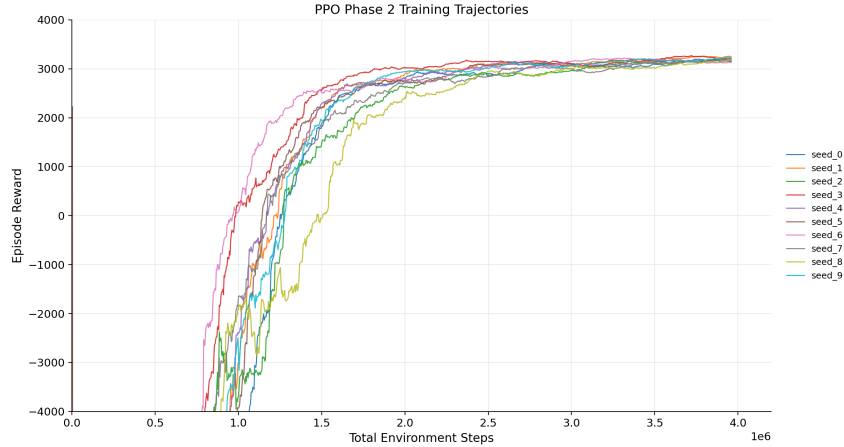


Figure 6.17: Training progress across the 10 PPO seeds in Phase 2. Curves are smoothed via moving average over 80 episodes, and the y-axis is clipped to ± 4000 .

Seed runs. PPO is the most reliable agent in Phase 2. After the initial reward shock, all 10 seeds recover to nearly identical learning curves (Figure 6.17) and the evaluation RMSE collapses to 3.13 ± 0.07 km/h — below the Phase 1 10-seed mean of 4.19 km/h, and with a per-seed standard

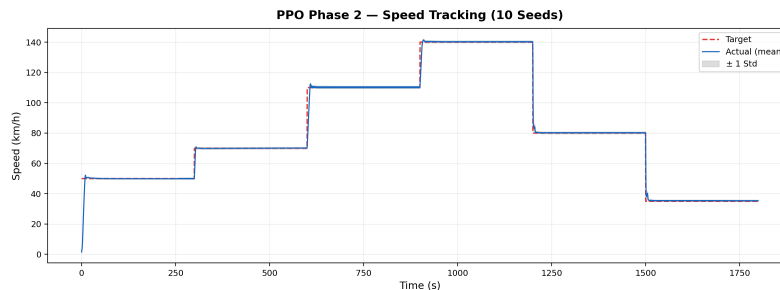


Figure 6.18: Evaluation speed-tracking trajectory for PPO in Phase 2.

deviation more than an order of magnitude tighter than the 0.98 km/h of Phase 1. The speed-tracking trajectory in Figure 6.18 is essentially indistinguishable from the Phase 1 PPO baseline, while the evaluation metrics in Table 6.4 show the energy side transformed: mean cumulative NO_x falls to 4.94 ± 0.72 g (from the Phase 1 10-seed mean of 48.3 g) and fuel to 52.76 ± 1.25 g (from 79.6 g), and the charge drift tightens to $|\Delta\text{SOC}| = 0.029$ from the Phase 1 10-seed mean of +0.247. PPO therefore converts an already-clean but battery-saturating Phase 1 policy into a clean and charge-sustaining one at no cost to tracking accuracy.

The best seed. The best Phase 2 PPO seed is seed 4 with a composite score J of 4.23. This value is determined by the NO_x term only, since the SOC and RMSE constraints are satisfied.

The policy preserves the main qualitative features of the Phase 1 PPO strategy: the engine remains continuously on, the ICE speed still saturates at 4,000 rpm, and mechanical braking is avoided. The difference lies in the EM2 and fuel commands, both of which become more volatile. During the second, third, fifth, and sixth phases of the evaluation cycle, the EM2 torque rapidly switches between approximately -340 and 421 Nm. The density plot therefore forms three clusters: the two torque extremes and an additional cluster around 50 Nm. The fuel-injection command also broadens around 20 mg and now spans roughly 0–20 mg, showing similarly volatile behaviour. These results suggest that the more aggressive torque and fuel modulation is part of the mechanism by which PPO reduces NO_x substantially. Provided that these commands are mechanically feasible and that the surrogate models remain reliable in this operating regime, this behaviour may offer useful insight; however, mechanical feasibility is not validated within the scope of this thesis.

The resulting velocity, SOC, and tailpipe NO_x emissions are depicted in Figure 6.20.

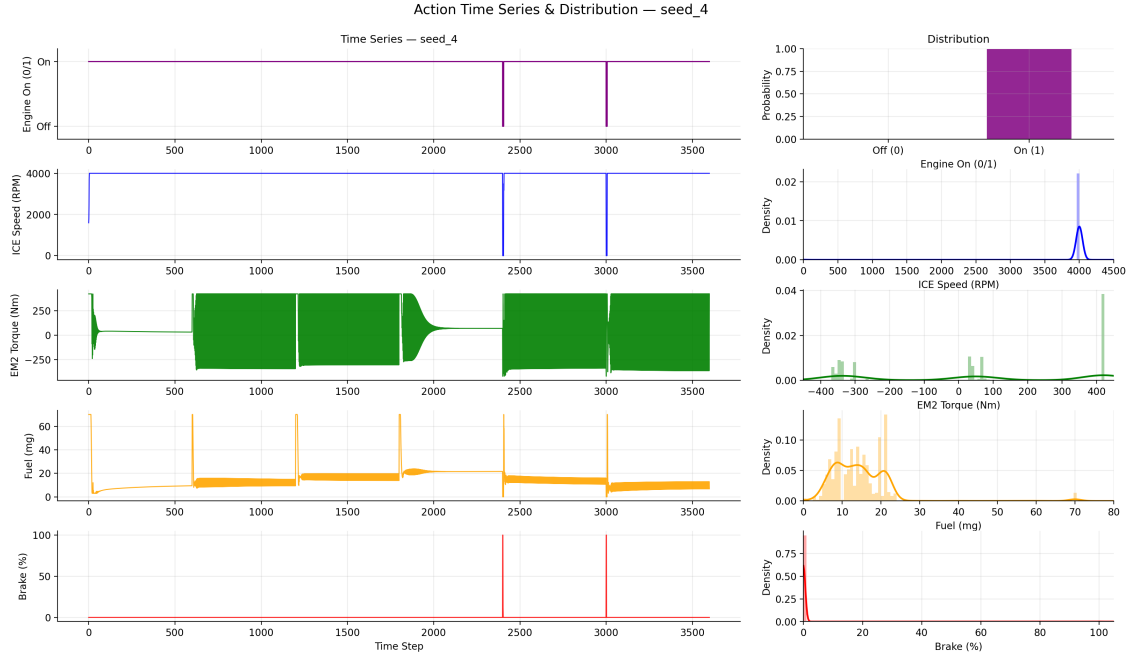


Figure 6.19: Action summary for the best PPO checkpoint, seed 4, in Phase 2. The left panels show the PPO agent's actions during the evaluation cycle; the right panels show the corresponding densities.

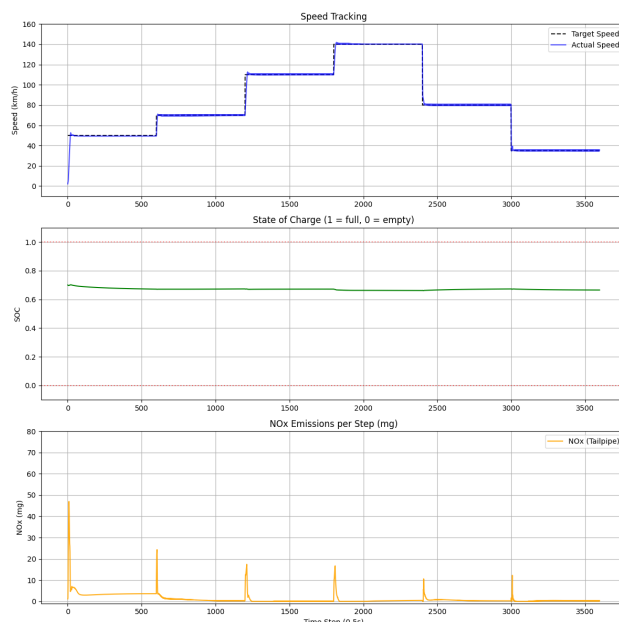


Figure 6.20: Evaluation results for the best PPO checkpoint, seed 4, in Phase 2. The panels show speed tracking, the SOC trajectory, and tailpipe NO_x emissions over the evaluation cycle.

6.2.2 TD3

Reward-weight search. TD3’s study ran 20 valid trials (15 completed, 5 pruned) and selected the lowest SOC weight of the three agents, $W_{\text{soc}^2} = 64.50$, paired with a high emission weight $W_{\text{emi}} = 1.010$. The comparably low SOC weight leads to the largest residual drift of the three agents ($\max |\Delta \text{SOC}| = 0.086$, Table 6.3). The corresponding reward-weight trial history is provided in Figure C.2.

Seed runs. TD3 does not produce a stable controller in Phase 2. Its seed-wise learning curves (Figure 6.21) exhibit large-amplitude reward collapses that occur throughout the full 4,000,000-step budget rather than dying out after the initial reward shock. Two of the ten seeds (1 and 3) never recover a usable speed-tracking policy: seed 3 gets stuck at a low-speed plateau and never follows the higher staircase setpoints, reaching 35.57 km/h RMSE, and seed 1 is a milder version of the same failure at 6.78 km/h. These two seeds inflate the all-seed RMSE to 7.41 ± 9.94 km/h; removing them recovers a mean of 3.96 ± 0.47 km/h (Table 6.4, third column), comparable to SAC. However, the volatility in Figure 6.21 is present in *all* seeds, not only the two that diverge, so even the surviving runs cannot be considered robust and reliable. **TD3 is therefore not considered a viable candidate for the control problem**, and the favourable 8/10 column is reported only to characterise the failure rather than to reclaim the agent’s competence.

Best-seed analysis omitted. The analogous best-seed action-summary and evaluation-result panels are omitted for TD3. Although the scalar selection score would nominate seed 9, the Phase 2

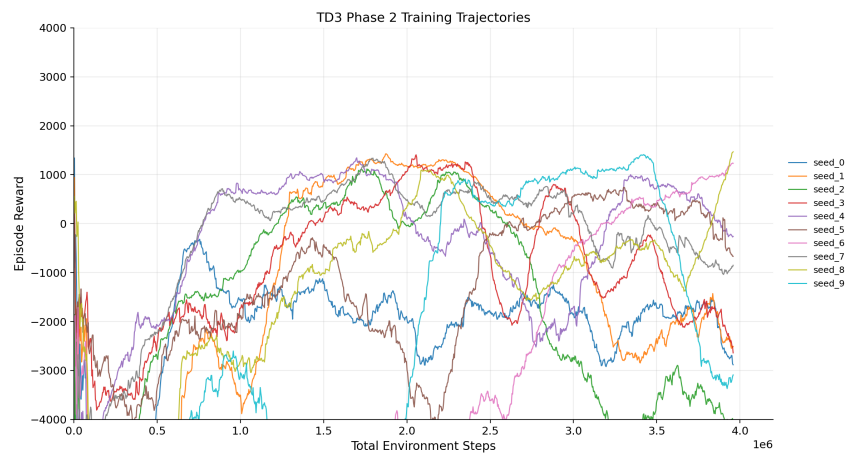


Figure 6.21: Training progress across the 10 TD3 seeds in Phase 2. Curves are smoothed via moving average over 80 episodes, and the y-axis is clipped to ± 4000 ; the large-amplitude reward collapses persist throughout training.

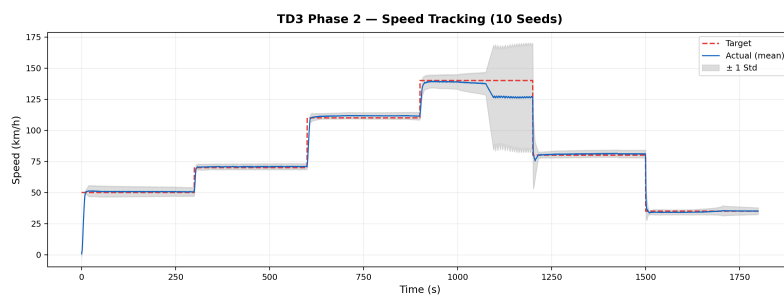


Figure 6.22: Evaluation speed-tracking trajectory for TD3 in Phase 2. The spread across seeds is far larger than for PPO or SAC.

TD3 population is unstable and includes outright failed policies; interpreting a single surviving checkpoint would therefore overstate the reliability of the algorithm.

6.2.3 SAC

Reward-weight search. SAC’s study ran 20 valid trials (11 completed, 9 pruned) and produced the cleanest single trial of the entire Phase 2 search (2.44 g NO_x at max $|\Delta\text{SOC}| = 0.013$, Table 6.3). It selected a moderate emission weight ($W_{\text{emi}} = 0.495$) but the highest SOC weight of the three agents ($W_{\text{soc}^2} = 167.01$). The corresponding reward-weight trial history is provided in Figure C.3. This is consistent with SAC carrying the larger NO_x reduction burden among the two stable agents: its transferred checkpoint (48.94 g NO_x, Table 6.2) emitted substantially more than PPO’s (5.57 g).

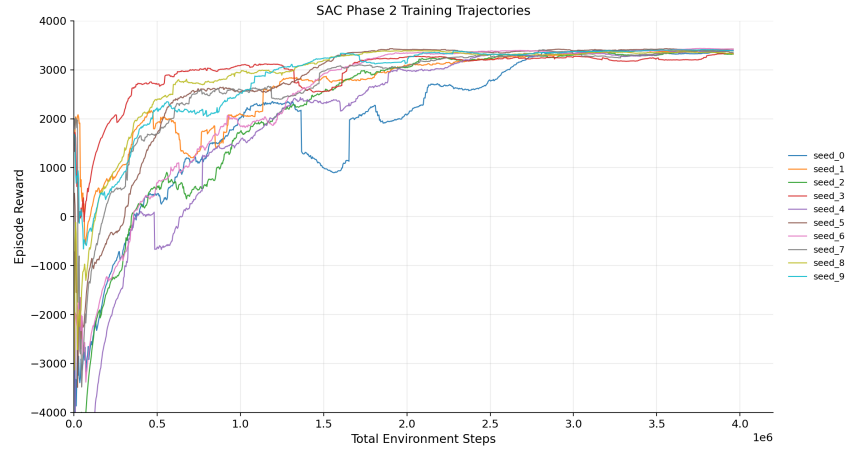


Figure 6.23: Training progress across the 10 SAC seeds in Phase 2. Curves are smoothed via moving average over 80 episodes, and the y-axis is clipped to ± 4000 .

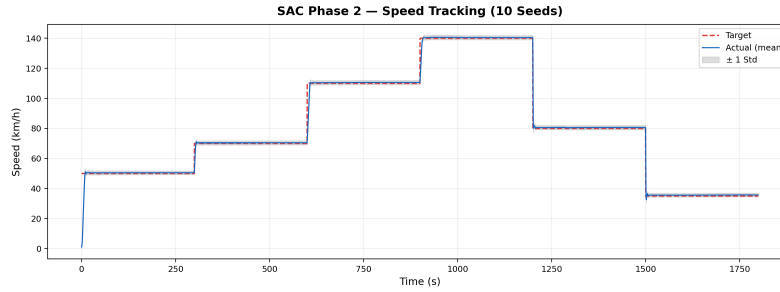


Figure 6.24: Evaluation speed-tracking trajectory for SAC in Phase 2.

Seed runs. SAC achieves the largest absolute and relative emission reduction. Its learning curves (Figure 6.23) are stabilising after the initial shock, and the evaluation RMSE of 3.33 ± 0.33 km/h

is slightly better than its Phase 1 10-seed mean of 3.45 km/h. The energy-side metrics in Table 6.4 show cumulative NO_x cut by 93% against the Phase 1 10-seed mean of 63.8 g to 4.51 ± 1.46 g — the lowest mean of the three agents — and the charge drift reduced to $|\Delta\text{SOC}| = 0.010$ (from the Phase 1 10-seed mean of +0.191), roughly three times tighter than PPO’s 0.029.

The best seed. The best Phase 2 SAC seed is seed 7 with a composite score J of 2.06. As for the selected PPO seed, the score is determined by the NO_x term because the SOC and RMSE constraints are satisfied. The corresponding action summary and evaluation results are shown in Figure 6.25 and Figure 6.26, respectively.

SAC’s braking and engine on/off behaviour remains largely unchanged compared with Phase 1. The ICE speed, however, shifts to lower operating points, with one cluster around 2,400 rpm and another around 1,000 rpm during the first stage of the evaluation cycle. A volatile pattern similar to that of PPO is visible in the EM2 command, which produces a wider density distribution. Stages 2–5 contribute most strongly to this effect, with the command rapidly switching between approximately -180 and 250 Nm. The fuel command also shifts downward: its main density peak is now around 15 mg rather than approximately 35 mg in Phase 1. This suggests that the same increase in command volatility already observed for PPO in Phase 2 contributes to SAC’s substantial NO_x reduction. Whether this behaviour reflects a transferable control insight or an exploitation of the surrogate system cannot be determined within the scope of this thesis.

The resulting velocity, SOC, and tailpipe NO_x emissions are depicted in Figure 6.26.

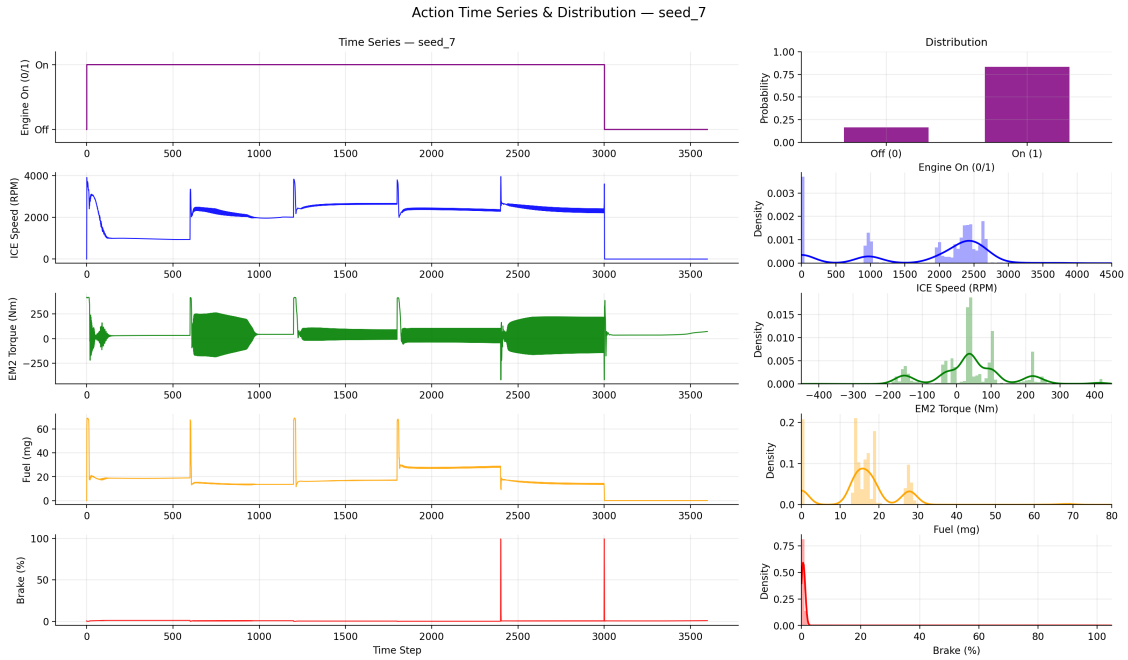


Figure 6.25: Action summary for the best SAC checkpoint, seed 7, in Phase 2. The left panels show the SAC agent’s actions during the evaluation cycle; the right panels show the corresponding densities.

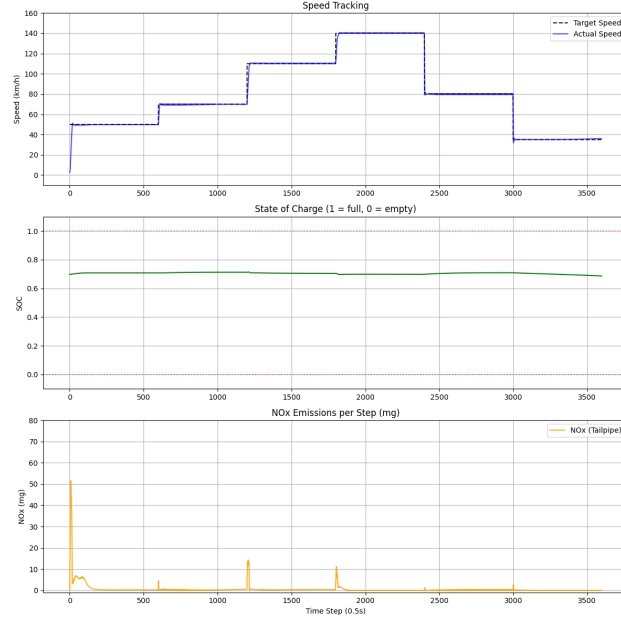


Figure 6.26: Evaluation results for the best SAC checkpoint, seed 7, in Phase 2. The panels show speed tracking, the SOC trajectory, and tailpipe NO_x emissions over the evaluation cycle.

6.2.4 Summary

Training convergence. As discussed above, the Phase 2 results reveal clear differences in control performance across the algorithms. This pattern is also apparent in the seed-averaged training trajectories used to assess convergence. Figure 6.27 aggregates the 10-seed training performance for each algorithm and shows that TD3 fails to converge to a robust policy, whereas PPO and Soft Actor-Critic (SAC) stabilise and eventually reach episodic rewards close to those observed in Phase 1.

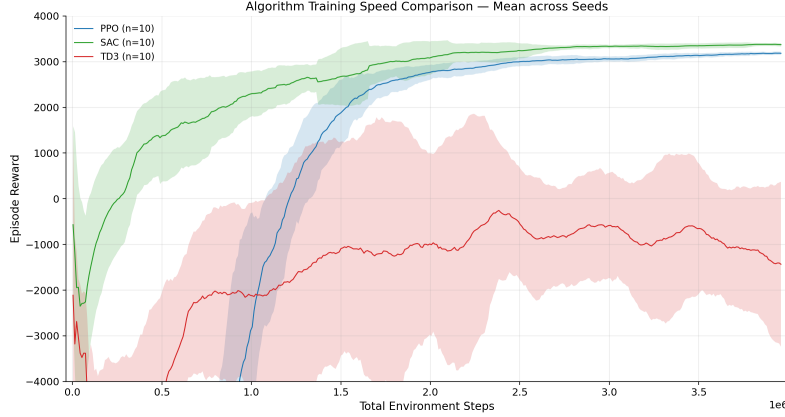


Figure 6.27: Comparison of seed-averaged training progress across all Phase 2 algorithms. Curves are smoothed with a moving average over 80 episodes, y-axis clipped to ± 4000 and shaded bands represent $\pm$ one standard deviation across the 10 seed curves at each x-grid point.

Multi-objective feasibility. Activating the emission and SOC reward terms and fine-tuning from the Phase 1 checkpoints reduces cumulative NO_x and the SOC drift by roughly an order of magnitude (when neglecting TD3). Comparing the 10-seed statistics with the Phase 1 10-seed mean (Table 6.1): mean speed tracking is not degraded but slightly improved for the two stable agents (PPO 4.19 $\rightarrow$ 3.13, SAC 3.45 $\rightarrow$ 3.33 km/h), with the its RMSE variance reduced. The per-agent change against these Phase 1 10-seed baselines is summarised in Table 6.5.

Metric	PPO	TD3 (8/10)	SAC
RMSE (km/h)	4.19 $\rightarrow$ 3.13	3.81 $\rightarrow$ 3.96	3.45 $\rightarrow$ 3.33
RMSE σ	0.98 $\rightarrow$ 0.07	0.29 $\rightarrow$ 0.47	0.43 $\rightarrow$ 0.33
NO_x (g)	48.3 $\rightarrow$ 4.94	95.4 $\rightarrow$ 8.65	63.8 $\rightarrow$ 4.51
NO_x (mg/km)	1,165 $\rightarrow$ 122	2,327 $\rightarrow$ 213	1,564 $\rightarrow$ 111
Fuel (g)	79.6 $\rightarrow$ 52.8	122.6 $\rightarrow$ 79.5	113.5 $\rightarrow$ 64.0
$ \Delta\text{SOC} $	0.247 $\rightarrow$ 0.029	0.146 $\rightarrow$ 0.069	0.191 $\rightarrow$ 0.010

Table 6.5: Phase 1 $\rightarrow$ Phase 2 change per algorithm. Phase 1 figures are the 10-seed means from Table 6.1; Phase 2 figures are from Table 6.4 (TD3 with the two divergent seeds removed). NO_x and fuel fall together because the Phase 1 policies burned extra fuel at high engine load purely to overcharge the battery, an effect the SOC term removes.

The overall progression of this reduction across the full training pipeline is shown in Figure 6.28, which traces the per-agent mean cumulative NO_x from the Phase 1 baseline through the two reward-weight search stages to the Phase 2 fine-tune. PPO and SAC settle just above the indicative Euro 6 line at ≈ 5 g per cycle, whereas TD3 plateaus at (10 g).

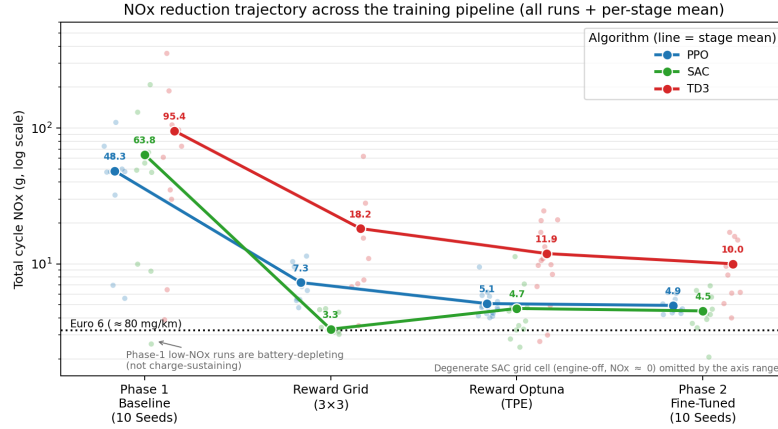


Figure 6.28: Per-stage cumulative tail-pipe NO_x (total grams over the identical staircase evaluation cycle, logarithmic y-axis) across the four pipeline stages: the Phase 1 baseline (10 seeds, emission term off), the 3×3 reward-weight grid, the Optuna Tree-structured Parzen Estimator (TPE) refinement, and the Phase 2 10-seed fine-tuned. Faint markers denote individual runs; the bold line connects the per-stage mean of each algorithm. The dotted line marks the gram-equivalent of the Euro 6 limit (80 mg/km over the ≈ 40.4 km cycle). Note that the grid and Optuna stages hold one run per weight configuration, so their means are not a controlled comparison against the 10-seed Phase 1/Phase 2 statistics; the single degenerate engine-off SAC grid cell ($\text{NO}_x \approx 0$) lies below the axis range and is the reason why NO_x for SAC in that experimental stage is so low.

Resolving the Phase 1 charge–emissions trade-off. The Phase 2 parallel-coordinate plot in Figure 6.30 shows that the scatter observed in Phase 1 (Figure 6.15) is replaced by a much more coherent low-emission, charge-sustaining bundle. In Phase 1, the tracking axes (RMSE, MAE) were already tightly clustered, but fuel, NO_x , and ΔSOC spanned more than two orders of magnitude. Phase 2 removes this degeneracy: ΔSOC contracts to a narrow band around zero for every seed, NO_x shrinks from a 2.6–354 g span to roughly 2–17 g, and fuel tightens correspondingly. For PPO (blue) and SAC (green), the polylines now remain compact across all six axes, meaning that seeds which track well are also clean and SOC-maintaining; the only remaining crossings and spikes are the volatile TD3 seeds, which jump on fuel and NO_x and collapse on reward. The Pareto overlay in Figure 6.29 highlights that only one Phase 1 seed is charge-sustaining ($|\Delta\text{SOC}| < 0.10$), and it still emits 60.9 g of NO_x , whereas all Phase 2 seeds are charge-sustaining.

Operating-point shift by reducing NO_x . The emission reduction originates in a shift of the engine operating point toward lower load. Because the tailpipe NO_x rate rises monotonically with engine torque (Figure 6.31; the measured bench data give Pearson correlation($\text{torque}, \text{NO}_x$) = 0.84), and because the low-BSFC efficiency optimum overlaps with the high-load region, lowering NO_x requires moving *away* from the BSFC sweet spot (Figure 6.16). The per-agent engine-load distributions confirm the shift (Figure 6.32), and the operating-point overlay on the measured NO_x map (Figure 6.33) shows the median operating point dropping out of the brighter high- NO_x regions into the darker low- NO_x regions for all three agents.

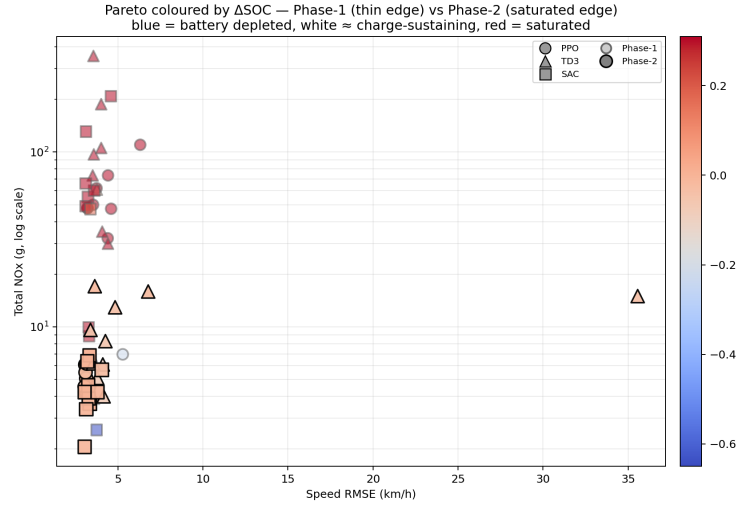


Figure 6.29: Speed-tracking accuracy versus cumulative NO_x (log axis) for all 30 Phase 1 seeds (thin edge) and 30 Phase 2 seeds (saturated edge). Marker shape encodes the algorithm and fill colour encodes ΔSOC (blue = battery depleted, white $\approx$ charge-sustaining, red = saturated). The lone deep-blue low- NO_x Phase 1 marker is SAC seed 8, isolating the single “clean” Phase 1 seed as a battery-depleter.

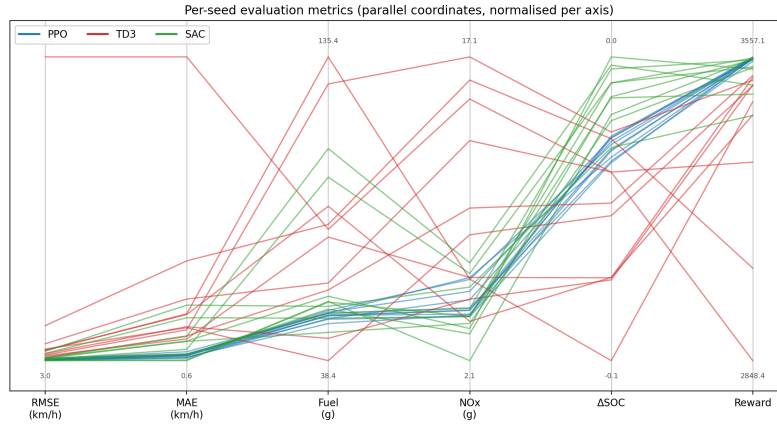


Figure 6.30: Per-seed Phase 2 evaluation metrics in parallel coordinates. Each polyline is one seed, each axis is normalised independently to its own min–max range (true extrema printed at the axis ends), and colour encodes the algorithm. Contrast with the Phase 1 version in Figure 6.15.

A notable consequence of the operating-point shift is that fuel consumption falls alongside NO_x rather than trading off against it (Table 6.5). The Phase 1 policies, lacking any SOC penalty, overcharged the battery to its upper clip ($\Delta\text{SOC} \approx +0.30$, subsection 6.1.4). Assuming that overcharge required additional high-load engine operation purely to charge the battery which inflates both fuel and NO_x . Once the SOC term enforces SOC maintenance, the engine performs only the tractive work the cycle demands, so both quantities drop together. The same shift is visible on the BSFC map (Figure 6.34), read as a slight increase in distance from the efficiency optimum; representing a trade-off, in brake-specific terms, for the large NO_x reduction.

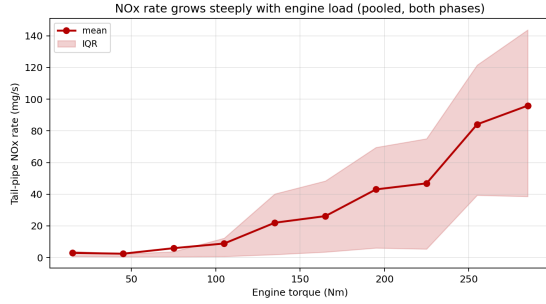


Figure 6.31: Pooled tail-pipe NO_x rate as a function of engine torque for both phases and engine-on steps, with the inter-quartile band. The monotonic trend explains why moving to lower engine load reduces NO_x .

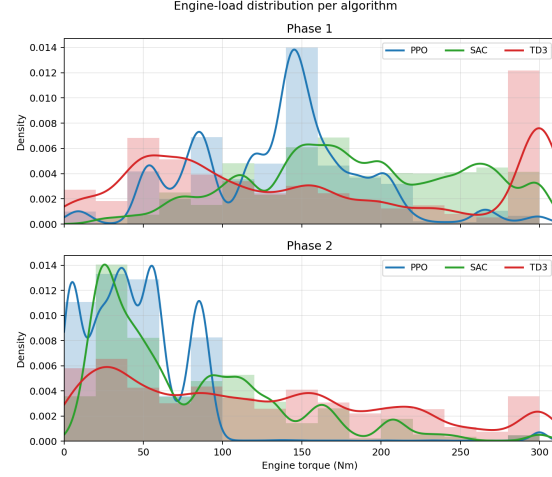


Figure 6.32: Per-agent engine-torque distributions for Phase 1 (dotted) and Phase 2 (solid). Phase 2 concentrates engine operation at markedly lower load, where NO_x production is low.

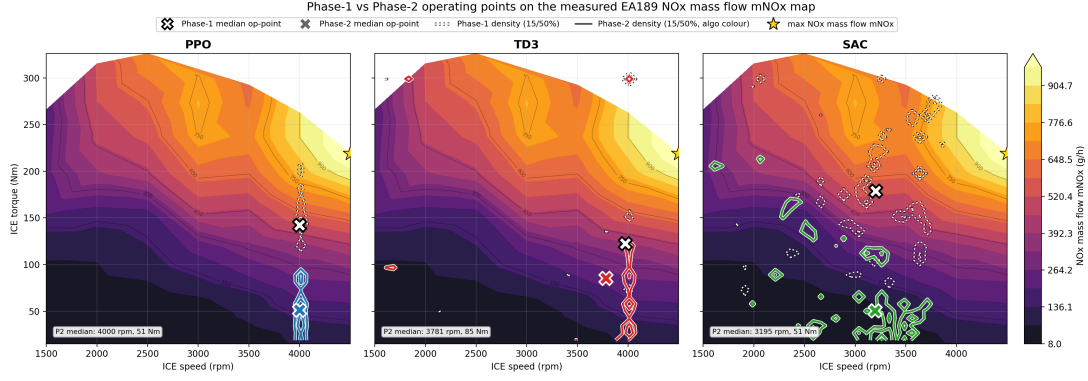


Figure 6.33: Engine operating points over the measured EA189 NO_x mass-flow-rate ($\dot{m}_{\text{NO}_x, \text{tp}}$) map, interpolated from the same 74 bench points as the BSFC map. White markers/contours show the Phase 1 median operating point and density; coloured markers/contours show Phase 2. All three agents move from the bright high- NO_x band into the dark low- NO_x band.

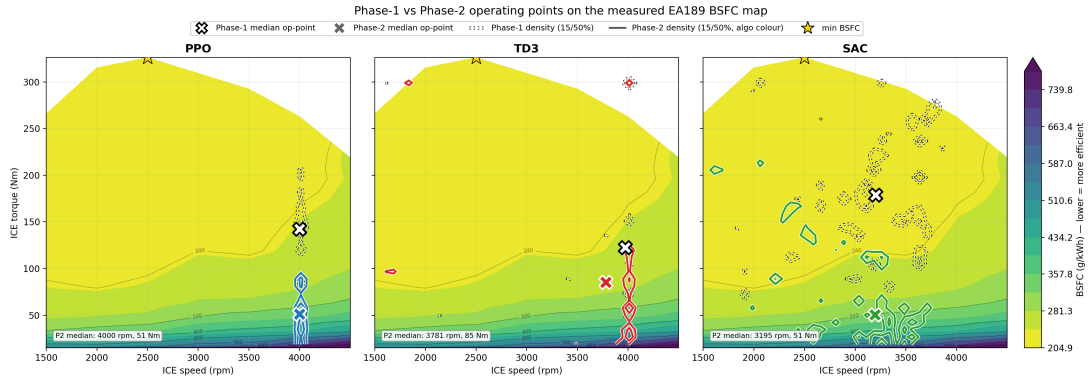


Figure 6.34: Engine operating points over the measured EA189 BSFC map. White markers/contours show Phase 1, coloured markers/contours show Phase 2. The Phase 2 policies move to lower load, slightly away from the most efficient measured point (gold star), trading a small amount of brake-specific efficiency for the large NO_x reduction shown in Figure 6.33.

Algorithm recommendation. Where Phase 1 SAC had the lowest speed tracking variance, Phase 2 highlights that **PPO is the most accurate candidate**; it transfers with the smallest tracking penalty and the lowest seed-to-seed variance. Meanwhile **SAC is the strongest emission performer** and the natural choice when minimum NO_x is the priority. **TD3 is excluded** as a candidate due to the training instability documented in subsection 6.2.2. The validation that motivated Phase 2 is met for the two stable agents: emissions are reduced and the battery charge is maintained, without a meaningful loss of speed-tracking quality. The remaining gap to the Euro 6 limit, the staircase-versus-WLTC caveat, and the TD3 instability are taken up in chapter 7.

6.3 Generalisation Evaluation on the WLTC

The preceding analysis was specific to the synthetic staircase speed trajectory used for training and evaluation. To assess whether the resulting policies generalise to a more realistic driving mission, the Phase 2 PPO and SAC agents are evaluated zero-shot on the standardised WLTC and compared with a rule-based controller. This comparison is motivated by current industry practice, which still relies heavily on rule-based solutions because of their interpretability and reliability, as discussed in section 1.1. Frey et al. [5] provide the reference benchmark used here by developing such a rule-based strategy and evaluating it on the WLTC.

All ten PPO and SAC seeds are evaluated without retraining; TD3 is omitted because the Phase 2 results showed insufficient training stability. For a fair, charge-sustaining comparison, the RL agents, which start and return to $\text{SOC} \approx 0.70$, are matched against the rule-based *isoSOC* run ($\text{SOC} \approx 0.56$); both settings are charge-neutral over the cycle.

6.3.1 Population-Level Picture (10 seeds)

Table 6.6 aggregates the ten seeds per algorithm as mean $\pm$ standard deviation. Both RL algorithms track the WLTC target more tightly than the rule-based controller (PPO RMSE 0.81 vs 2.29 km/h), demonstrating that the learned policies transfer zero-shot to an unseen, realistic cycle. However, they do so at higher tailpipe- NO_x and fuel cost: the phase-2 reward places no weight on fuel ($w_{\text{fuel}} = 0$), so the agents keep the engine running almost continuously (PPO 0% idle-off) and burn $1.6\text{--}2.2\times$ the rule-based fuel while emitting $\sim 17\%$ more NO_x .

Controller	NO_x (mg/km)	Fuel (g)	RMSE (km/h)	MAE (km/h)	ΔSOC	Eng. off (%)
PPO (10)	172 ± 13	3060 ± 52	0.81 ± 0.18	0.61 ± 0.16	-0.027 ± 0.006	0 ± 0
SAC (10)	173 ± 41	2223 ± 285	1.64 ± 0.68	1.30 ± 0.73	$+0.004 \pm 0.013$	13 ± 24
Rule-based	147	1376	2.29	1.73	+0.003	16

Table 6.6: WLTC, 10-seed mean $\pm$ std vs. the rule-based controller.

These averages must be read with care. The standard deviations are partly large and, more fundamentally, the ten seeds still inherit variations in the control policy and are not noisy realisations of one controller. Accordingly, averaging distinct strategies should be avoided since it describes no controller that actually exists and obscures the best achievable, individual behaviour. For the purpose of further development a *single* seed must therefore be selected and carried forward, rather than the seed-averaged number.

6.3.2 Best-Seed Comparison

Selecting one seed per algorithm with the composite score used throughout phase 2 yields **PPO seed 2** and **SAC seed 7** (Table 6.7).

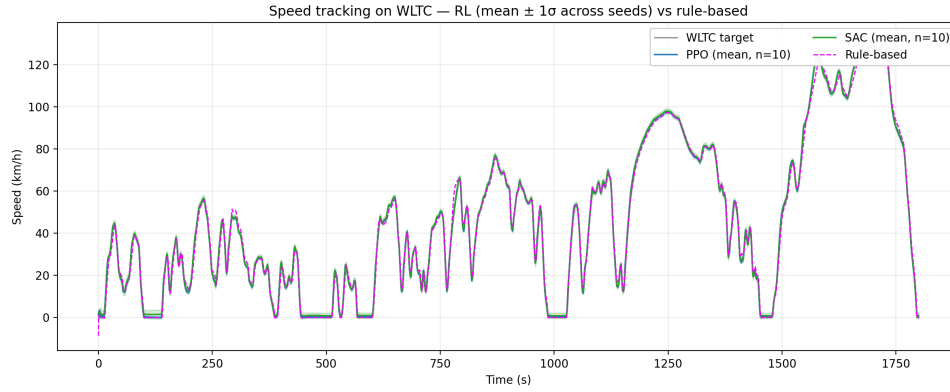


Figure 6.35: Speed tracking on the WLTC. RL mean $\pm 1\sigma$ across 10 seeds per algorithm vs. the rule-based controller. Both RL algorithms follow the target trace closely across the full cycle.

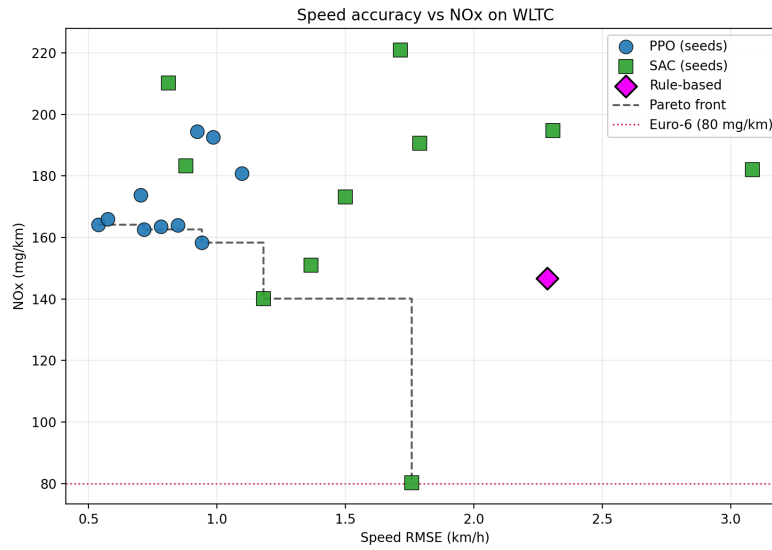


Figure 6.36: Speed RMSE vs. tailpipe NOx on the WLTC, one marker per seed. The single SAC seed on the Euro-6 line (80 mg/km) is seed 7, the best candidate.

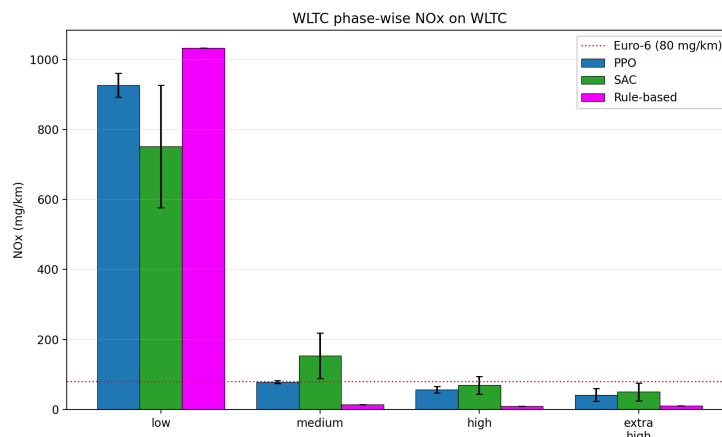


Figure 6.37: WLTC phase-wise tailpipe NOx (mg/km), per-algorithm seed mean vs. the rule-based controller, against the Euro-6 80 mg/km limit.

Controller	NOx (mg/km)	Fuel (g)	RMSE (km/h)	MAE (km/h)	Δ SOC	Eng. off (%)
PPO seed 2	158	3087	0.94	0.75	−0.024	0
SAC seed 7	80	1730	1.76	1.09	+0.010	57
Rule-based	147	1376	2.29	1.73	+0.003	16

Table 6.7: WLTC, best seed per algorithm vs. the rule-based controller.

The selected seeds reverse the unfavourable population-level reading. *SAC seed 7 outperforms the rule-based controller on both NO_x and speed tracking*: it reduces tailpipe NO_x by $\sim 45\%$ (80 mg/km vs. 147 mg/km, i.e. to the Euro 6 limit), tracks the target more accurately (RMSE 1.76 vs. 2.29 km/h), and remains charge-sustaining (Δ SOC = +0.010). This performance is achieved by switching the engine off for 57% of the cycle — far more than the rule-based controller’s 16% — at the cost of $\sim 26\%$ higher fuel consumption. The EM2 command spans the full feasible range, supporting SOC sustainment and vehicle propulsion, but remains concentrated around 50. Compared with the staircase cycle, braking is used more frequently, which is consistent with the greater variability of the WLTC speed trajectory. The ICE speed distribution has a much larger mass at 0 rpm due to the engine-off phases, while fuel injection shows engine start-up spikes followed by rapid cutback to approximately half of the spike value. The action summary and the resulting velocity, SOC, and tailpipe NO_x emissions are depicted in Figure 6.38 and Figure 6.39, respectively. PPO seed 2, by contrast, attains the best speed tracking of all controllers (0.94 km/h) but does not beat the rule-based baseline on NO_x and retains the always-on, high-fuel behaviour. **SAC seed 7 is therefore the selected candidate for continued development.**

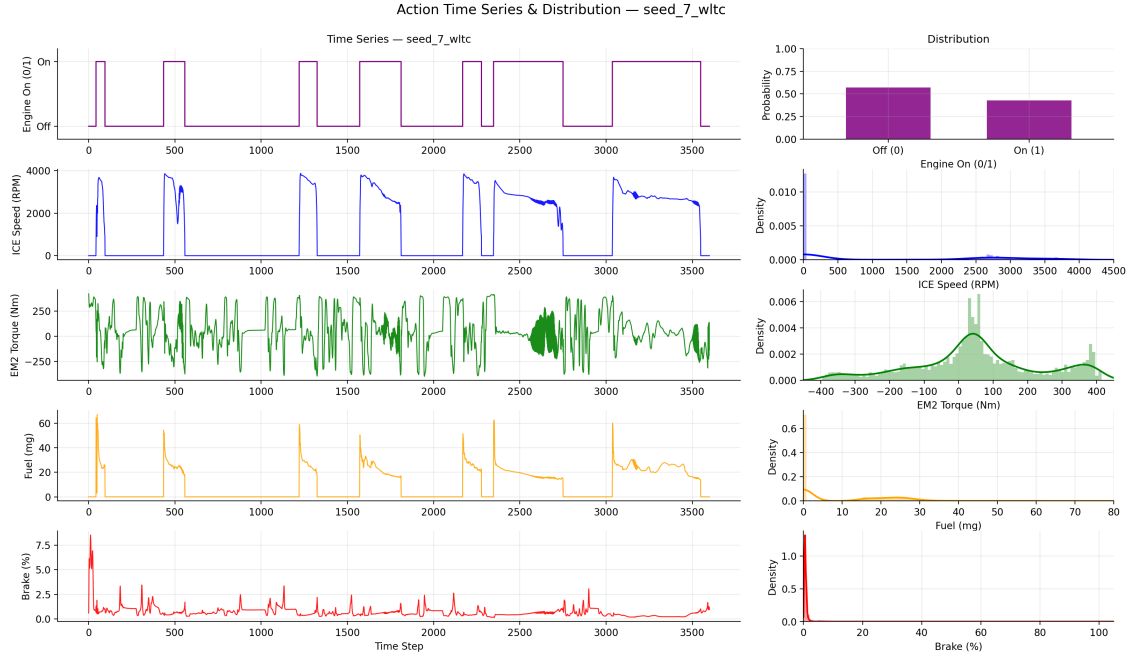


Figure 6.38: Action summary for the best SAC checkpoint, seed 7, on the WLTC. The left panels show the SAC agent’s actions during the WLTC; the right panels show the corresponding densities.

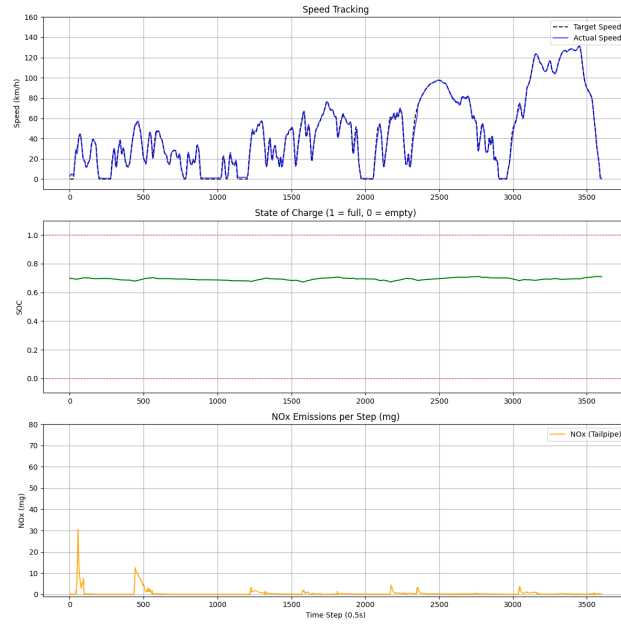


Figure 6.39: Evaluation results for the best SAC checkpoint, seed 7, on the WLTC. The panels show speed tracking, the SOC trajectory, and tailpipe NO_x emissions over the evaluation cycle.

Chapter 7

Discussion

This chapter discusses the obtained results by summarising the key insights and findings. Furthermore, it provides a critical perspective on the results chapter (chapter 6) by examining the results against the study’s underlying assumptions and limitations.

7.1 Findings and Takeaways

Main lessons from the staged training procedure, the Phase 2 multi-objective fine-tuning, and the subsequent zero-shot WLTC evaluation are presented while the central message is, that, the proposed approach can learn charge-sustaining, low-Nitrogen Oxides (NO_x) behaviour in the surrogate environment. This result remains bounded by the selected reward terms and the comparison with the rule-based baseline.

7.1.1 Overall Outcome

Reinforcement learning as an HEV emission controller. The staged curriculum-learning approach can be considered successful within the restrictions of the conducted experiments. The methodology proposed in chapter 4 first isolates the speed-tracking task and then fine-tunes the resulting policies with additional NO_x and State of Charge (SOC)-squared penalties. This second stage reduces cycle NO_x and SOC drift by approximately one order of magnitude each while preserving the speed-tracking performance of the two stable agents Proximal Policy Optimisation (PPO) and Soft Actor-Critic (SAC) (see Table 6.5).

The main conclusion is therefore not simply that the agents can track a speed trajectory, but that speed tracking, low NO_x , and SOC sustainment can be combined through a scalarised reward and a constrained reward-weight search. By transferring a speed-tracking policy into Phase 2, the multi-objective fine-tuning was able to extent the learned capability without having to relearn basic controllability from scratch.

Charge-sustaining emission reduction. The main contribution should be framed as NO_x reduction under a charge-sustaining constraint. A comparison based on NO_x alone would be misleading, because Phase 1 already produced an apparently clean seed: SAC seed 8 reached only

2.57 g cycle NO_x , but did so by draining the battery. Only one of the 30 Phase 1 seeds was relatively charge-sustaining, but emitted 60.9 g NO_x . By contrast, the Phase 2 policies reduce SOC drift substantially and improve on this fair Phase 1 comparator by a factor of 4 to 30 in cumulative NO_x .

This distinction is important for interpreting the whole thesis. Low NO_x is only meaningful if the controller does not obtain it by simply utilising battery energy. The fair result is therefore not “minimum NO_x at any cost”, but low NO_x while maintaining battery SOC within the specified evaluation tolerance.

7.1.2 Mechanistic and Algorithmic Insights

Mechanism of the NO_x reduction. The Phase 2 emission reduction is mechanistically interpretable rather than only a black-box policy effect. It is associated with a shift towards lower Internal Combustion Engine (ICE) torque, where the tailpipe NO_x rate is substantially lower (Figure 6.31). Since the Phase 1 policies often overcharged the battery through high-load engine operation, enforcing charge sustainment in Phase 2 removes this wasteful operating mode and reduces NO_x and fuel consumption simultaneously.

This simultaneous reduction should not be interpreted as a general NO_x –fuel synergy. It occurs only because the Phase 1 policies were using additional engine work to charge the battery beyond sustainment. Once this behaviour is penalised, both fuel and NO_x decrease. At the same time, the Brake-Specific Fuel Consumption (BSFC) map alone is not the right lens for judging the NO_x emission, because the low-BSFC region overlaps with high- NO_x operation. A policy that moves away from the better BSFC can therefore still be desirable if the objective is tailpipe NO_x reduction.

Learning rate as a fine-tuning asset. PPO’s small learning rate appears to be beneficial during Phase 2 fine-tuning. Under the changed reward function, the small update steps keep PPO close to its Phase 1 speed-tracking policy, which helps avoid catastrophic forgetting and yields the most stable seed population. What looked like a Phase 1 disadvantage, due to slow convergence, therefore becomes useful once the objective changes.

This explanation remains a hypothesis rather than a proven causal mechanism. Learning rate alone cannot explain the full pattern: SAC uses a higher learning rate than TD3 but remains stable, whereas TD3 becomes unstable. The observed stability therefore likely depends on the interaction between learning rate, entropy regularisation, policy stochasticity, and on-/off-policy updates. A dedicated learning-rate ablation would be required to confirm this interpretation.

TD3 instability. Twin Delayed Deep Deterministic Policy Gradient (TD3) cannot be recommended for this control problem in its current configuration. If the two divergent runs are excluded, the remaining eight Phase 2 seeds reach a mean Root Mean Square Error (RMSE) of 3.96 km/h; however, seeds 1 and 3 diverge to RMSE values of 6.78 and 35.57 km/h. This 20 % seed-failure rate, combined with the algorithm’s overall instability, represents a clear robustness failure, even if some seeds remain competitive with SAC in speed tracking.

A plausible explanation for this behaviour is the effective disabling of delayed actor updates through the Phase 1 hyperparameter selection `policy_delay=1`. As discussed in subsection 2.3.6, delayed actor updates are one of the defining mechanisms of TD3. The Phase 1 speed-tracking objective may have been simple enough to solve without this mechanism, whereas the increased complexity of Phase 2 could have benefited from it. For a controller, the relevant question is whether

training reliably produces usable policies; under the present conditions, TD3 does not meet that requirement.

7.1.3 Policy Selection and WLTC Generalisation

Best seed per algorithm. Ranking the validation seeds by the composite score on the Phase 2 experiments selects PPO seed 4 ($J = 4.23$), SAC seed 7 ($J = 2.06$), and TD3 seed 9 ($J = 8.67$). Because TD3 is unstable, the practical comparison is between PPO and SAC. SAC seed 7 is the cleanest individual policy across all experiments, whereas PPO provides the most stable seed population. Thus, SAC offers the strongest single candidate for continued development.

Generalisation to WLTC. The zero-shot WLTC evaluation gives a more cautious view of the learned policies. PPO and SAC track the WLTC reference more accurately than the rule-based controller under the isoSOC comparison condition (PPO RMSE 0.81 km/h, SAC 1.64 km/h, rule-based 2.29 km/h). However, the 10-seed averages emit about 17 % more NO_x (~ 172 versus 147 mg/km) and burn 1.6 to $2.2\times$ more fuel than the rule-based controller. All seeds exceed the Euro 6 limit of 80 mg/km except the one near-threshold SAC seed 7.

The WLTC result therefore demonstrate generalisation capabilities but also highlight the per-siting importance of a rule-based solution. The learned policies transfer their speed-tracking competence to a cycle that was not used during training, but the population-level emission and fuel performance do not yet surpass the expert-designed rule-based solution.

The fuel and CO_2 blind spot. The main reason for the WLTC fuel penalty is that fuel economy was not part of the reward. The Phase 2 fuel reduction on the staircase cycle is only a side effect of charge-sustaining NO_x optimisation, not evidence of fuel-optimal control. On the WLTC, this limitation becomes visible: PPO keeps the engine on throughout the cycle, and both learned controllers burn more fuel than the rule-based baseline.

The controller should consequently be described as NO_x - and SOC-oriented, not energy-optimal. Adding an explicit fuel or CO_2 term is the most direct next extension of the reward formulation.

7.2 Limitations of the Study

7.2.1 Scope and Generalisability

Engine-specific models. First and foremost, the generalisability of this study is limited by the highly customised plant Long Short-Term Memory (LSTM)s used to model the EA189 engine. Consequently, if a different power-split Hybrid Electric Vehicle (HEV) is to be controlled, the developed models are unlikely to be directly applicable because of differences in internal powertrain functionality. However, although the trained models themselves are not transferable, the underlying methodology remains valid.

Assumption of near-complete plant observability. Compared with the prior LDCI2027 work, which incorporated only four observations—namely target velocity, actual velocity, NO_x , and SOC—this study uses an extended observation vector with 12 observations. It also includes the SCR1 wall-temperature variable, which is known not to be available during deployment. This

creates a simulation-to-reality gap that hinders future deployment. Therefore, each observation should be reassessed to determine whether it is a feasible member of the observation vector.

Single run rule-based baseline. The comparison with the rule-based solution proposed by Frey et al. [5] should be interpreted with caution. The learned policies are reported as 10-seed averages to account for stochastic variability during training, whereas the rule-based controller is represented by a single evaluation. This does not invalidate the benchmark, since a rule-based controller does not exhibit seed-dependent training variance under identical conditions, but it makes the variability assessment asymmetric. The comparison is therefore informative as a practical reference point, but it should not be interpreted as a fully statistically balanced superiority test.

7.2.2 Experimental Design and Training Procedure

Limited training trajectory diversity. The training speed trajectory had several characteristics that could potentially have been modified to accelerate training, improve robustness at speeds above 150 km/h, which were not included in the training range, or produce policies that change speed more reliably and rapidly while remaining efficient. Possible improvements include increasing the maximum speed of the training cycle or shortening the plateaus in the staircase cycle.

Different HPO depths. Contrary to the initial assumption that only the trials within one cluster Hyperparameter Optimisation (HPO) job would be considered for Phase 1, Optuna considered all entries in `study_journal.log`. As a result, the study included 70 PPO trials, 45 TD3 trials, and 44 SAC trials, rather than the intended 20 trials per algorithm. This does not invalidate the resulting models, but it does make the HPO depth uneven across algorithms because premature test runs were included in the final optimisation history.

Ordering of analysis steps. All multi-step experimental pipelines require a decision on how the individual stages are ordered. Instead of the methodology used here, each Phase 1 hyperparameter configuration could have been evaluated across 10 seeds before selecting the best configuration. This would have provided a statistically more robust basis for model selection, but it would also have increased the computational cost substantially. The chosen methodology therefore reflects a compromise between computational feasibility and experimental reliability. Nevertheless, a more exhaustive seed-wise evaluation before checkpoint selection could further improve the robustness of the reported algorithm ranking.

Phase 2 transfer strategy. In the current experimental design, both the Phase 1 checkpoints and the corresponding hyperparameters are transferred directly to Phase 2. This strategy reduces computational cost and provides stable starting policies that have already learned the basic speed-tracking task. However, it may also constrain the search for control strategies that are better suited to the full multi-objective reward, since the inherited policies were specialised to the speed-tracking-only objective. The same limitation applies to the transferred hyperparameters: an architecture or update schedule that is effective in Phase 1 is not necessarily optimal once NO_x reduction and SOC sustainment are introduced. For example, SAC’s three-layer network may have supported its successful adaptation, whereas TD3’s Phase 1 setting `policy_delay=1` may have been detrimental in Phase 2. Since training from scratch and Phase 2-specific hyperparameter optimisation were not evaluated directly, this transfer trade-off remains hypothetical.

7.2.3 Reward and Control Formulation

Fine-tuning modifications. Furthermore than the general Phase 2 transfer strategy, two specific adjustments could have further improved the overall quality of the work.

- First, the mechanical shaping terms, in particular the mechanical braking penalty defined in Equation 5.3, could have been removed after Phase 1 training. These terms were included to guide early learning, because the agents were not able to stabilise speed tracking without a more restrictive reward function, for example by avoiding continuous braking. However, once speed-tracking competence had been established, removing these shaping terms could have given the Deep Reinforcement Learning (DRL) agents more freedom to explore alternative control strategies without this manually imposed restriction.
- Second, fine-tuning is often accompanied by a reduction in the learning rate. This was not applied in this study, since all hyperparameters were transferred unchanged from Phase 1 to Phase 2. Although this still produced models that solved the multi-objective control problem successfully, a reduced learning rate may have stabilised TD3 training or enabled further policy refinement.

Over-regulation of SOC and speed tracking. The final models are partly a consequence of the acceptance thresholds defined in this study. In particular, the comparison with the rule-based solution highlights the very tight speed-tracking capability of the learned policies. Relaxing the speed-tracking constraint in the composite score could have allowed the optimisation to favour controllers that more closely match the rule-based solution in terms of NO_x minimisation, at the cost of a small loss in tracking precision, for example on the order of 1–2 km/h. Similarly, relaxing the SOC constraint could have enabled greater use of Electric Motor 2 (EM2) and thereby further reduced NO_x . Overall, this limitation arises from heuristically defined tolerance thresholds for speed error and SOC deviation (subsection 5.7.2), which could be aligned more explicitly with the Forschungsvereinigung Verbrennungskraftmaschinen (FVV) project requirements that provide the framework and motivation for this thesis.

7.2.4 Scope of the Control Objective

Component longevity. The implementation already includes simplified measures to support component durability, such as the engine on/off cooldown and limits on harmful SOC states. However, these mechanisms do not constitute a holistic component-longevity model. They capture only selected mechanical-engineering considerations at a simplified level, whereas a more complete durability-aware controller would require closer integration of powertrain-domain expertise.

Single-pollutant objective. The *most severe limitation* is arguably that, although this study demonstrates successful reductions in NO_x emissions, the environmental impact of vehicle operation cannot be described by NO_x alone. Other pollutants and climate-relevant quantities, including the main well-mixed greenhouse gases carbon dioxide (CO_2), methane (CH_4), and nitrous oxide (N_2O), as well as ozone (O_3), also contribute to the regulatory and environmental assessment of a powertrain [73]. Consequently, a controller that optimises only NO_x may shift the operating strategy in ways that are beneficial for one pollutant but unfavourable for others, especially since N_2O can be a potent greenhouse-gas by-product of Selective Catalytic Reduction (SCR) aftertreatment. The

results should therefore be interpreted as evidence for the feasibility of NO_x-aware control, not as proof of a globally emissions-optimal strategy. **A more complete formulation would require a multi-pollutant objective that explicitly accounts for the relevant emission types and their trade-offs.**

7.3 Sustainability Analysis

Following the UPC guidelines [74], this section assesses the sustainability of the thesis along the three classical dimensions — environmental, economic and social — and, for each, distinguishes between the *development* of the academic work itself, the *execution* of the project it proposes (here a hypothetical deployment of the learned controller), and the associated *risks and limitations*. Consistent with the proof-of-concept scope defined in section 1.1, and because the environmental motivation is the very reason this topic was chosen, the analysis is weighted towards the environmental dimension; the economic and social dimensions and the ethical implications are treated more briefly. In line with the UPC guideline, the emphasis lies on the *ability* to analyse these impacts rather than on the absolute sustainability level of the work.

7.3.1 Environmental Dimension

The environmental footprint of an applied machine-learning thesis is best understood as two opposing contributions: an *intrinsic* cost caused while developing the controller, and an *extrinsic* effect that the controller could deliver if deployed. A meaningful assessment must weigh the former against the latter.

Development of the work - the intrinsic cost. The dominant environmental cost of this thesis is the electrical energy consumed during training and hyperparameter optimisation. The cumulative computational effort is not negligible. A single seed run of 4,000,000 timesteps requires between approximately 8 h (PPO) and 15 h (SAC) of wall-clock time (Table 6.1), and the study comprises tens of such seed runs across both phases, together with a larger number of partially completed hyperparameter-optimisation trials. Several design choices helped to limit this footprint: median pruning terminated unpromising HPO trials early instead of running them to completion; the plant model migration to the CTTC-UPC library reduced per-step inference time; environment vectorisation improved hardware utilisation; and the curriculum-inspired learning strategy fine-tuned Phase 2 from the Phase 1 checkpoints rather than training from scratch, which likely reduced the number of timesteps required to reach a proficient policy, as discussed earlier in this chapter. A further mitigating factor is the location of the computation: the training was executed on the Joan Francesc Fernandez (JFF) cluster at CTTC [67], which is located in Spain. At the time of comparison, Spain had a comparatively low electricity carbon intensity (168 gCO₂eq/kWh) relative to, for example, Germany (394 gCO₂eq/kWh); these snapshot values were taken from [75]. This illustrates the more general principle that the carbon intensity of Artificial Intelligence (AI) development can be lowered simply by scheduling compute in regions with a high share of renewable energy. A precise CO₂e estimate would require the cluster’s energy accounting and is therefore not quantified here. Importantly, however, this footprint represents a one-time offline cost.

Hypothetical project execution - the extrinsic potential. The environmental relevance of the proposed controller is that the resulting artefact is *software*: a trained policy that executes

in under a millisecond per step on existing electronic control hardware and requires no additional material or manufacturing. If such a controller delivered a robust per-vehicle reduction in tailpipe emissions, the one-off offline training cost could be amortised over the operating life of every vehicle using it, and even moderate improvements could become relevant at higher scale. The extrinsic sustainability contribution of this thesis should therefore be understood primarily as deployment potential and research insight rather than as a directly quantified emission reduction. The study shows how a scalable software controller can learn emission-aware behaviour under selected conditions, and it documents which design choices, constraints and failure modes shape that behaviour. These findings cannot yet be converted into a quantified environmental benefit, because their future use within the Light-Duty CI-ICE 2027 (LDCI2027) group and beyond is uncertain. Nevertheless, they remain relevant because technological progress often depends on such intermediate evidence, negative results and design guidance.

Environmental risks and limitations. Three caveats accompany this potential: First, the objective optimises NO_x alone. As discussed earlier in this chapter, a controller that minimises NO_x may shift other quantities (carbon monoxide, hydrocarbons, particulates, CO_2) unfavourably, so the demonstrated reduction is not proof of a globally emissions-optimal strategy. Second, the comparison with the rule-based reference on the realistic Worldwide Harmonised Light Vehicles Test Cycle (WLTC) limits the strength of the sustainability claim (section 6.3). The learned policies track the target more tightly, but their 10-seed averages consume $1.6\text{--}2.2\times$ the fuel and emit about 17% more NO_x ; only the selected SAC seed 7 reaches a clearly favourable NO_x result, and it still does so with higher fuel consumption. A net environmental benefit over the established rule-based solution is therefore not yet demonstrated. Third, the entire evaluation rests on LSTM plant surrogates rather than a physical vehicle, so simulation-to-reality effects remain unresolved. The honest reading is therefore that the work demonstrates the *feasibility* of emission-aware learned control, not a delivered environmental gain.

7.3.2 Economic Dimension

Development of the work - the development cost. Expressed as the invoice a client would receive, the cost of the work is dominated by the author’s time, including literature review, study and system design, implementation and debugging, and academic writing. The project started in January 2026 and extended over approximately 21 weeks until June 2026. Assuming an average workload of 30 hours per week yields 630 hours of work; at a hypothetical rate of EUR 20 per hour, this corresponds to EUR 12,600. These values are heuristic, so it should be noted that the estimated cost would increase, for example, if a German salary level were assumed, which is typically higher [76]. Conversely, EUR 20 per hour is already a comparatively generous assumption for student work in Catalonia.

All software (Stable-Baselines3 [69], Optuna [77], the underlying runtime of the CTTC-UPC models, and the Python scientific stack) is open-source, so the licensing cost is zero, and the marginal monetary cost of computation on an existing academic cluster is small and, crucially, not carried by the author. However, a two-month Claude Pro [78] subscription was purchased for support during development of the thesis at EUR 21.42 per month. The hardware used for the work, namely the author’s personal laptop, had already been purchased two years earlier and is therefore not included in this calculation. Overall, this leads to a total estimated cost of *EUR 12,642.84*.

Hypothetical project execution - the project viability. A learned Energy Management System (EMS) is economically attractive because, once trained, it is software with a near-zero marginal cost per vehicle. It could also replace part of the meticulous manual calibration required for rule-based controllers (section 1.1) with an automated, data-driven procedure, potentially reducing the engineering effort needed to develop and recalibrate an EMS across platforms and operating conditions. Avoiding regulatory non-compliance penalties represents a further indirect economic benefit.

Economic risks and limitations. The primary recurring cost arises from the engine-specific nature of the plant surrogates (subsection 7.2.1): a different powertrain would require new data collection, model training and controller re-optimisation, which eliminates the scalability advantage. In addition, the simulation-to-reality gap implies substantial validation costs before deployment. The approval of a learned, less interpretable controller for a safety-relevant function may also be expensive or uncertain. In such a scenario, a simpler rule-based controller might remain the more economical choice.

7.3.3 Social Dimension

Development of the work - the impact on the developers of the work. On a personal level, the thesis required to integrate several previously separate areas: reinforcement learning, automotive powertrain engineering, and sustainability assessment. The work was motivated by the aspiration to direct technical skills towards an environmental problem, which made it personally fulfilling. It was carried out within the international LDCI2027 collaboration in accordance with good professional and academic practice, and uses inclusive, non-discriminatory language throughout the report. The work complies with the UPC Code of Ethics [79]. All this contributed substantially to personal growth and expertise expansion.

Hypothetical project execution - the social impact of the project. Lower tailpipe NO_x can benefit public health and urban air quality, since NO_x emissions are associated with respiratory disease [25], [26], [73]. Vehicle manufacturers and regulators may also benefit from methods that support emission compliance. More broadly, the work contributes to the societal effort to reduce the environmental impact of mobility, which is connected to the wider challenge of mitigating global warming and its associated social and health consequences [73].

Social risks and limitations. A learned controller is less transparent than an explicit rule set, which makes it harder for engineers, regulators and users to understand why a specific action was taken. Beyond this interpretability concern, the main social risks are:

- **Safety and robustness.** If the policy behaves unexpectedly outside its validated operating domain, the consequences could affect drivers, passengers and other road users, even though the controller is not intended as an autonomous-driving system (section 1.3).
- **Human oversight and deskilling.** Excessive reliance on learned controllers could weaken manual-calibration expertise and reduce the ability of engineers to intervene when model behaviour malfunctions. This represents a standard risk of data-driven methods and is not specific to this study only.

Regarding data-protection concerns, the work only uses simulated engine data and no personal data, so the GDPR [80] is not engaged beyond the standard institutional commitments.

7.3.4 Ethical Implications

Reflecting on the consequences of this study raises several ethical considerations that are specific to a learned, sustainability-motivated controller.

Conflict of interest and research independence. This thesis is embedded in the LDCI2027 project under the FVV [21], an industry research association for *combustion engines*. Work funded to improve combustion-based powertrains inherits a structural incentive to present that pathway favourably and, indirectly, to prolong the relevance of the ICE. This potential conflict is disclosed here in the interest of transparency. The analysis throughout the thesis, including the unfavourable WLTC comparison on seed-population level in section 6.3, is reported independently of that incentive, and no result has been framed to support the sponsor’s technology.

Reward design as a choice. A reinforcement-learning controller optimises exactly what its reward function encodes. The relative weighting of emissions, speed tracking and charge sustainment (W_{emi} , W_{soc^2} and the shaping terms) is therefore not a neutral engineering parameter but an implicit ethical decision about whose interests are prioritised: public health (minimal NO_x), driver convenience (tight tracking), or manufacturer cost (fuel and component longevity). Making these weights and their selection procedure explicit (subsection 4.3.3) is an ethical safeguard, because it allows the interests encoded in the objective to be inspected rather than hidden inside a tuning constant.

Accountability for an opaque controller. Because the policy is inhibiting black box characteristics while acting on a safety-relevant powertrain function, a malfunction creates a responsibility gap: it must be avoided to displace blame onto “the algorithm”. Responsibility must remain with identifiable human and institutional actors, including the engineers who design the reward and validation procedure and the organisation that eventually deploys the controller. This is the ethical motivation for retaining human oversight and a verifiable validation process before any real-world use.

Precautionary deployment. The controller is validated only against LSTM surrogates and the objectives of this thesis explicitly exclude autonomous driving or production use (section 1.3). Treating the result as a proof-of-concept rather than a deployable artefact is therefore the ethically required precautionary position. An unvalidated learned controller should not influence a physical vehicle on public roads until its real-world behaviour and failure modes are demonstrably safe.

Greenwashing. Given the framing of this work, a positive proof-of-concept result must not be overstated. Presenting selected, simulation-only NO_x -aware behaviour as evidence of a “clean” or “sustainable” powertrain would constitute greenwashing. Similarly, overstating improvements in a combustion-based HEV could lend overstated legitimacy to continued combustion-engine development and thereby slow the broader transition to fully electric mobility or other low-emission alternatives. Any such claim must therefore be accompanied by the limited scope of the work: a

single-pollutant, in-simulation feasibility study. The reporting of unfavourable outcomes — including the WLTC fuel penalty, the single-pollutant scope, and the TD3 instability — follows directly from this commitment.

7.3.5 Relationship with the Sustainable Development Goals

The thesis relates most directly to the following United Nations Sustainable Development Goals (SDGs) [81]:

- **SDG 3 (Good Health and Well-being)**, target 3.9: reducing tailpipe NO_x directly addresses illness and deaths from ambient air pollution.
- **SDG 9 (Industry, Innovation and Infrastructure)**, target 9.4: the work exemplifies upgrading existing industry for greater resource-use efficiency and cleaner technology through innovation.
- **SDG 11 (Sustainable Cities and Communities)**, target 11.6: lower vehicle emissions reduce the adverse per-capita environmental impact of cities, chiefly through improved air quality.
- **SDG 12 (Responsible Consumption and Production)**: more efficient powertrain operation makes more economical use of fuel per kilometre travelled.
- **SDG 13 (Climate Action)**: approaching fuel and emission reduction supports the broader decarbonisation effort.

The contribution to each goal is indirect and conditional on eventual deployment, but the intent and direction of the work are clearly aligned with their spirit.

Chapter 8

Conclusion

This thesis presented Reinforcement Learning (RL)-based strategies for minimising Nitrogen Oxides (NO_x) emissions in a Hybrid Electric Vehicle (HEV). This final chapter summarises the main aspects of the work and outlines promising directions for future research based on the identified limitations.

8.1 Summary

This thesis demonstrated that RL-based energy-management controllers can be trained in a surrogate HEV environment represented by a cascaded Long Short-Term Memory (LSTM) architecture. The proposed curriculum-inspired pipeline first established a basic speed-tracking capability in Phase 1 and subsequently refined the learned policies in Phase 2 by adding reward terms for State of Charge (SOC) sustainment and tailpipe NO_x reduction. This staged formulation progressively transformed a single-objective into a multi-objective control problem and produced two stable candidate controllers: Proximal Policy Optimisation (PPO) and Soft Actor-Critic (SAC). Twin Delayed Deep Deterministic Policy Gradient (TD3), by contrast, exhibits unstable behaviour due to the Phase 2 reward change and was therefore not considered a robust final candidate.

The Phase 1 agents were able to track the reference speed, but often did so with large SOC drift and high cumulative NO_x . Activating the multi-objective reward in Phase 2 significantly reduced both quantities while preserving speed-tracking quality. For the two stable agents, cumulative NO_x fell by roughly one order of magnitude and the SOC practically stayed constant. The operating-point analysis showed that the learned policies shifted engine operation away from high-load, high- NO_x regions of the measured map towards lower-load operating points, although this also moved parts of the operation away from the most efficient Brake-Specific Fuel Consumption (BSFC) region.

The zero-shot Worldwide Harmonised Light Vehicles Test Cycle (WLTC) comparison further proved the agent’s ability to generalise to unseen situations. The learned controllers tracked the realistic driving cycle more accurately than the rule-based reference controller, but they still consumed more fuel and did not yet provide a clear population-level environmental advantage. However, **SAC seed 7 reached the Euro 6 NO_x limit of 80 mg/km, improved speed tracking relative to the rule-based controller, and remained charge-sustaining**, but at a higher fuel cost.

8.2 Future Work

Promising directions for future research follow directly from the limitations discussed in section 7.2; these are therefore not repeated in full here. However, the limitations of this thesis do not capture all possible extensions, because the initial scope necessarily constrained the directions that could be explored. The following paragraphs highlight additional opportunities for extending the proposed framework.

Hybrid rule-based and learning-based control. A particularly promising direction is closer integration of expert knowledge into the learning-based controller. Combining a rule-based high-level supervisor with a learned low-level Deep Reinforcement Learning (DRL) controller could exploit the strengths of both approaches: the reliability, interpretability, and safety of hand-crafted rules, together with the ability of RL to optimise complex nonlinear control problems. The rule-based layer could enforce safety and component-protection constraints, while the learned policy would optimise local actuator decisions within this constrained operating space.

Algorithmic improvements. Although this work addressed the complex integration of HEV control and Artificial Intelligence (AI), the scope of the thesis allowed only a limited subset of possible algorithmic improvements to be explored. Future work could therefore investigate more advanced RL methods and training refinements to improve stability, performance and widen the scope. Examples could include dedicated learning-rate annealing and prioritised experience replay instead of a standard replay buffer. Most importantly, the underlying control problem could be treated more explicitly as a Partially Observable Markov Decision Process (POMDP) by using recurrent RL agents, which would allow the policy to model hidden internal states, such as the thermal state of the exhaust system, in order to exploit its emission-reduction capabilities.

A more holistic view. Although this point is already addressed in the limitations, it is important enough to emphasise again here: this thesis represents only a partial view of the broader control problem. Future research could use the proposed framework to extend the multi-objective reward design by including additional objectives, such as CO₂ emissions, direct fuel consumption, and component longevity.

8.3 Final Remarks

Overall, this thesis should be read as a proof of concept rather than as a deployable control solution. It shows that emission-aware RL-based HEV control is technically feasible in a surrogate environment, but it also highlights that environmental advantages must be assessed with care: reducing one pollutant in simulation is not equivalent to solving the broader sustainability challenge.

The main contribution is therefore methodological. The developed framework, training pipeline, and evaluation procedure provide a basis for future work within the Light-Duty CI-ICE 2027 (LDCI2027) project and beyond. With additional environmental objectives and validation in the real environment, hopefully, this work contributes to cleaner and more efficient sustainable mobility.

Appendix A

Neural Network Interfaces

This appendix provides an exhaustive overview of all inputs and outputs of the surrogate LSTMs provided by Frey et al. [5]. The tables are derived from the models' development documentation. Their layout has been adjusted, and the entry for $\dot{m}_{\text{NO}_x, \text{tp}}$ [g/s], Tailpipe NOx mass-flow rate, has been standardised to match the notation used throughout this thesis.

A.1 ICE - Inputs and Outputs

Group	Parameter	Description
Input	n_{ICE} [rpm]	Control input to all neural networks
Input	m_{fuel} [mg/cyl/cycle]	Fuel-mass control input
Input	T_{amb} [K]	Boundary input
Input	p_{amb} [bar]	Boundary input
Output	τ_{ICE} [Nm]	Transfer output to the drivetrain model
Output	$m_{fuel_{tot}}$ [g/s]	Total burned fuel mass
Output	$NO_{x_{eo}}$ [g/s]	Raw emission
Output	CO_{eo} [g/s]	Raw emission
Output	THC_{eo} [g/s]	Raw emission
Output	$T_{gas_{eo}}$ [K]	Engine-out gas temperature
Output	$\dot{m}_{\text{NO}_x, \text{tp}}$ [g/s]	Tailpipe NOx mass-flow rate
Output	CO_{tp} [g/s]	Tailpipe emission
Output	CO_{2tp} [g/s]	Tailpipe emission
Output	THC_{tp} [g/s]	Tailpipe emission
Output	$T_{wall_{SCR1}}$ [K]	SCR1 wall temperature
Output	$T_{wall_{DOC}}$ [K]	DOC wall temperature
Output	$T_{sub_{DPF}}$ [K]	DPF wall temperature
Output	$T_{wall_{SCR2}}$ [K]	SCR2 wall temperature
Output	$T_{wall_{SCR3}}$ [K]	SCR3 wall temperature
Output	$T_{gas_{tp}}$ [K]	Tailpipe gas temperature

Table A.1: Inputs and outputs of the ICE neural network.

A.2 Drivetrain - Inputs and Outputs

Group	Parameter	Description
Input	n_{ICE} [rpm]	Control input to all neural networks
Input	τ_{ICE} [Nm]	Transfer input from ICE
Input	τ_{EM2} [Nm]	Control input
Input	Brakemech [%]	Control input
Output	v_{Car} [km/h]	Vehicle speed
Output	SoC [-]	Battery state of charge

Table A.2: Inputs and outputs of the Drivetrain neural network.

A.3 Drivetrain Plus - Inputs and Outputs

Group	Parameter	Description
Input	n_{ICE} [rpm]	Control input to all neural networks
Input	τ_{ICE} [Nm]	Transfer input from ICE
Input	τ_{EM2} [Nm]	Control input
Input	Brakemech [%]	Control input
Output	$\tau_{EM1,out}$ [Nm]	Actual EM1 torque
Output	$\tau_{EM2,out}$ [Nm]	Actual EM2 torque
Output	n_{Sun} [rpm]	Actual sun speed
Output	n_{Ring} [rpm]	Actual ring speed

Table A.3: Inputs and outputs of the Drivetrain Plus neural network.

Appendix B

Phase 1 - Hyperparameter Optimisation Details

This appendix chapter lists the Phase 1 hyperparameter-optimisation search space, the Optuna trial histories, and the final best hyperparameter configuration for each algorithm. The corresponding Phase 2 reward-weight optimisation diagnostics are collected separately in Appendix C.

B.1 Phase 1 - Optuna Search Spaces

Hyperparameter	Range / choices
<i>Shared across all algorithms</i>	
net_arch	{[64, 64], [128, 128], [256, 256], [128, 128, 128]}
<i>PPO</i>	
learning_rate	log-uniform $[10^{-5}, 10^{-3}]$
n_steps	{1024, 2048, 4096}
batch_size	{64, 128, 256, 512}
n_epochs	int [3, 15]
gamma	uniform [0.95, 0.999]
gae_lambda	uniform [0.9, 1.0]
clip_range	uniform [0.1, 0.3]
<i>SAC</i>	
learning_rate	log-uniform $[10^{-5}, 10^{-3}]$
batch_size	{128, 256, 512, 1024}
tau	uniform $[10^{-3}, 2 \times 10^{-2}]$
gamma	uniform [0.95, 0.999]
train_freq	{1, 4, 8}
gradient_steps	{1, 4, 8}
learning_starts	{1000, 5000, 10000, 20000}
ent_coef	{auto, 0.01, 0.05, 0.1, 0.2}
use_sde	{True, False}
<i>TD3</i>	
learning_rate	log-uniform $[10^{-5}, 10^{-3}]$
batch_size	{128, 256, 512, 1024}
tau	uniform $[10^{-3}, 2 \times 10^{-2}]$
gamma	uniform [0.95, 0.999]
train_freq	{1, 4, 8}
gradient_steps	{1, 4, 8}
learning_starts	{1000, 5000, 10000, 20000}
policy_delay	{1, 2, 4}
target_policy_noise	uniform [0.1, 0.4]
target_noise_clip	uniform [0.3, 0.7]
action_noise_sigma	uniform [0.05, 0.3]

Table B.1: Phase-1 Optuna search spaces per algorithm. The shared **net_arch** discrete set is sampled once per trial and applied to both actor and critic networks.

B.2 Phase 1 - Optuna Trial Histories

The following plots document the Phase 1 Optuna studies used to select the hyperparameter configurations reported in Table B.2. They are included here as optimisation diagnostics; the main results chapter focuses on the seed-wise retraining and evaluation behaviour of the selected configurations.

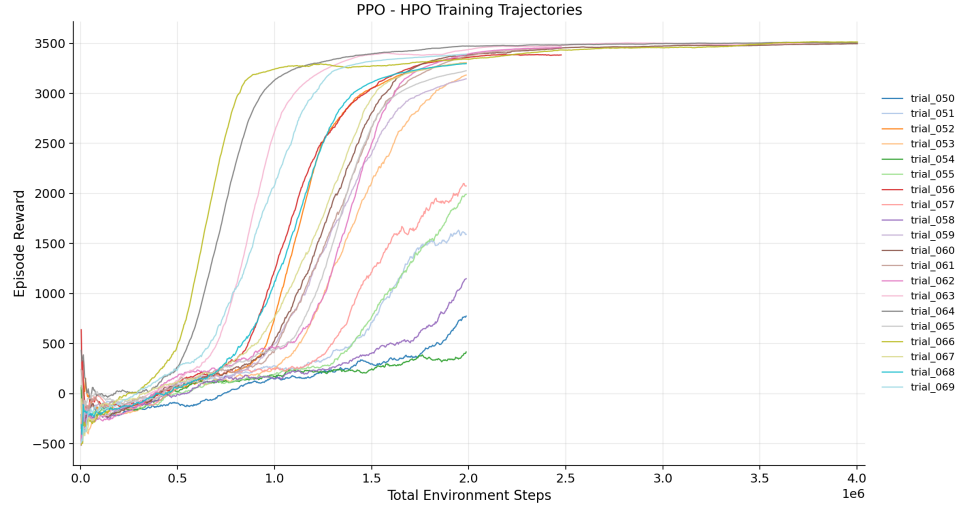


Figure B.1: Optuna trial results for PPO in Phase 1; curves are smoothed via moving average over 80 episodes.

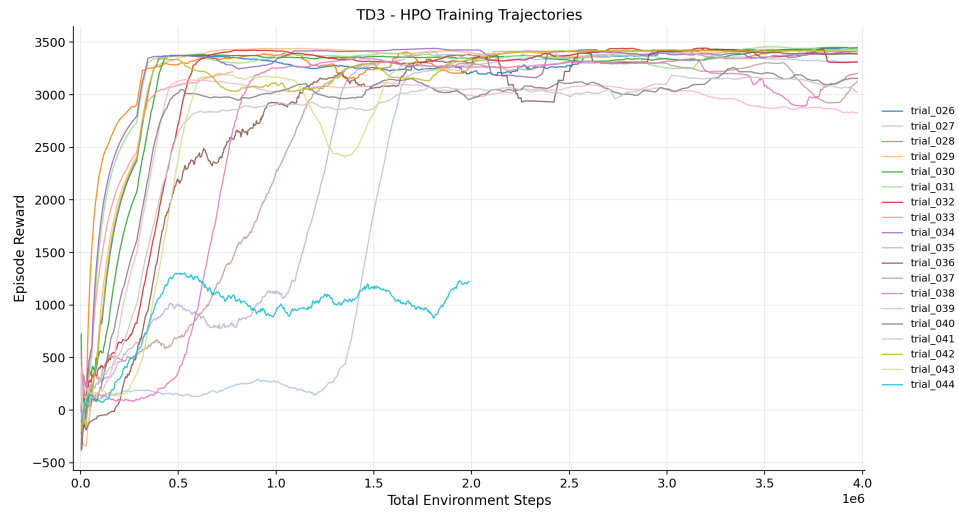


Figure B.2: Optuna trial results for TD3 in Phase 1; curves are smoothed via moving average over 80 episodes.

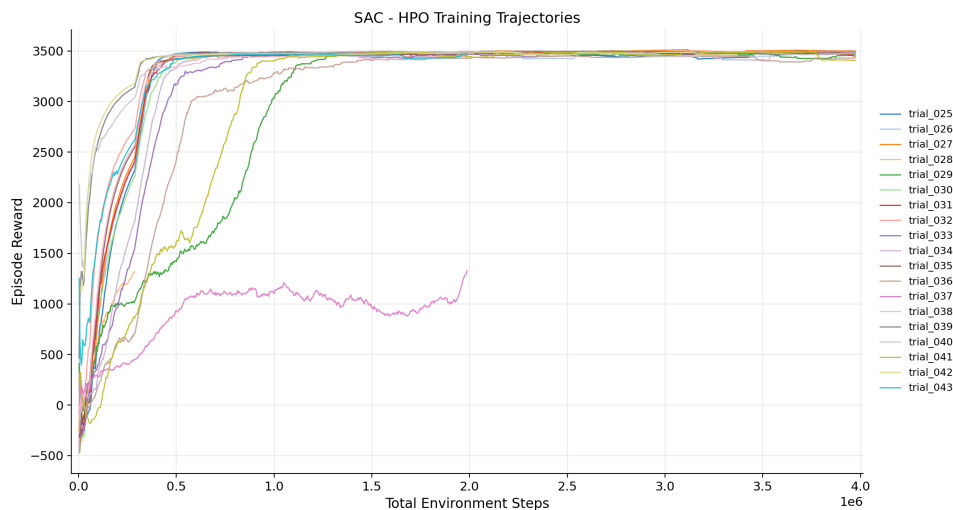


Figure B.3: Optuna trial results for SAC in Phase 1; curves are smoothed via moving average over 80 episodes.

B.3 Phase 1 - Best Hyperparameter Sets

Parameter	PPO	SAC	TD3
learning_rate	1.28×10^{-5}	9.20×10^{-4}	1.79×10^{-4}
gamma	0.9647	0.9601	0.9647
batch_size	64	512	1024
net_arch	[256, 256]	[128, 128, 128]	[128, 128]
n_steps	2048	—	—
train_freq	—	1	1
gradient_steps	—	8	8
n_epochs	11	—	—
gae_lambda	0.965	—	—
clip_range	0.203	—	—
tau	—	0.0080	0.0082
learning_starts	—	1,000	20,000
ent_coef	—	auto	—
use_sde	—	False	—
policy_delay	—	—	1
target_policy_noise	—	—	0.116
target_noise_clip	—	—	0.699
action_noise_sigma	—	—	0.144

Table B.2: Best hyperparameter configurations identified for Phase 1 across all evaluated Reinforcement Learning (RL) algorithms.

Appendix C

Phase 2 - Reward Weight Optimisation Details

This appendix chapter documents the Phase 2 reward-weight Optuna studies used to select the emission and quadratic State of Charge (SOC) weights reported in Table 6.3. The plots are included as additional diagnostics; the main results chapter reports the selected Phase 2 reward weights and trial results in Table 5.6 and Table 6.3, respectively.

C.1 Phase 2 - Reward-Weight Trial Histories

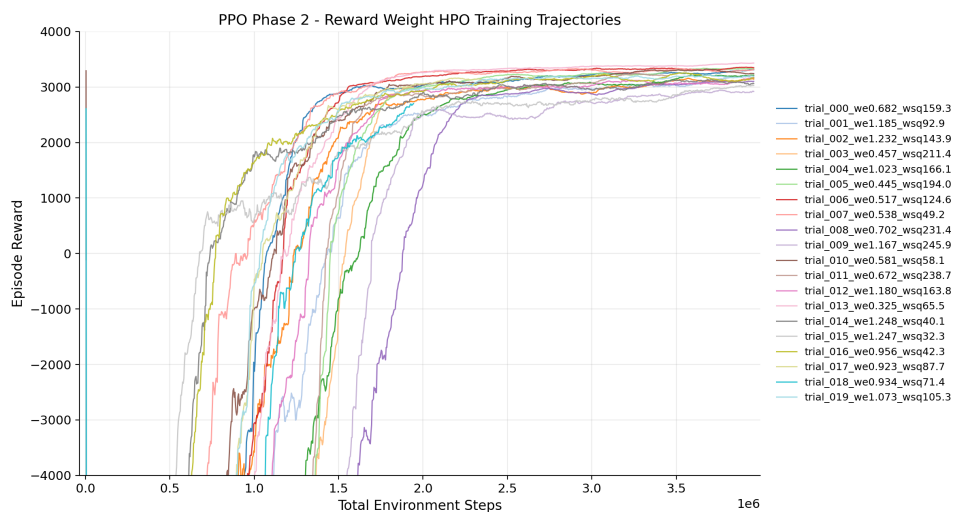


Figure C.1: Reward-weight Optuna trial results for PPO in Phase 2. The trials tune the active emission and quadratic SOC weights while keeping the Phase 1 speed-tracking configuration fixed; curves are smoothed via moving average over 80 episodes.

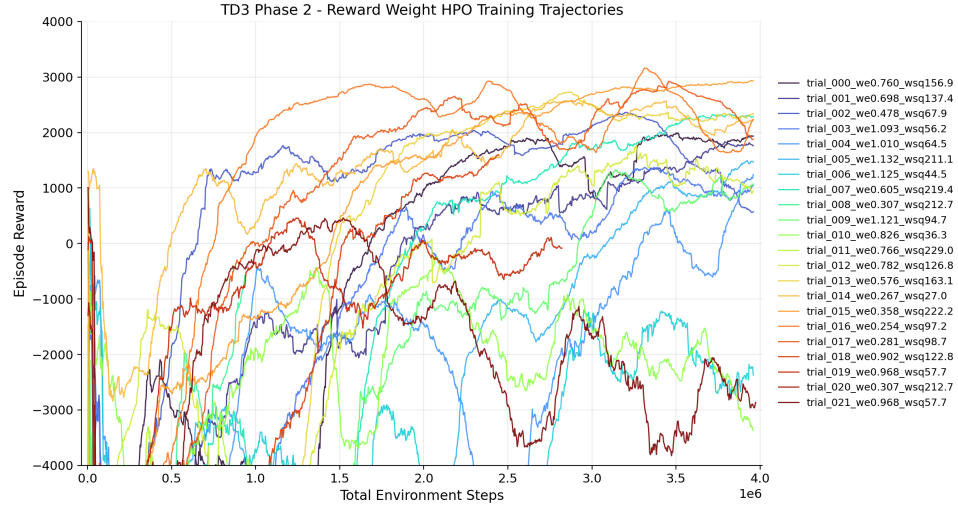


Figure C.2: Reward-weight Optuna trial results for TD3 in Phase 2. The trials tune the active emission and quadratic SOC weights while keeping the Phase 1 speed-tracking configuration fixed; curves are smoothed via moving average over 80 episodes.

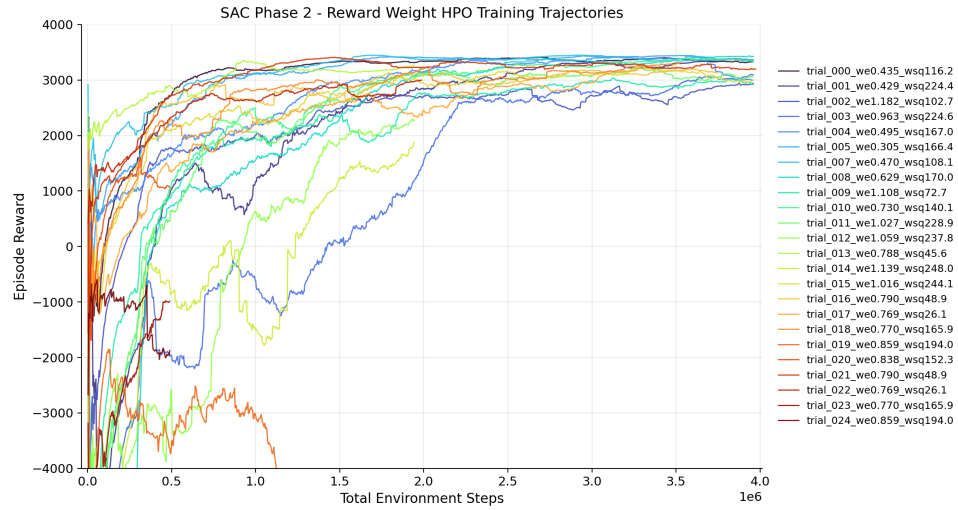


Figure C.3: Reward-weight Optuna trial results for SAC in Phase 2. The trials tune the active emission and quadratic SOC weights while keeping the Phase 1 speed-tracking configuration fixed; curves are smoothed via moving average over 80 episodes.

Bibliography

- [1] European Environment Agency, *Sustainability of Europe's mobility systems 2025*, eng. LU: Publications Office of the European Union, 2026. DOI: 10.2800/4890019 Accessed: Jun. 8, 2026. [Online]. Available: <https://data.europa.eu/doi/10.2800/4890019>
- [2] ICCT - International Council on Clean Transportation, *European Car Market Monitor: April 2026*, en-US. Accessed: Jun. 8, 2026. [Online]. Available: <https://theicct.org/publication/european-car-market-monitor-apr-2026/>
- [3] W. Enang and C. Bannister, "Modelling and control of hybrid electric vehicles (A comprehensive review)," en, *Renewable and Sustainable Energy Reviews*, vol. 74, pp. 1210–1239, Jul. 2017. DOI: 10.1016/j.rser.2017.01.075 Accessed: Apr. 13, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S1364032117300850>
- [4] A. García, J. Monsalve-Serrano, R. Lago Sari, and S. Tripathi, "Life cycle CO2 footprint reduction comparison of hybrid and electric buses for bus transit networks," en, *Applied Energy*, vol. 308, p. 118354, Feb. 2022. DOI: 10.1016/j.apenergy.2021.118354 Accessed: Jun. 13, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0306261921016007>
- [5] M. Frey, D. Itzen, Q. Yang, M. Grill, and A. C. Kulzer, "Data-Based Modeling of Power-Split Hybrids Using Cascaded Neural Networks," en, Stuttgart, Germany, Nov. 2025, pp. 2025–01–0522. DOI: 10.4271/2025-01-0522 Accessed: Apr. 7, 2026. [Online]. Available: <https://saemobilus.sae.org/papers/data-based-modeling-power-split-hybrids-using-cascaded-neural-networks-2025-01-0522>
- [6] H. Wang and X. Wang, "Energy Management of Intelligent FCHEVs Under Adaptive Cruise Control Based on Multi-Objective Deep Reinforcement Learning," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 5, pp. 7339–7350, May 2025. DOI: 10.1109/TVT.2025.3528420 Accessed: Mar. 7, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/10851447>
- [7] L. Serrao, S. Onori, and G. Rizzoni, "A Comparative Analysis of Energy Management Strategies for Hybrid Electric Vehicles," en, *Journal of Dynamic Systems, Measurement, and Control*, vol. 133, no. 3, p. 031012, May 2011. DOI: 10.1115/1.4003267 Accessed: Apr. 24, 2026. [Online]. Available: <https://asmedigitalcollection.asme.org/dynamicsystems/article/doi/10.1115/1.4003267/456399/A-Comparative-Analysis-of-Energy-Management>

- [8] B. Xu et al., “Parametric study on reinforcement learning optimized energy management strategy for a hybrid electric vehicle,” *Applied Energy*, vol. 259, p. 114200, Feb. 2020. DOI: 10.1016/j.apenergy.2019.114200 Accessed: Mar. 2, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306261919318872>
- [9] X. Tang, J. Chen, H. Pu, T. Liu, and A. Khajepour, “Double Deep Reinforcement Learning-Based Energy Management for a Parallel Hybrid Electric Vehicle With Engine Start-Stop Strategy,” en, *IEEE Transactions on Transportation Electrification*, vol. 8, no. 1, pp. 1376–1388, Mar. 2022. DOI: 10.1109/TTE.2021.3101470 Accessed: Jan. 16, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/9502729/>
- [10] J. Peng, H. Fan, H. He, and D. Pan, “A Rule-Based Energy Management Strategy for a Plug-in Hybrid School Bus Based on a Controller Area Network Bus,” en, *Energies*, vol. 8, no. 6, pp. 5122–5142, Jun. 2015. DOI: 10.3390/en8065122 Accessed: Jun. 13, 2026. [Online]. Available: <https://www.mdpi.com/1996-1073/8/6/5122>
- [11] Y. Lin et al., “Progress and summary of reinforcement learning on energy management of MPS-EV,” en, *Heliyon*, vol. 10, no. 1, e23014, Jan. 2024. DOI: 10.1016/j.heliyon.2023.e23014 Accessed: Jun. 13, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S2405844023102222>
- [12] J. Peng, H. He, and R. Xiong, “Rule based energy management strategy for a series-parallel plug-in hybrid electric bus optimized by dynamic programming,” *Applied Energy, Clean, Efficient and Affordable Energy for a Sustainable Future*, vol. 185, pp. 1633–1643, Jan. 2017. DOI: 10.1016/j.apenergy.2015.12.031 Accessed: Apr. 24, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0306261915016049>
- [13] G. Du, Y. Zou, X. Zhang, L. Guo, and N. Guo, “Heuristic Energy Management Strategy of Hybrid Electric Vehicle Based on Deep Reinforcement Learning With Accelerated Gradient Optimization,” *IEEE Transactions on Transportation Electrification*, vol. 7, no. 4, pp. 2194–2208, Dec. 2021. DOI: 10.1109/TTE.2021.3088853 Accessed: Apr. 24, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/9453762>
- [14] J. Zhou, S. Xue, Y. Xue, Y. Liao, J. Liu, and W. Zhao, “A novel energy management strategy of hybrid electric vehicle via an improved TD3 deep reinforcement learning,” en, *Energy*, vol. 224, p. 120118, Jun. 2021. DOI: 10.1016/j.energy.2021.120118 Accessed: Jan. 16, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0360544221003674>
- [15] R. S. Sutton and A. Barto, *Reinforcement learning: an introduction* (Adaptive computation and machine learning), eng, Second edition. Cambridge, Massachusetts London, England: The MIT Press, 2020.
- [16] H. Tan, H. Zhang, J. Peng, Z. Jiang, and Y. Wu, “Energy management of hybrid electric bus based on deep reinforcement learning in continuous state and action space,” *Energy Conversion and Management*, vol. 195, pp. 548–560, Sep. 2019. DOI: 10.1016/j.enconman.2019.05.038 Accessed: Apr. 27, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S019689041930593X>

- [17] L. Tresca, L. Pulvirenti, and L. Rolando, “A Cutting-Edge Energy Management System for a Hybrid Electric Vehicle relying on Soft Actor–Critic Deep Reinforcement Learning,” *Transportation Engineering*, vol. 19, p. 100308, Mar. 2025. DOI: 10.1016/j.treng.2025.100308 Accessed: Feb. 20, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2666691X25000089>
- [18] T. P. Lillicrap et al., *Continuous control with deep reinforcement learning*, arXiv:1509.02971 [cs.LG], Jul. 2019. DOI: 10.48550/arXiv.1509.02971 Accessed: Jun. 2, 2026. [Online]. Available: <http://arxiv.org/abs/1509.02971>
- [19] A. Biswas and A. Emadi, “Energy Management Systems for Electrified Powertrains: State-of-the-Art Review and Future Trends,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 7, pp. 6453–6467, Jul. 2019. DOI: 10.1109/TVT.2019.2914457 Accessed: Jun. 12, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/8704902>
- [20] Cornet - Collective Research Networking, *CORNET - Homepage*. Accessed: Jun. 13, 2026. [Online]. Available: https://www.cornet.online/EN/Home/Home_node.html
- [21] FVV e. V. - Forschungsvereinigung Verbrennungskraftmaschinen, *FVV: Home*, de-DE. Accessed: Jun. 13, 2026. [Online]. Available: <https://www.fvv-net.de/>
- [22] Universitat Politècnica de Catalunya, *UPC*, ca. Accessed: Jun. 13, 2026. [Online]. Available: <https://www.upc.edu/ca/homepage>
- [23] Institut für Fahrzeugtechnik Stuttgart, *Institut für Fahrzeugtechnik Stuttgart — Universität Stuttgart*, de, Jun. 2026. Accessed: Jun. 13, 2026. [Online]. Available: <https://www.ifs.uni-stuttgart.de/>
- [24] Technische Universität Berlin, *Technische Universität Berlin*, de, Jun. 2026. Accessed: Jun. 13, 2026. [Online]. Available: <https://www.tu.berlin/>
- [25] European Union, *Regulation (EU) 2024/1257 of the European Parliament and of the Council of 24 April 2024 on type-approval of motor vehicles and engines and of systems, components and separate technical units intended for such vehicles, with respect to their emissions and battery durability (Euro 7), amending Regulation (EU) 2018/858 of the European Parliament and of the Council and repealing Regulations (EC) No 715/2007 and (EC) No 595/2009 of the European Parliament and of the Council, Commission Regulation (EU) No 582/2011, Commission Regulation (EU) 2017/1151, Commission Regulation (EU) 2017/2400 and Commission Implementing Regulation (EU) 2022/1362 (Text with EEA relevance)*, en, Legislative Body: EP, CONSIL, Apr. 2024. Accessed: Jun. 8, 2026. [Online]. Available: <http://data.europa.eu/eli/reg/2024/1257/oj>
- [26] European Union, *Regulation (EC) No 715/2007 of the European Parliament and of the Council of 20 June 2007 on type approval of motor vehicles with respect to emissions from light passenger and commercial vehicles (Euro 5 and Euro 6) and on access to vehicle repair and maintenance information (Text with EEA relevance)*, en, Legislative Body: EP, CONSIL, Jun. 2007. Accessed: Jun. 8, 2026. [Online]. Available: <http://data.europa.eu/eli/reg/2007/715/oj>
- [27] F. J. Cuberos Muñoz, “Developing control strategies to minimize NOx emissions in a power-split hybrid electric vehicle,” eng, Jan. 2025. Accessed: May 12, 2026. [Online]. Available: <https://hdl.handle.net/2117/428856>

- [28] Universitat Politècnica de Catalunya, “Development of AI-Based Operation Strategy: Project Assessment 4, Report and Slides,” Facultat d’Informàtica de Barcelona (FIB), Universitat Politècnica de Catalunya BarcelonaTech, Project Report, Oct. 2025.
- [29] World Forum for Harmonisation of Vehicle Regulations (WP.29), “Mutual Resolution No. 2 (MR2) of the 1958 and the 1998 Agreements - Containing Vehicle Propulsion System Definitions,” United Nations Economic Commission for Europe (UNECE), Tech. Rep., Nov. 2015. [Online]. Available: <https://ec.europa.eu/docsroom/documents/31821/attachments/1/translations/en/renditions/pdf>
- [30] M. Kebriaei, A. H. Niasar, and B. Asaei, “Hybrid electric vehicles: An overview,” in *2015 International Conference on Connected Vehicles and Expo (ICCVE)*, ISSN: 2378-1297, Oct. 2015, pp. 299–305. DOI: 10.1109/ICCVE.2015.84 Accessed: Apr. 6, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/7447615/>
- [31] J. King and D. Nelson, “Model-Based Design of a Plug-In Hybrid Electric Vehicle Control Strategy,” en, Detroit, Michigan, United States, Apr. 2013, pp. 2013-01-1753. DOI: 10.4271/2013-01-1753 Accessed: Apr. 6, 2026. [Online]. Available: <https://saemobilus.sae.org/papers/model-based-design-a-plug-hybrid-electric-vehicle-control-strategy-2013-01-1753>
- [32] P. Hofmann, *Hybridfahrzeuge: Grundlagen, Komponenten, Fahrzeugbeispiele*, de. Berlin, Heidelberg: Springer Berlin Heidelberg, 2023. DOI: 10.1007/978-3-662-66894-8 Accessed: Jun. 10, 2026. [Online]. Available: <https://link.springer.com/10.1007/978-3-662-66894-8>
- [33] C. C. Chan, A. Bouscayrol, and K. Chen, “Electric, Hybrid, and Fuel-Cell Vehicles: Architectures and Modeling,” *IEEE Transactions on Vehicular Technology*, vol. 59, no. 2, pp. 589–598, Feb. 2010. DOI: 10.1109/TVT.2009.2033605 Accessed: Apr. 6, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/5276874>
- [34] X. Zhang, “Design of Power Split Hybrid Powertrains with Multiple Planetary Gears and Clutches,” Jan. 2015.
- [35] A. Casal Kulzer, M. Bargende, and D. Schmidt, “Grundlagen der Fahrzeugantriebe, Band 2: Vorlesungsmanuskript,” Edition: 3. Auflage, Stuttgart, 2023.
- [36] J. Zhou, Y. Xue, D. Xu, C. Li, and W. Zhao, “Self-learning energy management strategy for hybrid electric vehicle via curiosity-inspired asynchronous deep reinforcement learning,” en, *Energy*, vol. 242, p. 122548, Mar. 2022. DOI: 10.1016/j.energy.2021.122548 Accessed: Jan. 16, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0360544221027973>
- [37] S. Chakraborty, S. Adhikari, and R. Ganguli, “The role of surrogate models in the development of digital twins of dynamic systems,” en, *Applied Mathematical Modelling*, vol. 90, pp. 662–681, Feb. 2021. DOI: 10.1016/j.apm.2020.09.037 Accessed: Apr. 14, 2026. [Online]. Available: <https://linkinghub.elsevier.com/retrieve/pii/S0307904X20305588>
- [38] Gamma Technologies, *GT-SUITE - Gamma Technologies*. Accessed: Apr. 15, 2026. [Online]. Available: <https://www.gtisoft.com/gt-suite/>
- [39] I. Goodfellow, Y. Bengio, and A. Courville, *Deep Learning*. MIT Press, 2016.

- [40] S. Hochreiter and J. Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, vol. 9, no. 8, pp. 1735–1780, Nov. 1997. DOI: 10.1162/neco.1997.9.8.1735 Accessed: Apr. 20, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/6795963>
- [41] A. Zhang, Z. C. Lipton, M. Li, and A. J. Smola, *Dive into Deep Learning*. Cambridge University Press, 2023.
- [42] M. Hausknecht and P. Stone, *Deep Recurrent Q-Learning for Partially Observable MDPs*, arXiv:1507.06527 [cs], Jan. 2017. DOI: 10.48550/arXiv.1507.06527 Accessed: Mar. 4, 2026. [Online]. Available: <http://arxiv.org/abs/1507.06527>
- [43] T. Ni, B. Eysenbach, and R. Salakhutdinov, *Recurrent Model-Free RL Can Be a Strong Baseline for Many POMDPs*, en, arXiv:2110.05038 [cs], Jun. 2022. DOI: 10.48550/arXiv.2110.05038 Accessed: Mar. 22, 2026. [Online]. Available: <http://arxiv.org/abs/2110.05038>
- [44] A. G. Barto, R. S. Sutton, and C. W. Anderson, “Neuronlike adaptive elements that can solve difficult learning control problems,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. 13, pp. 834–846, Sep. 1983, ADS Bibcode: 1983ITSMC..13..834B. DOI: 10.1109/TSMC.1983.6313077 Accessed: Apr. 20, 2026. [Online]. Available: <https://ui.adsabs.harvard.edu/abs/1983ITSMC..13..834B>
- [45] OpenAI, *Part 2: Kinds of RL Algorithms — Spinning Up documentation*. Accessed: Jun. 14, 2026. [Online]. Available: https://spinningup.openai.com/en/latest/spinningup/rl_intro2.html
- [46] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, *Proximal Policy Optimization Algorithms*, Version Number: 2, 2017. DOI: 10.48550/ARXIV.1707.06347 Accessed: Jun. 1, 2026. [Online]. Available: <https://arxiv.org/abs/1707.06347>
- [47] L.-J. Lin, “Self-improving reactive agents based on reinforcement learning, planning and teaching,” en, *Machine Learning*, vol. 8, no. 3, pp. 293–321, May 1992. DOI: 10.1007/BF00992699 Accessed: May 6, 2026. [Online]. Available: <https://doi.org/10.1007/BF00992699>
- [48] R. S. Sutton and A. G. Barto, “Reinforcement Learning: An Introduction Second Edition: 2014, 2015,” 2014.
- [49] J. Schulman, S. Levine, P. Moritz, M. I. Jordan, and P. Abbeel, *Trust Region Policy Optimization*, Version Number: 5, 2015. DOI: 10.48550/ARXIV.1502.05477 Accessed: Jun. 2, 2026. [Online]. Available: <https://arxiv.org/abs/1502.05477>
- [50] OpenAI, *Proximal Policy Optimization — Spinning Up documentation*. Accessed: Jun. 1, 2026. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/ppo.html>
- [51] D. P. Kingma and J. Ba, *Adam: A Method for Stochastic Optimization*, arXiv:1412.6980 [cs.LG], Jan. 2017. DOI: 10.48550/arXiv.1412.6980 Accessed: Jun. 2, 2026. [Online]. Available: <http://arxiv.org/abs/1412.6980>
- [52] S. Fujimoto, H. van Hoof, and D. Meger, *Addressing Function Approximation Error in Actor-Critic Methods*, Version Number: 3, 2018. DOI: 10.48550/ARXIV.1802.09477 Accessed: Jun. 1, 2026. [Online]. Available: <https://arxiv.org/abs/1802.09477>
- [53] OpenAI, *Twin Delayed DDPG — Spinning Up documentation*. Accessed: Jun. 1, 2026. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/td3.html>

- [54] T. Haarnoja, A. Zhou, P. Abbeel, and S. Levine, *Soft Actor-Critic: Off-Policy Maximum Entropy Deep Reinforcement Learning with a Stochastic Actor*, Version Number: 2, 2018. DOI: 10.48550/ARXIV.1801.01290 Accessed: Jun. 1, 2026. [Online]. Available: <https://arxiv.org/abs/1801.01290>
- [55] OpenAI, *Soft Actor-Critic — Spinning Up documentation*. Accessed: Jun. 1, 2026. [Online]. Available: <https://spinningup.openai.com/en/latest/algorithms/sac.html>
- [56] E. Liang et al., *RLlib: Abstractions for Distributed Reinforcement Learning*, Version Number: 4, 2017. DOI: 10.48550/ARXIV.1712.09381 Accessed: Jun. 13, 2026. [Online]. Available: <https://arxiv.org/abs/1712.09381>
- [57] Z. Wang et al., “A comparative study of deep reinforcement learning based energy management strategy for hybrid electric vehicle,” *Energy Conversion and Management*, vol. 293, p. 117442, Oct. 2023. DOI: 10.1016/j.enconman.2023.117442 Accessed: Jun. 2, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S0196890423007884>
- [58] Z. E. Liu, Q. Zhou, Y. Li, and S. Shuai, “An Intelligent Energy Management Strategy for Hybrid Vehicle with irrational actions using Twin Delayed Deep Deterministic Policy Gradient,” *IFAC-PapersOnLine*, 6th IFAC Conference on Engine Powertrain Control, Simulation and Modeling E-COSM 2021, vol. 54, no. 10, pp. 546–551, Jan. 2021. DOI: 10.1016/j.ifacol.2021.10.219 Accessed: Jun. 2, 2026. [Online]. Available: <https://www.sciencedirect.com/science/article/pii/S2405896321016219>
- [59] Z. Liu, L. Xu, Y. Li, B. Shuai, and S. Shuai, *Deep Reinforcement Learning Based Energy Management Strategy for Hybrid Vehicles in Consideration of Engine Start-up Conditions*, Feb. 2022. DOI: 10.46855/energy-proceedings-9327 Accessed: Jun. 2, 2026. [Online]. Available: <https://www.energy-proceedings.org/?p=9327>
- [60] T. Liu, X. Hu, S. E. Li, and D. Cao, “Reinforcement Learning Optimized Look-Ahead Energy Management of a Parallel Hybrid Electric Vehicle,” *IEEE/ASME Transactions on Mechatronics*, vol. 22, no. 4, pp. 1497–1507, Aug. 2017. DOI: 10.1109/TMECH.2017.2707338 Accessed: Mar. 7, 2026. [Online]. Available: <https://ieeexplore.ieee.org/abstract/document/7932983>
- [61] Y. Bengio, J. Louradour, R. Collobert, and J. Weston, “Curriculum learning,” en, in *Proceedings of the 26th Annual International Conference on Machine Learning*, Montreal Quebec Canada: ACM, Jun. 2009, pp. 41–48. DOI: 10.1145/1553374.1553380 Accessed: Jun. 8, 2026. [Online]. Available: <https://dl.acm.org/doi/10.1145/1553374.1553380>
- [62] S. Ibrahim, M. Mostafa, A. Jnadi, H. Salloum, and P. Osinenko, *Comprehensive Overview of Reward Engineering and Shaping in Advancing Reinforcement Learning Applications*, arXiv:2408.10215 [cs.LG], Dec. 2024. DOI: 10.48550/arXiv.2408.10215 Accessed: Jun. 13, 2026. [Online]. Available: <http://arxiv.org/abs/2408.10215>
- [63] United Nations Economic Commission for Europe, “Global Technical Regulation No. 15: Worldwide Harmonized Light Vehicles Test Procedures (WLTP),” United Nations, Geneva, Switzerland, Global Technical Regulation ECE/TRANS/180/Add.15, 2014. [Online]. Available: <https://unece.org/>
- [64] M. Towers et al., “Gymnasium: A Standard Interface for Reinforcement Learning Environments,” *arXiv preprint arXiv:2407.17032*, 2024.

- [65] T. Hagenbucher, S. Milojevic, M. Grill, and A. C. Kulzer, “Initialization of LSTM Networks for Data-Driven Modeling Based on Real-Driving Data,” in *2023 28th International Conference on Automation and Computing (ICAC)*, Aug. 2023, pp. 1–6. DOI: 10.1109/ICAC57885.2023.10275195 Accessed: Apr. 20, 2026. [Online]. Available: <https://ieeexplore.ieee.org/document/10275195>
- [66] GeeksforGeeks, *Tf.function in TensorFlow*, en, Section: Data Science. Accessed: Jun. 13, 2026. [Online]. Available: <https://www.geeksforgeeks.org/data-science/tf-function-in-tensorflow/>
- [67] Centre Tecnològic de Transferència de Calor (CTTC-UPC), *Heat and Mass Transfer Technological Center — Universitat Politècnica de Catalunya*, 2026. [Online]. Available: <https://cttc.upc.es/>
- [68] M. Alshiekh, R. Bloem, R. Ehlers, B. Könighofer, S. Niekum, and U. Topcu, *Safe Reinforcement Learning via Shielding*, en, arXiv:1708.08611 [cs], Sep. 2017. DOI: 10.48550/arXiv.1708.08611 Accessed: Apr. 3, 2026. [Online]. Available: <http://arxiv.org/abs/1708.08611>
- [69] A. Raffin, A. Hill, A. Gleave, A. Kanervisto, M. Ernestus, and N. Dormann, “Stable-Baselines3: Reliable Reinforcement Learning Implementations,” *Journal of Machine Learning Research*, vol. 22, no. 268, pp. 1–8, 2021. [Online]. Available: <http://jmlr.org/papers/v22/20-1364.html>
- [70] A. Pan, K. Bhatia, and J. Steinhardt, *The Effects of Reward Misspecification: Mapping and Mitigating Misaligned Models*, Version Number: 2, 2022. DOI: 10.48550/ARXIV.2201.03544 Accessed: Jun. 1, 2026. [Online]. Available: <https://arxiv.org/abs/2201.03544>
- [71] S. Helfrich, A. Herzel, S. Ruzika, and C. Thielen, “Using scalarizations for the approximation of multiobjective optimization problems: Towards a general theory,” en, *Mathematical Methods of Operations Research*, vol. 100, no. 1, pp. 27–63, Aug. 2024. DOI: 10.1007/s00186-023-00823-2 Accessed: Jun. 12, 2026. [Online]. Available: <https://doi.org/10.1007/s00186-023-00823-2>
- [72] Y. Y. Haimes, L. S. Lasdon, and D. A. Wismer, “On a bicriterion formulation of the problems of integrated system identification and system optimization,” *IEEE Transactions on Systems, Man, and Cybernetics*, vol. SMC-1, no. 3, pp. 296–297, Jul. 1971. DOI: 10.1109/tsmc.1971.4308298
- [73] Core Writing Team, *Climate Change 2023: synthesis report*, eng, H. Lee and J. Romero, Eds. Geneva, Switzerland: Intergovernmental Panel on Climate Change, 2023, OCLC: 1391416974.
- [74] Universitat Politècnica de Catalunya, “Guia per incorporar l’anàlisi de sostenibilitat i implicacions ètiques en els TFE,” Vicerectorat de Política Acadèmica, Universitat Politècnica de Catalunya, Barcelona, Spain, Document informatiu, Jun. 2023. [Online]. Available: <https://govern.upc.edu>
- [75] Electricity Maps, *Live Electricity Climate Impact Map*, 2026. [Online]. Available: https://app.electricitymaps.com/map/live/fifteen_minutes
- [76] Statistisches Bundesamt, *Earnings by economic activity April 2024*, en. Accessed: Jun. 15, 2026. [Online]. Available: <https://www.destatis.de/EN/Themes/Labour/Earnings/Branch-Occupation/Tables/quarterly-earnings.html>

- [77] T. Akiba, S. Sano, T. Yanase, T. Ohta, and M. Koyama, “Optuna: A Next-Generation Hyperparameter Optimization Framework,” in *The 25th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, 2019, pp. 2623–2631.
- [78] Anthropic, *Claude*, en. Accessed: Jun. 12, 2026. [Online]. Available: <https://claude.com>
- [79] Universitat Politècnica de Catalunya, *Code of ethics of the Universitat Politècnica de Catalunya*, Published: Comitè d’Ètica de la UPC, 2022. Accessed: Jun. 12, 2026. [Online]. Available: <https://comite-etica.upc.edu/ca/documentacio/guia-i-codis/codi-etic-upc>
- [80] European Union, *Regulation (EU) 2016/679 of the European Parliament and of the Council of 27 April 2016 on the protection of natural persons with regard to the processing of personal data and on the free movement of such data, and repealing Directive 95/46/EC (General Data Protection Regulation) (Text with EEA relevance)*, en, Legislative Body: OP_DATPRO, May 2016. Accessed: Jun. 18, 2026. [Online]. Available: <http://data.europa.eu/eli/reg/2016/679/2016-05-04>
- [81] United Nations, *THE 17 GOALS — Sustainable Development*. Accessed: Jun. 12, 2026. [Online]. Available: <https://sdgs.un.org/goals>